



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Specifica Tecnica



Stato:	Completato
Versione:	1.0.0
Responsabile:	Gabriele Scaggiante
Redattori:	Gabriele Scaggiante
	Dennis Parolin
	Ferdinando Fracasso
	Giovanni Visentin
	Marco Sanguin
	Riccardo Manisi
Verificatori:	Riccardo Manisi
	Dennis Parolin
	Marco Sanguin
	Giovanni Visentin
	Gabriele Scaggiante
	Ferdinando Fracasso
Destinatari:	Miriade srl
	Prof. Tullio Vardanega
	Prof. Riccardo Cardin
	Gruppo BitByBit



Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
1.0.0	2026-05-13	Giovanni Visentin	Approvazione del documento	Gabriele Scaggianti
0.10.0	2026-05-04	Marco Sanguin	Aggiornamento dell'Architettura frontend con corrispettivi UML	Riccardo Manisi
0.9.0	2026-05-02	Riccardo Manisi	Correzioni a sezioni Architettura logica, e di Deployment	Gabriele Scaggianti
0.8.0	2026-05-02	Giovanni Visentin	Sistemazione della tabella dei requisiti	Dennis Parolin
0.7.1	2026-05-01	Ferdinando Fracasso	Aggiornato sezione Librerie	Riccardo Manisi
0.7.0	2026-04-29	Riccardo Manisi	Redatte le sezioni Architettura classi microservizio contatti fidati , Architettura classi microservizio luoghi sicuri	Gabriele Scaggianti
0.6.1	2026-04-27	Gabriele Scaggianti	Aggiunte librerie nella sezione Librerie	Ferdinando Fracasso
0.6.0	2026-04-26	Gabriele Scaggianti	Completata la sezione Architettura chatbot introducendo la sezione Architettura logica chatbot e aggiornando la sezione dedicata alle API	Dennis Parolin
0.5.3	2026-04-24	Ferdinando Fracasso	Aggiornato la sezione Architettura funzionalità diari	Gabriele Scaggianti



0.5.2	2026-04-23	Gabriele Scaggianti	Corretta la sezione Architettura chatbot dopo l'introduzione di due tabelle DynamoDB	Giovanni Visentin
0.5.1	2026-04-20	Dennis Parolin	Aggiornati i pattern presenti nella la sezione Design pattern utilizzati, aggiunti i pattern Dependency Injection , Command e Adapter .	Gabriele Scaggianti
0.5.0	2026-04-18	Gabriele Scaggianti	Nella sezione Architettura backend , aggiornata Architettura Auth, Luoghi Sicuri e Materiale Informativo; redatte sezioni Diario, Chatbot e DMS.	Dennis Parolin
0.4.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura invio SOS	Gabriele Scaggianti
0.3.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura auth	Ferdinando Fracasso
0.2.1	2026-04-12	Marco Sanguin	Redatta la sezione Architettura luoghi sicuri	Riccardo Manisi
0.2.0	2026-04-12	Marco Sanguin	Redatta la sezione Architettura funzionalità Dead man's switch	Ferdinando Fracasso



0.1.9	2026-04-12	Dennis Parolin	Creata la sezione Architettura Back End con relative sottosezioni Autenticazione , Luoghi sicuri e Materiale informativo	Giovanni Visentin
0.1.8	2026-04-11	Marco Sanguin	Aggiunta la sezione Stato dei requisiti funzionali	Ferdinando Fracasso
0.1.7	2026-04-10	Marco Sanguin	Redatta la sezione Architettura Materiale Informativo	Ferdinando Fracasso
0.1.6	2026-04-09	Ferdinando Fracasso	Redatta la sezione Design pattern utilizzati	Giovanni Visentin
0.1.5	2026-04-06	Ferdinando Fracasso	Redatta la sezione Architettura funzionalità diari	Marco Sanguin
0.1.4	2026-04-05	Giovanni Visentin	Aggiunta della funzionalità dei contatti fidati	Dennis Parolin
0.1.3	2026-04-04	Gabriele Scaggiante	Iniziata la sezione Architettura frontend , completata Architettura chatbot	Marco Sanguin
0.1.2	2026-04-02	Dennis Parolin	Redatta la sezione Architettura logica e Architettura di deployment	Marco Sanguin
0.1.1	2026-03-31	Gabriele Scaggiante	Aggiunta di EventBridge, SAM, CloudFormation e Docker nella sezione tecnologie.	Dennis Parolin
0.1.0	2026-03-30	Gabriele Scaggiante	Creazione del documento. Scrittura dell'introduzione e delle tecnologie utilizzate.	Riccardo Manisi





Indice

1	Introduzione	22
1.1	Scopo del documento	22
1.2	Riferimenti	22
1.2.1	Riferimenti normativi	22
1.2.2	Riferimenti informativi	23
2	Tecnologie	23
2.1	Framework	24
2.1.1	Flutter	24
2.2	Linguaggi di programmazione	24
2.2.1	Dart	24
2.2.2	Python	24
2.3	Servizi AWS	25
2.3.1	Lambda	25
2.3.2	API Gateway	25
2.3.3	DynamoDB	25
2.3.4	S3	25
2.3.5	Cognito	26
2.3.6	Bedrock	26
2.3.7	SES	26
2.3.8	CloudWatch	26
2.3.9	EventBridge	26
2.3.10	SAM	27
2.3.11	CloudFormation	27
2.4	Librerie	27
2.4.1	http	27
2.4.2	image_picker	27
2.4.3	path_provider	28
2.4.4	provider	28
2.4.5	url_launcher	28
2.4.6	flutter_dotenv	28
2.4.7	flutter_web_auth_2	29
2.4.8	flutter_map	29
2.4.9	geolocator	29
2.4.10	get_it	29
2.4.11	latlong2	30
2.4.12	path	30
2.4.13	flutter_secure_storage	30



2.4.14	just_audio	30
2.4.15	file_picker	30
2.4.16	url_launcher_platform_interface	31
2.4.17	command_it	31
2.4.18	listen_it	31
2.4.19	mocktail	31
2.4.20	shared_preferences	31
2.4.21	connectivity_plus	32
2.4.22	audio_session	32
2.4.23	python-ulid	32
2.4.24	rank-bm25	32
2.4.25	boto3	33
2.5	Tecnologie di analisi statica	33
2.5.1	SonarQube	33
2.6	Tecnologie di analisi dinamica	33
2.6.1	Pytest	33
2.6.2	Moto	33
2.7	Altro	34
2.7.1	Ollama	34
2.7.2	Docker	34
3	Architettura	34
3.1	Architettura logica	34
3.1.1	Frontend Mobile	34
3.1.2	Backend con architettura esagonale	35
3.2	Design pattern utilizzati	36
3.2.1	Model-view-viewmodel	36
3.2.1.1	Descrizione del pattern:	37
3.2.1.2	Ragioni per l'utilizzo:	37
3.2.1.3	Riferimenti nel progetto:	37
3.2.2	Command	38
3.2.2.1	Descrizione del pattern:	38
3.2.2.2	Ragioni per l'utilizzo:	38
3.2.2.3	Riferimenti nel progetto:	39
3.2.3	Observer	39
3.2.3.1	Descrizione del pattern:	39
3.2.3.2	Ragioni per l'utilizzo:	40
3.2.3.3	Riferimenti nel progetto:	40
3.2.4	Proxy	41
3.2.4.1	Descrizione del pattern:	41
3.2.4.2	Ragioni per l'utilizzo:	41



3.2.4.3	Riferimenti nel progetto:	41
3.2.5	Singleton	42
3.2.5.1	Descrizione del pattern:	42
3.2.5.2	Ragioni per l'utilizzo:	42
3.2.5.3	Riferimenti nel progetto:	42
3.2.6	Dependency Injection e Service Locator	43
3.2.6.1	Descrizione del pattern:	43
3.2.6.2	Ragioni per l'utilizzo:	43
3.2.6.3	Riferimenti nel progetto:	44
3.2.7	Adapter	44
3.2.7.1	Descrizione del pattern:	44
3.2.7.2	Ragioni per l'utilizzo:	44
3.2.7.3	Riferimenti nel progetto:	45
3.3	Architettura di deployment	45
3.3.1	Architettura a microservizi	45
3.4	Infrastruttura	45
3.4.1	Infrastructure as Code (IaC)	47
3.5	Architettura Front End	47
3.5.1	Autenticazione	47
3.5.1.1	LoginScreen	48
3.5.1.2	User	48
3.5.1.3	UserDTO	49
3.5.1.4	AuthViewModel	49
3.5.1.5	AuthRepository	50
3.5.1.6	AuthService	50
3.5.2	Chatbot	51
3.5.2.1	ChatbotScreen	52
3.5.2.2	ChatWidget	52
3.5.2.3	ChatHistoryWidget	53
3.5.2.4	ChatbotModeToggleWidget	53
3.5.2.5	ChatbotSendMessageWidget	53
3.5.2.6	ChatbotCreateChatWidget	54
3.5.2.7	ChatbotViewModel	54
3.5.2.8	Chat	54
3.5.2.9	ProxyChat	55
3.5.2.10	LocalChat	55
3.5.2.11	ChatMessage	56
3.5.2.12	ChatMode	56
3.5.2.13	MessageType	57
3.5.2.14	MessageResponse	57



3.5.2.15	ChatbotRepository	57
3.5.2.16	ChatbotService	58
3.5.2.17	ApiClient	58
3.5.2.18	HttpApiClient	58
3.5.2.19	ChatDTO	59
3.5.3	Diari	59
3.5.3.1	DiaryAccessScreen	60
3.5.3.2	PasswordFormWidget	60
3.5.3.3	DiaryAccessViewModel	60
3.5.3.4	DiaryAccountRepository	61
3.5.3.5	DiaryAccountService	61
3.5.3.6	DiaryAccessResult	61
3.5.3.7	DiaryScreen	62
3.5.3.8	NoteListWidget	62
3.5.3.9	NoteActionsWidget	62
3.5.3.10	NoteEditorWidget	62
3.5.3.11	DiaryViewModel	63
3.5.3.12	DiarySession	63
3.5.3.13	DiaryType	64
3.5.3.14	Note	64
3.5.3.15	ProxyNote	65
3.5.3.16	LocalNote	65
3.5.3.17	NoteElement	66
3.5.3.18	NoteTextElement	66
3.5.3.19	NoteImageElement	67
3.5.3.20	NoteAudioElement	67
3.5.3.21	NoteRepository	67
3.5.3.22	NoteService	68
3.5.3.23	NoteDTO	68
3.5.3.24	NoteElementDTO	69
3.5.4	Contatti Fidati	69
3.5.4.1	TrustedContactScreen	70
3.5.4.2	TrustedContactListWidget	70
3.5.4.3	TrustedContactFormWidget	71
3.5.4.4	TrustedContactActionsWidget	71
3.5.4.5	TrustedContactViewModel	71
3.5.4.6	TrustedContact	72
3.5.4.7	TrustedContactRepository	72
3.5.4.8	TrustedContactService	73
3.5.4.9	TrustedContactDTO	73



3.5.5	Luoghi Sicuri	73
3.5.5.1	SafePlaceMapScreen	74
3.5.5.2	SafePlaceMapWidget	74
3.5.5.3	SafePlaceViewModel	75
3.5.5.4	SafePlaceCategory	76
3.5.5.5	SafePlaceRepository	76
3.5.5.6	LocationService	77
3.5.5.7	SafePlaceService	77
3.5.5.8	SafePlaceDTO	77
3.5.6	Dead Man's Switch	78
3.5.6.1	SettingsScreen	78
3.5.6.2	DeadManFormWidget	79
3.5.6.3	DeadManViewModel	79
3.5.6.4	DeadManSettings	80
3.5.6.5	DeadManRepository	80
3.5.6.6	DeadManService	81
3.5.6.7	DeadManSettingsDTO	81
3.5.7	Materiale Informativo	82
3.5.7.1	MaterialScreen	82
3.5.7.2	MaterialListWidget	82
3.5.7.3	MaterialViewModel	83
3.5.7.4	Resource	83
3.5.7.5	ResourceType	84
3.5.7.6	MaterialRepository	84
3.5.7.7	MaterialService	85
3.5.7.8	ResourceDTO	85
3.5.8	Main App, Home e Sistema SOS	85
3.5.8.1	MainAppWidget	86
3.5.8.2	HomeScreen	86
3.5.8.3	HomeTabView	87
3.5.8.4	HomeDashboardWidget	87
3.5.8.5	HomeViewModel	87
3.5.8.6	SosPullTopWidget	88
3.5.8.7	SosViewModel	88
3.6	Architettura Back End	89
3.6.1	Autenticazione	89
3.6.1.1	Architettura e Flusso di Autenticazione: Modulo Federated Login	89
3.6.1.1.1	Inizializzazione del Flusso e Autenticazione Federata	89



3.6.1.1.2	Emissione dei Token di Sessione	90
3.6.1.1.3	Validazione Perimetrale e Instradamento Sicuro	90
3.6.2	Luoghi sicuri	90
3.6.2.1	Architettura e Flusso di Elaborazione: Modulo Luoghi Sicuri	91
3.6.2.1.1	Delega della Logica Geospaziale al Livello Client	91
3.6.2.2	API associate	91
3.6.2.3	Architettura logica: Modulo Luoghi Sicuri	93
3.6.2.3.1	Marker	93
3.6.2.3.2	SafePlacesController	94
3.6.2.3.3	GetMarkerPort	94
3.6.2.3.4	SafePlacesService	95
3.6.2.3.5	SafePlacesRepositoryPort	95
3.6.2.3.6	S3SafePlacesRepository	95
3.6.2.3.7	Materiale informativo	96
3.6.2.4	Architettura e Flusso di Elaborazione: Modulo Materiale Informativo	96
3.6.2.5	API associate	96
3.6.2.6	Architettura logica: Modulo Materiale Informativo	98
3.6.2.6.1	Resource	99
3.6.2.6.2	ResourcesController	100
3.6.2.6.3	GetResourcesPort	100
3.6.2.6.4	ResourcesService	100
3.6.2.6.5	ResourcesRepositoryPort	101
3.6.2.6.6	S3ResourcesRepository	101
3.6.3	Diario criptato e fittizio	102
3.6.3.1	Architettura e Flusso di Elaborazione: diario criptato e fittizio	102
3.6.3.2	API associate	104
3.6.3.3	Architettura logica: microservizio diario	111
3.6.3.3.1	DiaryType	112
3.6.3.3.2	Note	112
3.6.3.3.3	NoteElement	113
3.6.3.3.4	NoteElementDTO	113
3.6.3.3.5	NoteDTO	114
3.6.3.3.6	GetNoteCmd	114
3.6.3.3.7	GetNotesCmd	115
3.6.3.3.8	AddElementNewNoteCmd	115
3.6.3.3.9	AddNoteCmd	116
3.6.3.3.10	AddNoteElementCmd	116
3.6.3.3.11	DeleteNoteCmd	117



3.6.3.3.12	DeleteNoteElementCmd	117
3.6.3.3.13	ValidatePasswordCmd	118
3.6.3.3.14	ValidateTokenCmd	118
3.6.3.3.15	SetPasswordCmd	119
3.6.3.3.16	CheckPasswordCmd	119
3.6.3.3.17	GetNotePort	120
3.6.3.3.18	SetNotePort	120
3.6.3.3.19	DeleteNotePort	120
3.6.3.3.20	SetNoteElementPort	121
3.6.3.3.21	DeleteNoteElementPort	121
3.6.3.3.22	SetPasswordPort	121
3.6.3.3.23	ValidationPort	122
3.6.3.3.24	CheckPasswordPort	122
3.6.3.3.25	NoteRepositoryPort	123
3.6.3.3.26	FileRepositoryPort	123
3.6.3.3.27	DiaryAuthRepositoryPort	124
3.6.3.3.28	NoteService	124
3.6.3.3.29	DiaryAuthService	125
3.6.3.3.30	DynamoNoteAdapter	125
3.6.3.3.31	S3NoteAdapter	126
3.6.3.3.32	DynamoAuthAdapter	126
3.6.3.3.33	DiaryNoteController	127
3.6.3.3.34	DiaryAccessController	128
3.6.4	Chatbot	128
3.6.4.1	Architettura e Flusso di Elaborazione: chatbot (detective delle relazioni e specchio intelligente)	129
3.6.4.2	API associate	130
3.6.4.3	Architettura logica del chatbot	135
3.6.4.3.1	Chat	136
3.6.4.3.2	ChatMessage	136
3.6.4.3.3	ChatbotLLMPort	137
3.6.4.3.4	BedrockLLMAdapter	137
3.6.4.3.5	ChatbotLLMService	137
3.6.4.3.6	ChatsRepositoryPort	138
3.6.4.3.7	DynamoChatAdapter	138
3.6.4.3.8	CreateChatPort	139
3.6.4.3.9	GetChatPort	139
3.6.4.3.10	UpdateChatPort	140
3.6.4.3.11	DeleteChatPort	140
3.6.4.3.12	ChatMessagePort	140



3.6.4.3.13	ChatbotCRUDService	141
3.6.4.3.14	CreateChatCmd	141
3.6.4.3.15	GetChatCmd	142
3.6.4.3.16	GetChatListCmd	142
3.6.4.3.17	UpdateChatCmd	142
3.6.4.3.18	DeleteChatCmd	143
3.6.4.3.19	AddChatMessageCmd	143
3.6.4.3.20	ChatDTO	144
3.6.4.3.21	ChatMessageDTO	144
3.6.4.3.22	ChatBotController	145
3.6.5	Contatti fidati, invio SOS e Dead man's switch	145
3.6.5.1	Architettura e Flusso di Elaborazione: contatti fidati, SOS e Dead man's switch	146
3.6.5.2	API associate	147
3.6.5.3	Architettura logica: Modulo contatti fidati, SOS e Dead man's switch	153
3.6.5.3.1	TrustedContact	154
3.6.5.3.2	TrustedContactDTO	154
3.6.5.3.3	AddTrustedContactCmd	155
3.6.5.3.4	GetTrustedContactCmd	155
3.6.5.3.5	DeleteTrustedContactCmd	155
3.6.5.3.6	DmsConfigurationSettings	156
3.6.5.3.7	DmsConfigurationSettingsDTO	156
3.6.5.3.8	AlertCmd	157
3.6.5.3.9	EmailMessage	157
3.6.5.3.10	TrustedContactController	158
3.6.5.3.11	SetDmsHeartBeatPort	158
3.6.5.3.12	GetDmsConfigurationSettingsPort	159
3.6.5.3.13	SetDmsConfigurationSettingsPort	159
3.6.5.3.14	UpdateDmsAlertPort	159
3.6.5.3.15	SetTrustedContactPort	160
3.6.5.3.16	GetTrustedContactPort	160
3.6.5.3.17	DeleteTrustedContactPort	160
3.6.5.3.18	SendAlertPort	161
3.6.5.3.19	DmsCRUDService	161
3.6.5.3.20	DmsAlertService	161
3.6.5.3.21	TrustedContactCRUDService	162
3.6.5.3.22	SOSAlertService	162
3.6.5.3.23	DmsRepositoryPort	163
3.6.5.3.24	TrustedContactRepositoryPort	164

3.6.5.3.25	NotificationPort	164
3.6.5.3.26	DynamoDmsAdapter	165
3.6.5.3.27	DynamoTrustedContactAdapter	165
3.6.5.3.28	SesNotificationAdapter	165
4	Stato dei requisiti funzionali	166
4.1	Requisiti Funzionali obbligatori	166
4.2	Requisiti Funzionali desiderabili	178
4.3	Requisiti Funzionali opzionali	179



Elenco delle tabelle

2	Tabella dello stato dei requisiti funzionali obbligatori	178
3	Tabella Requisiti Funzionali Desiderabili	179
4	Tabella dello stato dei requisiti funzionali opzionali	180

Elenco delle figure

1	Rappresentazione del pattern MVVM nel modulo del materiale informativo	36
2	Rappresentazione del pattern Command	38
3	Rappresentazione del pattern Observer applicato nel modulo dell'autenticazione	39
4	Rappresentazione del pattern Proxy applicato alla gestione delle chat . . .	41
5	Rappresentazione del pattern Singleton applicato alla sessione del diario . .	42
6	Rappresentazione del pattern Dependency Injection nel modulo di Autenticazione	43
7	Rappresentazione del pattern Adapter all'ApiClient	44
8	Diagramma delle classi relativo all'autenticazione	48
9	Diagramma della classe LoginScreen	48
10	Diagramma della classe User	49
11	Diagramma della classe UserDTO	49
12	Diagramma della classe AuthViewModel	50
13	Diagramma della classe AuthRepository	50
14	Diagramma della classe AuthService	51
15	Diagramma delle classi relativo alle funzionalità del Chatbot	52
16	Diagramma della classe ChatbotScreen	52
17	Diagramma della classe ChatWidget	53
18	Diagramma della classe ChatHistoryWidget	53
19	Diagramma della classe ChatbotModeToggleWidget	53
20	Diagramma della classe ChatbotSendMessageWidget	54
21	Diagramma della classe ChatbotCreateChatWidget	54
22	Diagramma della classe ChatbotViewModel	54
23	Diagramma della classe Chat	55
24	Diagramma della classe ProxyChat	55
25	Diagramma della classe LocalChat	56
26	Diagramma della classe ChatMessage	56
27	Diagramma della classe ChatMode	57
28	Diagramma della classe MessageResponse	57
29	Diagramma della classe ChatbotRepository	57
30	Diagramma della classe ChatbotService	58



31	Diagramma dell'interfaccia ApiClient	58
32	Diagramma della classe HttpClient	58
33	Diagramma della classe ChatDTO	59
34	Diagramma delle classi relativo alle funzionalità dei Diari	59
35	Diagramma della classe DiaryAccessScreen	60
36	Diagramma della classe PasswordFormWidget	60
37	Diagramma della classe DiaryAccessViewModel	60
38	Diagramma della classe DiaryAccountRepository	61
39	Diagramma della classe DiaryAccountService	61
40	Diagramma della classe DiaryAccessResult	61
41	Diagramma della classe DiaryScreen	62
42	Diagramma della classe NoteListWidget	62
43	Diagramma della classe NoteActionsWidget	62
44	Diagramma della classe NoteEditorWidget	63
45	Diagramma della classe DiaryViewModel	63
46	Diagramma della classe DiarySession	64
47	Diagramma della classe DiaryType	64
48	Diagramma della classe Note	65
49	Diagramma della classe ProxyNote	65
50	Diagramma della classe LocalNote	66
51	Diagramma della classe NoteElement	66
52	Diagramma della classe NoteTextElement	66
53	Diagramma della classe NoteImageElement	67
54	Diagramma della classe NoteAudioElement	67
55	Diagramma della classe NoteRepository	68
56	Diagramma della classe NoteService	68
57	Diagramma della classe NoteDTO	69
58	Diagramma della classe NoteElementDTO	69
59	Diagramma delle classi relativo alle funzionalità dei Contatti Fidati	70
60	Diagramma della classe TrustedContactScreen	70
61	Diagramma della classe TrustedContactListWidget	70
62	Diagramma della classe TrustedContactFormWidget	71
63	Diagramma della classe TrustedContactActionsWidget	71
64	Diagramma della classe TrustedContactViewModel	72
65	Diagramma della classe TrustedContact	72
66	Diagramma della classe TrustedContactRepository	73
67	Diagramma della classe TrustedContactService	73
68	Diagramma della classe TrustedContactDTO	73
69	Diagramma delle classi complessivo della funzionalità Luoghi Sicuri	74
70	Diagramma della classe SafePlaceMapScreen	74



71	Diagramma della classe SafePlaceMapWidget	75
72	Diagramma della classe SafePlaceViewModel	75
73	Diagramma della classe SafePlace	76
74	Diagramma della classe SafePlaceCategory	76
75	Diagramma della classe SafePlaceRepository	77
76	Diagramma della classe LocationService	77
77	Diagramma della classe SafePlaceService	77
78	Diagramma della classe SafePlaceDTO	78
79	Diagramma delle classi relativo alla funzionalità del Dead Man's Switch . .	78
80	Diagramma della classe SettingsScreen	79
81	Diagramma della classe DeadManFormWidget	79
82	Diagramma della classe DeadManViewModel	80
83	Diagramma della classe DeadManSettings	80
84	Diagramma della classe DeadManRepository	81
85	Diagramma della classe DeadManService	81
86	Diagramma della classe DeadManSettingsDTO	81
87	Diagramma delle classi complessivo della funzionalità Materiale Informativo	82
88	Diagramma della classe MaterialScreen	82
89	Diagramma della classe MaterialListWidget	83
90	Diagramma della classe MaterialViewModel	83
91	Diagramma della classe Resource	84
92	Diagramma della classe ResourceType	84
93	Diagramma della classe MaterialRepository	84
94	Diagramma della classe MaterialService	85
95	Diagramma della classe ResourceDTO	85
96	Diagramma delle classi complessivo di Main App, Home e SOS	86
97	Diagramma della classe MainAppWidget	86
98	Diagramma della classe HomeScreen	87
99	Diagramma della classe HomeTabView	87
100	Diagramma della classe HomeDashboardWidget	87
101	Diagramma della classe HomeViewModel	88
102	Diagramma della classe SosPullTopWidget	88
103	Diagramma della classe SosViewModel	88
104	Architettura e Flusso di Autenticazione	89
105	Architettura moduli Luoghi Sicuri	90
106	Componenti del microservizio luoghi sicuri	93
107	Diagramma della classe Marker	93
108	Diagramma della classe SafePlacesController	94
109	Diagramma dell'interfaccia GetMarkerPort	94
110	Diagramma della classe SafePlacesService	95



111	Diagramma dell'interfaccia SafePlacesRepositoryPort	95
112	Diagramma della classe S3SafePlacesRepository	95
113	Architettura moduli Materiale Informativo	96
114	Componenti del microservizio materiale informativo	98
115	Diagramma della classe Resource	99
116	Diagramma della classe ResourcesController	100
117	Diagramma della classe GetResourcesPort	100
118	Diagramma della classe ResourcesService	100
119	Diagramma della classe ResourcesRepositoryPort	101
120	Diagramma della classe S3ResourcesRepository	101
121	Architettura diari	102
122	Componenti della Lambda per la gestione dei diari	111
123	Diagramma dell'enumerazione DiaryType	112
124	Diagramma della classe Note	112
125	Diagramma della classe NoteElement	113
126	Diagramma della classe NoteElementDTO	113
127	Diagramma della classe NoteDTO	114
128	Diagramma della classe GetNoteCmd	114
129	Diagramma della classe GetNotesCmd	115
130	Diagramma della classe AddElementNewNoteCmd	115
131	Diagramma della classe AddNoteCmd	116
132	Diagramma della classe AddNoteElementCmd	116
133	Diagramma della classe DeleteNoteCmd	117
134	Diagramma della classe DeleteNoteElementCmd	117
135	Diagramma della classe ValidatePasswordCmd	118
136	Diagramma della classe ValidateTokenCmd	118
137	Diagramma della classe SetPasswordCmd	119
138	Diagramma della classe CheckPasswordCmd	119
139	Diagramma dell'interfaccia GetNotePort	120
140	Diagramma dell'interfaccia SetNotePort	120
141	Diagramma dell'interfaccia DeleteNotePort	120
142	Diagramma dell'interfaccia SetNoteElementPort	121
143	Diagramma dell'interfaccia DeleteNoteElementPort	121
144	Diagramma dell'interfaccia SetPasswordPort	121
145	Diagramma dell'interfaccia ValidationPort	122
146	Diagramma dell'interfaccia CheckPasswordPort	122
147	Diagramma dell'interfaccia NoteRepositoryPort	123
148	Diagramma dell'interfaccia FileRepositoryPort	123
149	Diagramma dell'interfaccia DiaryAuthRepositoryPort	124
150	Diagramma della classe NoteService	124



151	Diagramma della classe DiaryAuthService	125
152	Diagramma della classe DynamoNoteAdapter	125
153	Diagramma della classe S3NoteAdapter	126
154	Diagramma della classe DynamoAuthAdapter	126
155	Diagramma della classe DiaryNoteController	127
156	Diagramma della classe DiaryAccessController	128
157	Architettura chatbot	128
158	Componenti del microservizio Chatbot	135
159	Diagramma della classe Chat	136
160	Diagramma della classe ChatMessage	136
161	Diagramma dell'interfaccia ChatbotLLMPort	137
162	Diagramma della classe BedrockLLMAdapter	137
163	Diagramma della classe ChatbotLLMService	137
164	Diagramma dell'interfaccia ChatsRepositoryPort	138
165	Diagramma della classe DynamoChatAdapter	138
166	Diagramma dell'interfaccia CreateChatPort	139
167	Diagramma dell'interfaccia GetChatPort	139
168	Diagramma dell'interfaccia UpdateChatPort	140
169	Diagramma dell'interfaccia DeleteChatPort	140
170	Diagramma dell'interfaccia ChatMessagePort	140
171	Diagramma della classe ChatbotCRUDService	141
172	Diagramma della classe CreateChatCmd	141
173	Diagramma della classe GetChatCmd	142
174	Diagramma della classe GetChatListCmd	142
175	Diagramma della classe UpdateChatCmd	142
176	Diagramma della classe DeleteChatCmd	143
177	Diagramma della classe AddChatMessageCmd	143
178	Diagramma della classe ChatDTO	144
179	Diagramma della classe ChatMessageDTO	144
180	Diagramma della classe LambdaHandler	145
181	Architettura del sistema di contatti fidati, SOS e Dead man's switch	145
182	Componenti del microservizio contatti fidati, SOS e Dead man's switch	153
183	Diagramma della classe TrustedContact	154
184	Diagramma della classe TrustedContactDTO	154
185	Diagramma della classe AddTrustedContactCmd	155
186	Diagramma della classe GetTrustedContactCmd	155
187	Diagramma della classe DeleteTrustedContactCmd	155
188	Diagramma della classe DmsConfigurationSettings	156
189	Diagramma della classe DmsConfigurationSettingsDTO	156
190	Diagramma della classe AlertCmd	157



191	Diagramma della classe EmailMessage	157
192	Diagramma della classe TrustedContactController	158
193	Diagramma della classe SetDmsHeartBeatPort	158
194	Diagramma della classe GetDmsConfigurationSettingsPort	159
195	Diagramma della classe SetDmsConfigurationSettingsPort	159
196	Diagramma della classe UpdateDmsAlertPort	159
197	Diagramma della classe SetTrustedContactPort	160
198	Diagramma della classe GetTrustedContactPort	160
199	Diagramma della classe DeleteTrustedContactPort	160
200	Diagramma della classe SendAlertPort	161
201	Diagramma della classe DmsCRUDService	161
202	Diagramma della classe DmsAlertService	161
203	Diagramma della classe TrustedContactCRUDService	162
204	Diagramma della classe SOSAlertService	162
205	Diagramma della classe DmsRepositoryPort	163
206	Diagramma della classe TrustedContactRepositoryPort	164
207	Diagramma della classe NotificationPort	164
208	Diagramma della classe DynamoDmsAdapter	165
209	Diagramma della classe DynamoTrustedContactAdapter	165
210	Diagramma della classe SesNotificationAdapter	165



1. Introduzione

1.1. Scopo del documento

Il presente documento ha lo scopo di fornire una descrizione completa, formale e strutturata del prodotto software, delineandone in modo rigoroso le caratteristiche tecniche e architetturali.

La Specifica Tecnica rappresenta il riferimento principale per tutte le attività di progettazione, sviluppo, verifica e manutenzione del sistema, assicurando coerenza rispetto ai requisiti definiti nel documento di Analisi dei requisiti e alle scelte progettuali adottate.

In particolare, il documento si propone di:

- Definire l'Architettura complessiva del sistema, descrivendo le componenti, le loro responsabilità e le modalità di interazione attraverso appositi diagrammi
- Documentare le tecnologie, i linguaggi di programmazione, i Framework e gli strumenti adottati, motivando le scelte effettuate in relazione ai requisiti di progetto e al capitolato
- Illustrare i principali design pattern e le soluzioni architetturali utilizzate.
- Il tracciamento dei requisiti, che attesta la copertura di quanto stabilito nel documento di Analisi dei Requisiti.

1.2. Riferimenti

Al fine di evitare ambiguità relative al linguaggio e ai termini utilizzati all'interno dei documenti formali, viene allegato il *Glossario*. I termini tecnici, gli acronimi e le parole con significato specifico all'interno del progetto sono contrassegnati da una lettera 'G' in apice (es. Agile) e sono descritti in dettaglio nel documento apposito.

1.2.1 Riferimenti normativi

- **Capitolato d'appalto C4: *L'app che Protegge e Trasforma***
Parti consultate: documento completo
Versione: 1.0
Ultimo accesso: 2026-01-14
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C4.pdf>
- **Norme di Progetto**
Parti consultate: documento completo
Versione: v.2.0.0
Ultimo accesso: 2026-04-30



https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/norme_di_progetto.pdf

1.2.2 Riferimenti informativi

- **Slide sulla Progettazione Software del Prof. Vardanega**

Parti consultate: documento completo

Versione: *A.A.2025/2026*

Ultimo accesso: *2026-03-30*

<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>

- **Materiale relativo alla parte pratica del corso di Ingegneria del Software**

Parti consultate: documento completo

Versione: *A.A.2022/2023*

Ultimo accesso: *2026-03-30*

<https://www.math.unipd.it/~rcardin/swea/2022/>

- ***repository Github* del Prof. Cardin**

Ultimo accesso: *2026-03-30*

<https://github.com/rcardin/swe-imdb>

- **Guida Flutter sull'Architettura di un'applicazione**

Ultimo accesso: *2026-03-30*

<https://docs.flutter.dev/app-architecture/guide>

- **Glossario**

Parti consultate: documento completo

Versione: *v.2.0.0*

Ultimo accesso: *2026-05-13*

https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/Glossario.pdf

2. Tecnologie

Le tecnologie utilizzate sono state scelte trovando un punto d'incontro tra la necessità di garantire la sicurezza e la tutela dei dati personali degli utenti e il bisogno di realizzare un'Architettura solida, modulare e facilmente estendibile. Di seguito sono riportate tutte le tecnologie adottate.



2.1. Framework

2.1.1 Flutter

Flutter è un Framework open-source sviluppato da Google per la creazione di applicazioni multi-piattaforma a partire da un unico codice sorgente. Consente di sviluppare interfacce per dispositivi mobili, web e desktop utilizzando il linguaggio Dart. Flutter si basa su un sistema di widget componibili, che possono rappresentare qualsiasi elemento dell'interfaccia utente. Flutter include un proprio motore grafico, Impeller, che disegna direttamente i componenti a schermo, garantendo elevata coerenza visiva e prestazioni indipendenti dalla piattaforma sottostante. È il Framework alla base dell'App Che Protegge e Trasforma, il che consente di poter generare build per iOS e Android da un unico codice sorgente, a fronte di un maggiore utilizzo di risorse rispetto alle soluzioni native e di una minore possibilità di controllo diretto e personalizzazione delle funzionalità native della piattaforma.

- **Versione utilizzata:** 3.41.6
- **Documentazione:** <https://docs.flutter.dev/>

2.2. Linguaggi di programmazione

2.2.1 Dart

Dart è un linguaggio di programmazione sviluppato da Google, progettato per la realizzazione di applicazioni client moderne. È un linguaggio tipizzato (con supporto sia statico che dinamico) e orientato agli oggetti. Viene utilizzato principalmente in combinazione con Flutter, dove il codice Dart viene compilato ahead-of-time in codice nativo per migliorare le prestazioni, oppure eseguito tramite compilazione just-in-time durante lo sviluppo. L'utilizzo di Flutter rende di fatto quasi obbligatoria l'adozione di Dart come linguaggio.

- **Versione utilizzata:** 3.11.4
- **Documentazione:** <https://dart.dev/docs>

2.2.2 Python

Python è un linguaggio di programmazione ad alto livello, interpretato e orientato agli oggetti, noto per la sua sintassi semplice e leggibile. Il codice Python viene eseguito da un interprete che traduce le istruzioni in bytecode, poi eseguito dalla macchina virtuale Python. Nel progetto è utilizzato lato Backend per la scrittura delle funzioni AWS Lambda e per i test tramite Pytest.

- **Versione utilizzata:** 3.14.3
- **Documentazione:** <https://docs.python.org/3/>



2.3. Servizi AWS

2.3.1 Lambda

AWS Lambda è un servizio Serverless che consente di eseguire codice senza gestire direttamente server o infrastruttura. Le funzioni vengono attivate in risposta a eventi e scalano automaticamente in base al carico. Nel progetto è utilizzato per implementare la logica Backend, eseguendo funzioni scritte in Python invocate tramite API Gateway.

- **Versione utilizzata:** Runtime Python 3.14
- **Documentazione:** <https://docs.aws.amazon.com/lambda/>

2.3.2 API Gateway

AWS API Gateway è un servizio che permette di creare, pubblicare e gestire API, fungendo da punto di ingresso per le richieste client. Gestisce autenticazione, routing e integrazione con altri servizi AWS. Nel progetto è utilizzato per esporre endpoint HTTP che invocano le funzioni Lambda.

- **Versione utilizzata:** HTTP API (v2)
- **Documentazione:** <https://docs.aws.amazon.com/apigateway/>

2.3.3 DynamoDB

Amazon DynamoDB è un database NoSQL completamente gestito, progettato per offrire alte prestazioni e scalabilità automatica. I dati sono memorizzati in tabelle con chiavi primarie e accessibili con bassa latenza. Nel progetto è utilizzato per memorizzare dati strutturati come le note testuali del diario criptato e le chat con il chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/dynamodb/>

2.3.4 S3

Amazon S3 (Simple Storage Service) è un servizio di storage a oggetti che consente di archiviare e recuperare dati in modo scalabile e sicuro. I file sono organizzati in bucket e accessibili tramite API. Nel progetto è utilizzato per memorizzare contenuti multimediali come immagini e file audio.

- **Documentazione:** <https://docs.aws.amazon.com/s3/>



2.3.5 Cognito

Amazon Cognito è un servizio di gestione delle identità che consente autenticazione, autorizzazione e gestione degli utenti. Supporta integrazione con provider esterni come Google. Nel progetto è utilizzato per gestire l'autenticazione degli utenti tramite account Google e per il controllo degli accessi.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/>

2.3.6 Bedrock

Amazon Bedrock è un servizio che consente di utilizzare modelli di intelligenza artificiale generativa tramite API, senza gestire direttamente l'infrastruttura sottostante. Permette l'accesso a diversi modelli foundation per task come generazione di testo. Nel progetto è utilizzato per implementare la funzionalità di chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/bedrock/>

2.3.7 SES

Amazon SES (Simple Email Service) è un servizio per l'invio e la ricezione di email in modo scalabile e affidabile. Supporta integrazione via API e SMTP. Nel progetto è utilizzato per inviare email ai contatti fidati degli utenti.

- **Documentazione:** <https://docs.aws.amazon.com/ses/>

2.3.8 CloudWatch

Amazon CloudWatch è un servizio di monitoraggio e osservabilità che raccoglie metriche, log ed eventi dai servizi AWS. Consente analisi, visualizzazione e configurazione di allarmi. Nel progetto è utilizzato per la raccolta e l'analisi dei log generati dalle funzioni Lambda.

- **Documentazione:** <https://docs.aws.amazon.com/cloudwatch/>

2.3.9 EventBridge

Amazon EventBridge è un servizio di event bus Serverless che consente di gestire e instradare eventi tra diversi servizi AWS. Esso permette di definire regole per reagire automaticamente a eventi generati da servizi AWS o da fonti esterne. Nel progetto è utilizzato per orchestrare i flussi di controllo di utilizzo dell'app da parte degli utenti, e l'invio delle email secondo le tempistiche stabilite dal Dead man's switch.

- **Documentazione:** <https://docs.aws.amazon.com/eventbridge/>



2.3.10 SAM

AWS Serverless Application Model (SAM) è un Framework open-source che semplifica la definizione e il Deployment di applicazioni Serverless su AWS. Estende AWS CloudFormation introducendo una sintassi più compatta per risorse come Lambda, API Gateway e DynamoDB e include strumenti per build, testing locale e deploy delle applicazioni. Nel progetto è utilizzato per definire e distribuire l'infrastruttura Serverless in modo semplificato.

- **Documentazione:** <https://docs.aws.amazon.com/serverless-application-model/>

2.3.11 CloudFormation

AWS CloudFormation è un servizio di Infrastructure as Code che consente di modellare e gestire risorse AWS tramite template dichiarativi in formato YAML o JSON. Nel progetto è utilizzato come base per il provisioning dell'infrastruttura, anche tramite template generati da SAM.

- **Documentazione:** <https://docs.aws.amazon.com/cloudformation/>
- **Versioni:** CloudFormation non prevede versioni di servizio esplicite; eventuali aggiornamenti sono gestiti direttamente da AWS. I template possono però utilizzare versioni di specifiche risorse e trasformazioni (es. AWS::Serverless).

2.4. Librerie

2.4.1 http

`http` è una libreria per il linguaggio Dart che consente di effettuare richieste HTTP in modo semplice e asincrono. Fornisce metodi per eseguire operazioni come GET, POST, PUT e DELETE, gestendo automaticamente la serializzazione delle richieste e la ricezione delle risposte. È progettata per essere leggera e facilmente integrabile in applicazioni client.

- **Versione utilizzata:** 1.6.0
- **Documentazione:** <https://pub.dev/packages/http>

2.4.2 image_picker

`image_picker` è una libreria Flutter che permette di accedere alla fotocamera e alla galleria del dispositivo per selezionare o acquisire immagini e video. Fornisce un'interfaccia unificata tra piattaforme diverse, gestendo le differenze tra iOS e Android.

- **Versione utilizzata:** 1.2.1
- **Documentazione:** https://pub.dev/packages/image_picker



2.4.3 path_provider

`path_provider` è una libreria Flutter che consente di ottenere i percorsi delle directory del file system del dispositivo, come directory temporanee o documenti applicativi. Facilita l'accesso a percorsi corretti e compatibili tra piattaforme per la gestione dei file.

- **Versione utilizzata:** 2.1.5
- **Documentazione:** https://pub.dev/packages/path_provider

2.4.4 provider

`provider` è una libreria per Flutter che offre un meccanismo di gestione dello stato e di propagazione delle dipendenze lungo l'albero dei widget. Si basa sul sistema nativo di `InheritedWidget` di Flutter, semplificandone notevolmente l'utilizzo attraverso un'API dichiarativa. Consente di rendere disponibili oggetti (come i `ViewModel`) a interi sottoalberi dell'interfaccia, facilitando la comunicazione reattiva tra i layer dell'applicazione.

- **Versione utilizzata:** 6.1.5+1
- **Documentazione:** <https://pub.dev/packages/provider>

2.4.5 url_launcher

`url_launcher` è una libreria Flutter che consente di aprire URL nel browser predefinito del dispositivo, avviare applicazioni di posta elettronica, effettuare chiamate telefoniche o aprire altri tipi di risorse tramite i rispettivi handler nativi. Fornisce un'interfaccia unificata e compatibile tra le diverse piattaforme supportate da Flutter.

- **Versione utilizzata:** 6.3.2
- **Documentazione:** https://pub.dev/packages/url_launcher

2.4.6 flutter_dotenv

`flutter_dotenv` è una libreria che permette di caricare variabili di configurazione da un file `.env` all'interno di un'applicazione Flutter. Consente di separare le configurazioni sensibili o dipendenti dall'ambiente (come chiavi API o endpoint) dal codice sorgente, rendendo più agevole la gestione di ambienti diversi senza modificare il codice dell'applicazione.

- **Versione utilizzata:** 6.0.0
- **Documentazione:** https://pub.dev/packages/flutter_dotenv



2.4.7 flutter_web_auth_2

`flutter_web_auth_2` è una libreria Flutter progettata per gestire flussi di autenticazione basati su protocolli web standard come OAuth 2.0 e OpenID Connect. Apre una sessione browser controllata, intercetta il redirect dell'URL di callback e restituisce il codice di autorizzazione all'applicazione, abilitando l'integrazione con provider di identità esterni senza esporre le credenziali nel codice nativo.

- **Versione utilizzata:** 5.0.2
- **Documentazione:** https://pub.dev/packages/flutter_web_auth_2

2.4.8 flutter_map

`flutter_map` è una libreria Flutter open-source per la visualizzazione di mappe interattive. Fornisce funzionalità per il rendering di layer multipli, gestione di marker, polilinee e poligoni, oltre al supporto per gesture come zoom e pan. È altamente configurabile e non dipende da servizi proprietari esterni come Google Maps.

- **Versione utilizzata:** 8.3.0
- **Documentazione:** https://pub.dev/packages/flutter_map

2.4.9 geolocator

`geolocator` è una libreria Flutter che consente di accedere ai servizi di geolocalizzazione del dispositivo. Permette di ottenere la posizione corrente, monitorare aggiornamenti di posizione in tempo reale e gestire i permessi richiesti dalle diverse piattaforme.

- **Versione utilizzata:** 14.0.2
- **Documentazione:** <https://pub.dev/packages/geolocator>

2.4.10 get_it

`get_it` è un service locator per Dart e Flutter utilizzato per la gestione delle dipendenze all'interno dell'applicazione. Consente di registrare e recuperare istanze di oggetti in modo centralizzato, facilitando l'implementazione di architetture modulari e testabili.

- **Versione utilizzata:** 9.2.1
- **Documentazione:** https://pub.dev/packages/get_it



2.4.11 latlong2

`latlong2` è una libreria Dart che fornisce strutture dati e utilities per la gestione di coordinate geografiche (latitudine e longitudine). Include funzionalità per il calcolo di distanze tra punti e altre operazioni geospaziali di base.

- **Versione utilizzata:** 0.10.1
- **Documentazione:** <https://pub.dev/packages/latlong2>

2.4.12 path

`path` è una libreria Dart che fornisce strumenti per la gestione e manipolazione dei percorsi file, indipendenti dalla piattaforma di esecuzione.

- **Versione utilizzata:** 1.9.1
- **Documentazione** <https://pub.dev/packages/path>

2.4.13 flutter_secure_storage

`flutter_secure_storage` è una libreria Flutter che consente il salvataggio di dati sensibili in modo sicuro e specifico per la piattaforma di esecuzione. I dati vengono salvati come coppie chiave-valore e possono essere criptati prima del salvataggio.

- **Versione utilizzata:** 10.0.0
- **Documentazione** https://pub.dev/packages/flutter_secure_storage

2.4.14 just_audio

`just_audio` è una libreria Flutter che fornisce strumenti per la creazione e utilizzo di player audio, e supporta molteplici formati di tracce audio.

- **Versione utilizzata:** 0.10.5
- **Documentazione** https://pub.dev/packages/just_audio

2.4.15 file_picker

`file_picker` è una libreria Flutter che permette l'utilizzo del file picker nativo per la selezione di uno o più file. La libreria permette anche di filtrare i file selezionabili dall'Utente in base al formato.

- **Versione utilizzata:** 11.0.2
- **Documentazione** https://pub.dev/packages/file_picker



2.4.16 url_launcher_platform_interface

`url_launcher_platform_interface` è una libreria flutter che permette estensioni platform-specific del plugin `url_launcher`.

- **Versione utilizzata:** 2.3.2
- **Documentazione** https://pub.dev/packages/url_launcher_platform_interface

2.4.17 command_it

`command_it` è una libreria Flutter che permette l'implementazione del pattern *Command*, tramite la creazione di wrap functions con gestione automatica dello stato.

- **Versione utilizzata:** 9.5.1
- **Documentazione** https://pub.dev/packages/command_it

2.4.18 listen_it

`listen_it` è una libreria Flutter che fornisce primitive reattive per la creazione di oggetti che notificano automaticamente eventuali ascoltatori in seguito a mutamenti del proprio stato.

- **Versione utilizzata:** 5.3.5
- **Documentazione** https://pub.dev/packages/listen_it

2.4.19 mocktail

`mocktail` è una libreria Dart che fornisce una API per la creazione di Mock senza doverli creare da zero.

- **Versione utilizzata:** 1.0.5
- **Documentazione** <https://pub.dev/packages/mocktail>

2.4.20 shared_preferences

`shared_preferences` è una libreria Flutter che permette l'utilizzo della memorizzazione persistente platform-specific per il salvataggio di dati semplici.

- **Versione utilizzata:** 2.5.5
- **Documentazione** https://pub.dev/packages/shared_preferences



2.4.21 connectivity_plus

`connectivity_plus` è una libreria Flutter che permette all'applicazione di conoscere i tipi di connessione disponibili in un dato momento, in modo da reagire di conseguenza.

- **Versione utilizzata:** 7.1.1
- **Documentazione** https://pub.dev/packages/connectivity_plus

2.4.22 audio_session

`audio_session` è una libreria Flutter per la gestione delle sessioni audio dell'applicazione. Permette una configurazione molto approfondita delle sessioni audio, permettendo di specificare caratteristiche quali il comportamento in presenza di audio appartenenti ad altre applicazioni o quali dispositivi possono riprodurre l'audio.

- **Versione utilizzata:** 0.2.3
- **Documentazione** https://pub.dev/packages/audio_session

2.4.23 python-ulid

`python-ulid` è una libreria Python per la generazione di identificatori univoci ULID (Universally Unique Lexicographically Sortable Identifier). Gli ULID combinano unicità e ordinamento temporale, risultando particolarmente utili in sistemi distribuiti dove è necessario generare ID ordinabili senza coordinamento centrale.

- **Versione utilizzata:** 3.1.0
- **Documentazione:** <https://pypi.org/project/python-ulid/>

2.4.24 rank-bm25

`rank-bm25` è una libreria Python che implementa l'algoritmo di ranking BM25 (Best Matching 25), utilizzato nel campo dell'information retrieval per valutare la rilevanza dei documenti rispetto a una query testuale. Fornisce diverse varianti dell'algoritmo BM25 e consente di costruire rapidamente sistemi di ricerca basati su punteggi statistici senza necessità di motori di ricerca completi.

- **Versione utilizzata:** 0.2.2
- **Documentazione:** <https://pypi.org/project/rank-bm25/>



2.4.25 boto3

`boto3` è una libreria Python che fornisce strumenti per la creazione, configurazione e gestione dei servizi Amazon Web Services.

- **Versione utilizzata:** 1.43.5
- **Documentazione:** <https://docs.aws.amazon.com/boto3/latest/>

2.5. Tecnologie di analisi statica

2.5.1 SonarQube

SonarQube è una piattaforma open-source per l'analisi continua della qualità del codice. Esegue Analisi Statica su diversi linguaggi di programmazione per individuare bug, vulnerabilità, code smells e problemi di copertura dei test. Il funzionamento si basa su scanner che analizzano il codice sorgente e inviano i risultati a un server centrale, dove vengono aggregati e visualizzati tramite una dashboard dedicata.

- **Versione utilizzata:** 26.3.0
- **Documentazione:** <https://docs.sonarsource.com/sonarqube/>

2.6. Tecnologie di analisi dinamica

2.6.1 Pytest

Pytest è un Framework di testing per il linguaggio Python che consente di scrivere test in modo semplice e scalabile. Supporta test funzionali, unitari e di integrazione, offrendo funzionalità come fixture, parametrizzazione dei test e scoperta automatica dei test.

- **Versione utilizzata:** 9.0.2
- **Documentazione:** <https://docs.pytest.org/>

2.6.2 Moto

Moto è una libreria Python che consente di mockare i servizi AWS durante i test. Simula il comportamento delle API AWS, permettendo di testare codice che interagisce con servizi Cloud senza effettuare chiamate reali. Funziona intercettando le richieste e restituendo risposte simulate coerenti con quelle dei servizi AWS.

- **Versione utilizzata:** 5.1.23
- **Documentazione:** <https://docs.getmoto.org/>



2.7. Altro

2.7.1 Ollama

Ollama è una piattaforma che consente di eseguire e gestire LLM in locale tramite una semplice interfaccia. Permette il download, l'esecuzione e l'interazione con modelli di intelligenza artificiale direttamente in ambiente locale, senza necessità di servizi Cloud esterni. Il funzionamento si basa su un runtime che gestisce i modelli e espone endpoint per inferenza.

- **Versione utilizzata:** 0.18.3
- **Documentazione:** <https://ollama.com/docs>

2.7.2 Docker

Docker è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati detti container. I container includono tutto il necessario per eseguire un'applicazione, garantendo portabilità e consistenza tra ambienti diversi. Nel progetto è utilizzato a supporto delle pipeline di CI/CD e per testare in locale servizi come DynamoDB e S3.

- **Documentazione:** <https://docs.docker.com/>

3. Architettura

3.1. Architettura logica

Il sistema software è progettato seguendo un approccio modulare e disaccoppiato, volto a massimizzare l'indipendenza tra la Business Logic, l'interfaccia utente e i meccanismi di persistenza dei dati. Per rispondere alle diverse esigenze tecnologiche e funzionali del client mobile e del backend, sono stati adottati due paradigmi architetturali distinti ma complementari, entrambi focalizzati sulla *Separation of Concerns* e sulla facilità di test. L'interazione garantisce che l'evoluzione tecnologica di una parte del sistema non influenzi negativamente le altre.

3.1.1 Frontend Mobile

Il client mobile è sviluppato seguendo i principi della *Layered Architecture* coniugata con il pattern architetturale **Model-View-ViewModel**. Questa scelta permette di isolare la logica di presentazione dalla business logic e dai protocolli di rete, suddividendo il codice nei seguenti strati logici:



- **Presentation Layer (View & ViewModel):** La View, costituita dai diversi widget Flutter, ha la responsabilità esclusiva di renderizzare l'interfaccia e catturare gli input Utente. Non contiene business logic. Il ViewModel funge da mediatore: gestisce lo stato della View notificandola di cambiamenti e incapsula la logica di presentazione tramite il pattern Command. Quest'ultimo permette di gestire operazioni asincrone, monitorare i caricamenti e centralizzare la gestione degli errori senza sporcare il codice dell'interfaccia.
- **Domain Layer (Business Logic):** Rappresentato dai modelli di dominio, che definiscono le entità base su cui opera l'intera app. In quanto modelli di dominio, sono completamente indipendenti da qualsiasi tecnologia di persistenza o protocollo di comunicazione, gestendo esclusivamente la business logic dell'applicazione.
- **Data Layer (Repository & Service):** Incaricato dell'acquisizione dei dati, il data layer utilizza classi Repository per trasformare i dati grezzi in arrivo dal backend in modelli di dominio, gestendo la logica di caching e di gestione degli errori. Le classi Service, invece, si fanno carico della gestione delle comunicazioni con fonti di dati esterne, esponendo metodi per inviare richieste HTTP al backend e ricevere le risposte sottoforma di JSON.

3.1.2 Backend con architettura esagonale

Ogni Lambda Function costituisce, insieme alle istanze dei servizi che utilizza, un microservizio indipendente. Il codice che ogni Lambda esegue è implementato seguendo il pattern dell'Architettura esagonale. Questo approccio garantisce un basso accoppiamento tra le componenti del sistema, isolando la domain logic dalle tecnologie esterne e facilitando la scrittura dei test.

In questo pattern il sistema comunica con le componenti esterne attraverso interfacce, le port, mentre gli adapter sono responsabili dell'implementazioni concrete di queste.

Il nucleo del sistema è rappresentato dalla domain logic, che racchiude sia la logica applicativa che quella di business. È implementata nelle classi **Service**, che non dipendono da alcuna tecnologia specifica e interagiscono con le risorse esterne esclusivamente attraverso interfacce. Le operazioni esposte dai **Service** accettano oggetti di dominio o oggetti Command come input. I Command costituiscono istanze di classi che incapsulano i dati necessari per eseguire un'operazione specifica.

Le port si dividono in due categorie:

- **Inbound port:** definiscono le interfacce attraverso cui le risorse esterne comunicano con la domain logic. Il loro utilizzo isola il nucleo del sistema da qualsiasi dipendenza tecnologica esterna, rendendo intercambiabili le implementazioni che le realizzano.



- Outbound port: definiscono le interfacce attraverso cui la domain logic interagisce con i servizi esterni, come le risorse di persistenza. In particolare, le Repository port specificano i metodi che gli adapter devono implementare per le operazioni di lettura e scrittura sui dati.

Gli adapter si dividono anche essi in 2 categorie:

- Inbound adapter: sono rappresentati dalle classi **Controller**, responsabili della ricezione degli eventi AWS Lambda e della loro traduzione in oggetti di dominio comprensibili alla domain logic. Il **Controller** determina la route dall'evento in ingresso, esegue il parsing del body e dei parametri, costruisce il body della risposta da restituire al client e delega l'esecuzione alla domain logic attraverso le inbound port.
- Outbound adapter: costituiscono l'implementazione concreta delle outbound port per i servizi AWS utilizzati da ciascun microservizio. Ogni adapter incapsula il client del servizio AWS corrispondente, ad esempio DynamoDB per la persistenza dei dati e S3 per la gestione dei file multimediali, esponendo i metodi definiti dalla porta. Questo approccio permette di sostituire il servizio sottostante modificando esclusivamente l'adapter, senza impatto sulla *domain logic*.

3.2. Design pattern utilizzati

3.2.1 Model-view-viewmodel

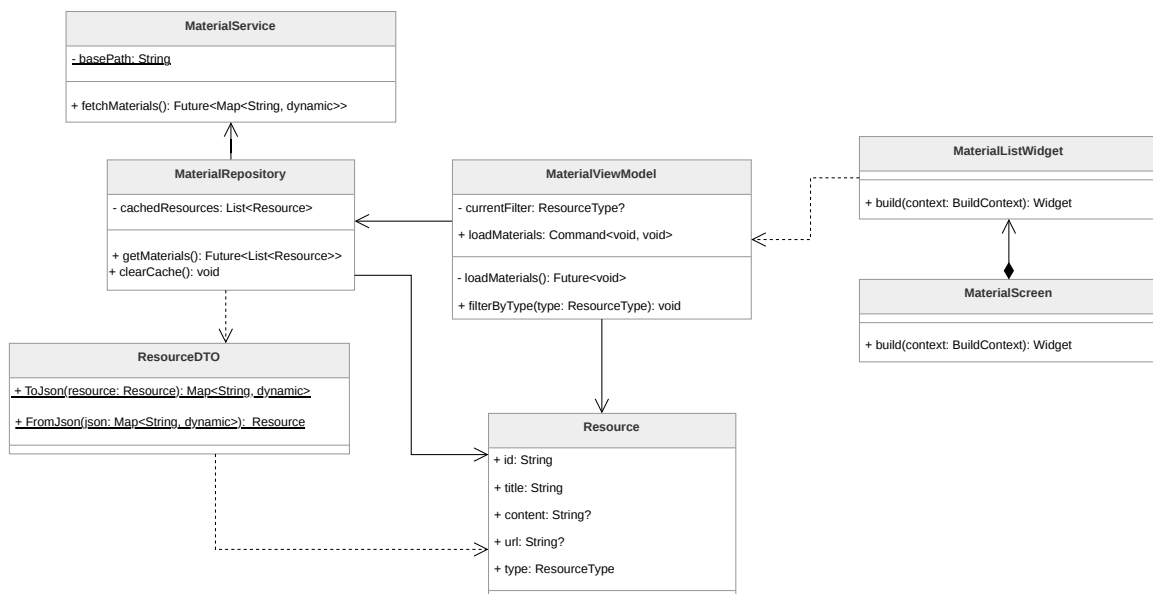


Figura 1: Rappresentazione del pattern MVVM nel modulo del materiale informativo



3.2.1.1 Descrizione del pattern:

Il pattern MVVM prevede la separazione tra la resa grafica, la Presentation Logic e la Business Logic in tre componenti principali:

- **Model:** contiene il codice responsabile per la Business Logic dell'applicazione.
- **View:** contiene la parte responsabile della resa grafica del software e della cattura delle interazioni dell'Utente, a cui reagisce invocando i metodi o i comandi messi a disposizione dal *ViewModel*.
- **ViewModel:** mantiene e gestisce lo stato della *View* e coordina le interazioni tra quest'ultima e il *Model*. La *View* può modificare il proprio stato esclusivamente attraverso l'interazione con i metodi e i **Command** esposti dal *ViewModel*.

Nel contesto del Framework Flutter, l'implementazione di questo pattern viene riadattata per assecondare la natura dichiarativa dell'interfaccia. Nello specifico, il *ViewModel* è tipicamente implementato sfruttando una classe o un mixin proprietario chiamato **ChangeNotifier**, il quale consente di notificare esplicitamente la *View* dei cambiamenti di stato interno. La *View*, composta a sua volta da *Widget*, ascolta reattivamente tali notifiche potendo ricostruire e aggiornare solo le specifiche porzioni grafiche necessarie.

3.2.1.2 Ragioni per l'utilizzo:

L'utilizzo del pattern garantisce notevoli vantaggi poiché permette una più semplice ed efficace gestione delle varie parti dell'applicazione, limitando il forte accoppiamento (*coupling*) tra l'interfaccia utente visiva e la logica di business o di presentazione sottostante. Questo netto disaccoppiamento migliora la manutenibilità del codice dell'applicativo e agevola considerevolmente lo sviluppo e l'esecuzione degli unit test, in quanto permette di testare i *ViewModel* e i *Model* isolandoli dal ciclo di vita visivo tipico del sistema operativo e del Framework.

3.2.1.3 Riferimenti nel progetto:

Il pattern MVVM è ampiamente utilizzato per strutturare l'interfaccia utente dell'applicazione e la logica ad essa associata, separando la resa grafica dalla gestione dello stato e dei dati. Di seguito sono elencati i punti principali dove viene impiegato; all'interno di tali sezioni sono fornite spiegazioni dettagliate su come il pattern interagisce con il sistema circostante:

- Autenticazione → **AuthViewModel**
- Chatbot → **ChatbotViewModel**
- Diario → **DiaryViewModel**
- Contatti Fidati → **TrustedContactViewModel**



- Materiale Informativo → **MaterialViewModel**
- Luoghi Sicuri → **SafePlaceViewModel**
- Impostazioni → **DeadManViewModel**

3.2.2 Command

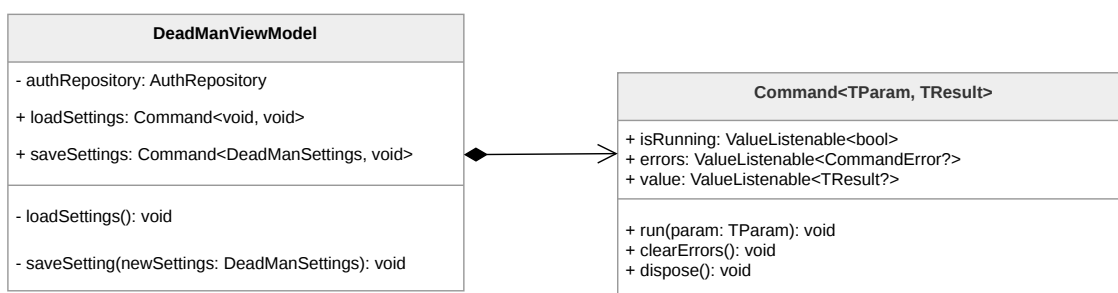


Figura 2: Rappresentazione del pattern Command

3.2.2.1 Descrizione del pattern:

Il pattern Command è un design pattern comportamentale che incapsula una richiesta o un'azione come un oggetto autonomo. Nel frontend dell'applicazione, questo pattern è implementato tramite il pacchetto **command_it**, che permette di standardizzare l'esecuzione della logica asincrona all'interno dei ViewModel.

A differenza di una semplice funzione, il comando così implementato agisce come un contenitore logico che gestisce automaticamente il Ciclo di vita di un'operazione: dall'invocazione alla gestione dei risultati, fino alla cattura di eventuali eccezioni. Ogni comando espone tre proprietà osservabili tramite **ValueListenable**:

- **isRunning**: un booleano che indica se l'operazione è attualmente in esecuzione.
- **errors**: contiene eventuali errori catturati durante l'esecuzione.
- **value**: contiene il risultato dell'operazione una volta completata con successo.

3.2.2.2 Ragioni per l'utilizzo:

L'adozione di **command_it** risponde alla necessità di eliminare il "boilerplate code" (codice ripetitivo) tipico della gestione dello stato asincrono in Flutter. Le ragioni principali includono:

- **Standardizzazione dello Stato**: evita la proliferazione manuale di variabili come **isLoading** o **errorMessage** per ogni singola azione nel ViewModel, accentrando tutto nell'oggetto Command.



- **Prevenzione di Esecuzioni Concorrenti:** il pattern impedisce automaticamente l'avvio accidentale di più istanze della stessa operazione (es. pressioni ripetute su un tasto di invio) finché la precedente non è terminata.
- **Disaccoppiamento UI-Logica:** la View non deve implementare blocchi **try-catch** o logiche di feedback: si limita a invocare il comando e "osservare" reattivamente i suoi cambiamenti di stato per mostrare indicatori di caricamento o dialoghi di errore.

3.2.2.3 Riferimenti nel progetto:

Il pattern viene implementato sistematicamente in tutti i ViewModel dell'applicazione per la gestione delle interazioni Utente e delle operazioni asincrone.

3.2.3 Observer

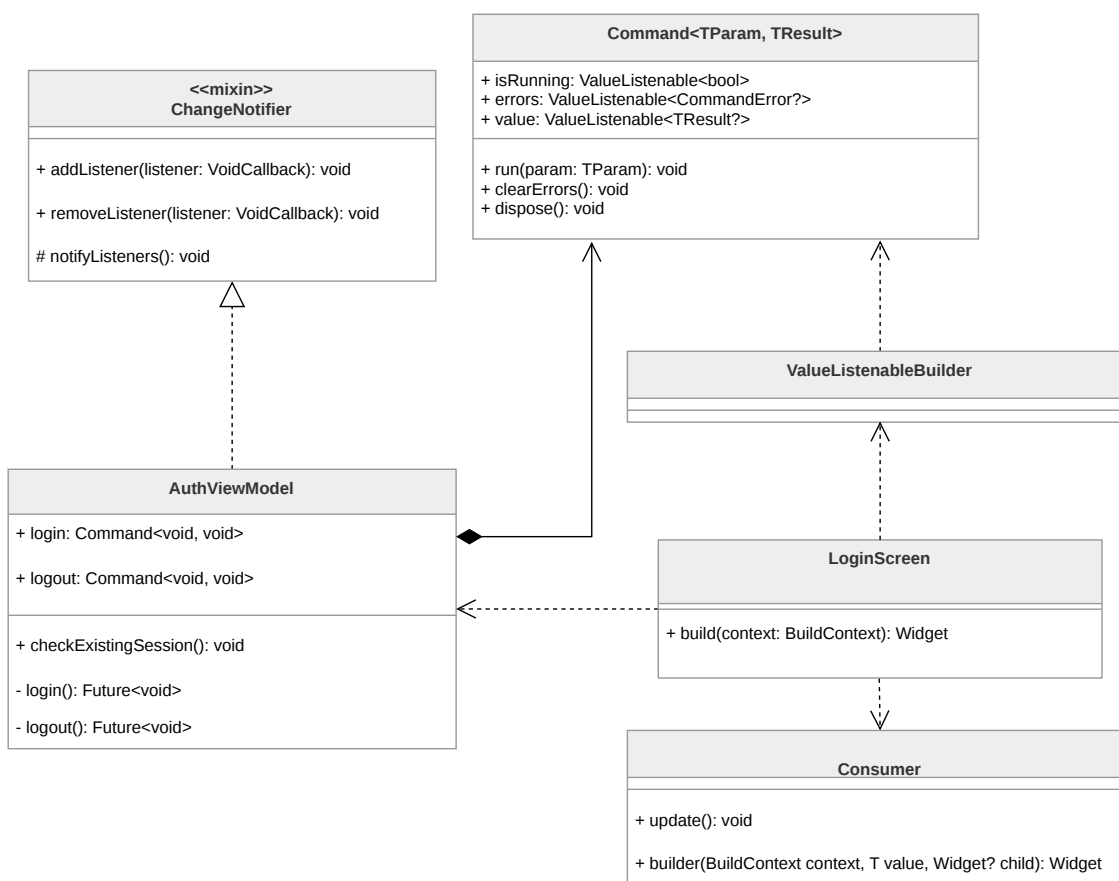


Figura 3: Rappresentazione del pattern Observer applicato nel modulo dell'autenticazione

3.2.3.1 Descrizione del pattern:

Il pattern Observer definisce un meccanismo di comunicazione in cui un oggetto, chiamato *Publisher*, mantiene una lista di oggetti in ascolto, chiamati *Listeners*, o *Subscribers*, e li



notifica automaticamente di ogni cambiamento di stato. Nel sistema sviluppato, questo pattern è stato implementato con due diversi livelli di granularità per gestire la reattività dell'interfaccia Utente:

- **Livello Macro (ChangeNotifier e Consumer):** Il ViewModel agisce come Publisher globale della schermata implementando la classe `ChangeNotifier`. Quando il ViewModel invoca il metodo `notifyListeners()`, segnala mutamento dello stato della View. Il widget `Consumer` funge da Subscriber, ricostruendo i rami della View che dipendono da quel ViewModel. Questo approccio è utilizzato per aggiornamenti di stato generali che influenzano più componenti contemporaneamente (per esempio lo stato di autenticazione).
- **Livello Micro (Command e ValueListenableBuilder):** Per una gestione più granulare, il sistema sfrutta l'integrazione tra il pattern Command e l'interfaccia `ValueListenable`. Ogni azione asincrona (es. `login`, `loadContacts`) è incapsulata in un oggetto `Command`, che agisce come un Publisher specializzato. Esso espone tre flussi di dati osservabili separatamente: `isRunning`, `errors` e `value`. La View utilizza widget `ValueListenableBuilder` per "mettersi in ascolto" solo di una specifica proprietà del comando. Ciò permette, ad esempio, di mostrare un indicatore di caricamento reagendo solo alla variazione di `isRunning`, senza richiedere la notifica di aggiornamento dell'intero ViewModel.

3.2.3.2 Ragioni per l'utilizzo:

L'adozione di questa doppia implementazione risponde a esigenze di efficienza prestazionale e pulizia del codice. L'utilizzo di `ValueListenableBuilder` accoppiato ai `Command` riduce drasticamente il numero di "rebuild" inutili dell'interfaccia: la View può reagire al cambiamento del risultato di una singola operazione asincrona senza dover processare nuovamente lo stato globale del modulo. Questo garantisce una UI fluida anche in presenza di logiche complesse o flussi di dati frequenti. Inoltre, questo approccio disaccoppia la gestione degli errori e del caricamento (loading state) dalla logica di business principale, centralizzandola negli oggetti Command osservabili.

3.2.3.3 Riferimenti nel progetto:

Essendo strettamente interconnesso al MVVM, il pattern Observer è utilizzato per la gestione dell'intera UI. Di seguito i punti principali dove i ViewModel agiscono da *Publisher* per le rispettive schermate:

- Autenticazione → `AuthViewModel`
- Chatbot → `ChatbotViewModel`
- Diario → `DiaryViewModel`



- Contatti Fidati → **TrustedContactViewModel**
- Materiale Informativo → **MaterialViewModel**
- Luoghi Sicuri → **SafePlaceViewModel**
- Impostazioni → **DeadManViewModel**

3.2.4 Proxy

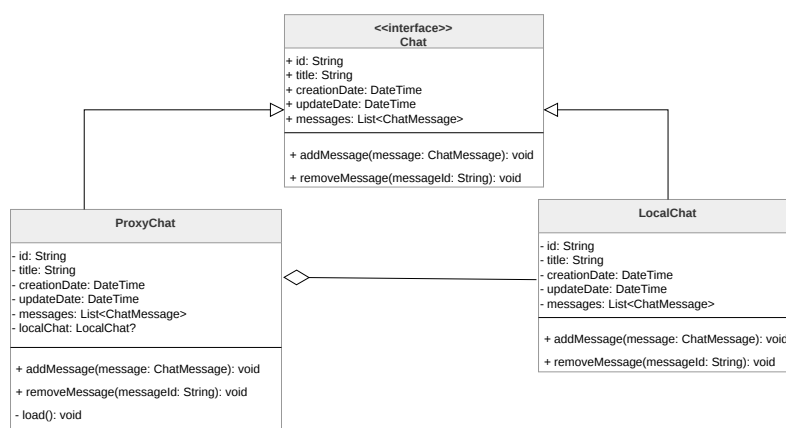


Figura 4: Rappresentazione del pattern Proxy applicato alla gestione delle chat

3.2.4.1 Descrizione del pattern:

Il pattern Proxy permette la creazione di un oggetto *Proxy* che controlla l'accesso all'oggetto reale. L'oggetto *Proxy* implementa la medesima interfaccia dell'oggetto reale, rendendolo perfettamente intercambiabile agli occhi del codice che lo utilizza, e ha il compito di delegare le richieste a un'istanza dell'oggetto reale solo quando necessario.

3.2.4.2 Ragioni per l'utilizzo:

L'utilizzo di questo pattern permette di implementare il caricamento ritardato di strutture dati potenzialmente voluminose; nello specifico, le classi che fungono da Proxy operano come segnaposti leggeri contenenti solo metadati essenziali, rimandando il caricamento completo dei contenuti (come i messaggi di una singola chat) al momento del loro effettivo accesso. Questo approccio permette di ridurre sensibilmente l'utilizzo di banda, limitando la quantità di dati recuperati dal Backend a quelli strettamente necessari, garantendo al contempo una maggiore fluidità nel caricamento degli elenchi e un consumo di risorse sul dispositivo più efficiente.

3.2.4.3 Riferimenti nel progetto:

Il pattern Proxy è utilizzato principalmente per la gestione dei modelli di dati persistenti:

- Chatbot → **ProxyChat**



- Diari Personali → ProxyNote

3.2.5 Singleton

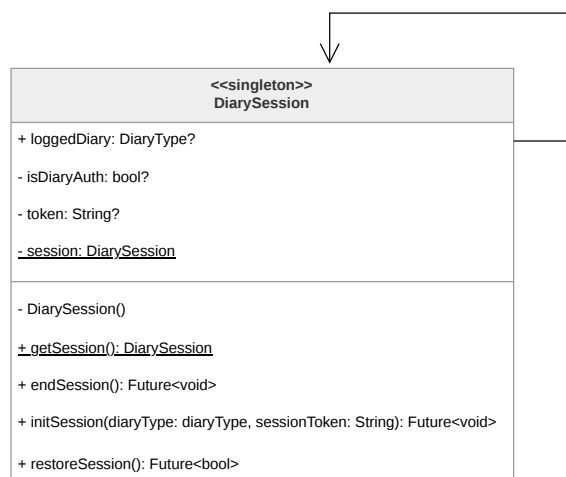


Figura 5: Rappresentazione del pattern Singleton applicato alla sessione del diario

3.2.5.1 Descrizione del pattern:

Il pattern Singleton garantisce che una classe abbia una sola istanza attiva in ogni momento e fornisce un punto di accesso globale a tale istanza. Questa proprietà si ottiene rendendo privato il costruttore della classe, impedendone l'istanziamento diretta dall'esterno, e mettendo a disposizione un metodo statico o una proprietà (*getter*) che restituisce l'unica istanza disponibile.

Nell'applicazione, questo pattern viene implementato seguendo gli standard del linguaggio Dart, utilizzando un costruttore privato e una variabile statica per mantenere il riferimento all'unica istanza creata al primo accesso.

3.2.5.2 Ragioni per l'utilizzo:

L'impiego del Singleton è fondamentale per gestire componenti che devono mantenere uno stato unico e coerente in tutto il sistema. Nello specifico, garantisce l'esistenza di una sola sessione attiva per il diario, impedendo che l'Utente possa accedere contemporaneamente alla versione criptata e a quella fittizia. Questo approccio previene la condivisione accidentale di dati sensibili e assicura che ogni operazione di lettura o scrittura avvenga sempre nel contesto dell'archivio corretto, eliminando alla radice potenziali conflitti di sincronizzazione.

3.2.5.3 Riferimenti nel progetto:

Il pattern Singleton è utilizzato per il coordinamento dei componenti di sessione:

- Diari Personali → DiarySession



3.2.6 Dependency Injection e Service Locator

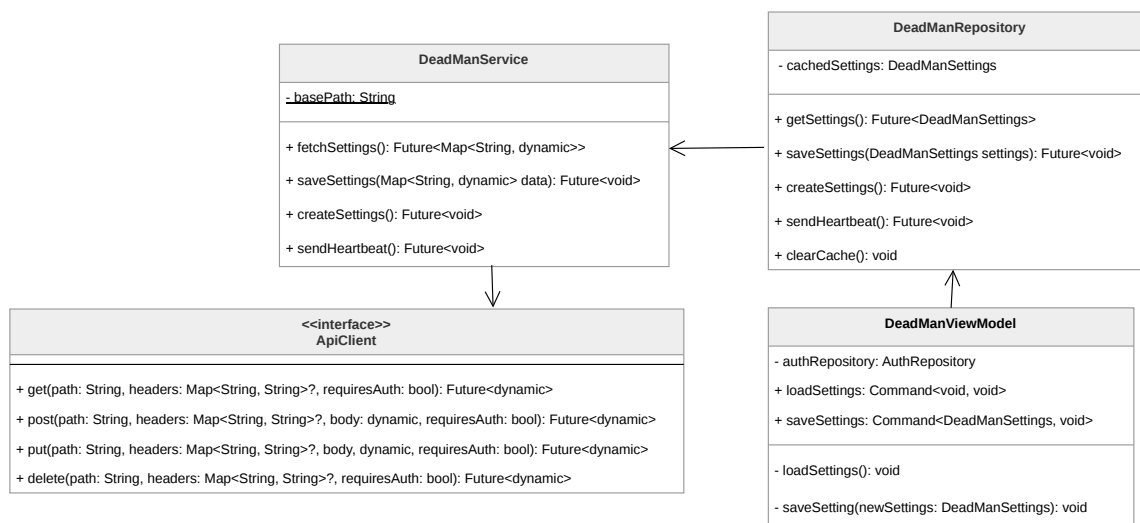


Figura 6: Rappresentazione del pattern Dependency Injection nel modulo del Dead Man' switch

3.2.6.1 Descrizione del pattern:

Il sistema adotta il pattern Dependency Injection (DI) supportato dal pattern **Service Locator**, implementato tramite la libreria `get_it`. Questo approccio permette di separare la creazione delle dipendenze dal loro effettivo utilizzo.

L'Architettura si basa su un punto di configurazione centralizzato (`setupLocator`) dove vengono registrate tutte le istanze delle classi d'interesse. La DI viene poi attuata tramite *Constructor Injection*: i componenti (come i ViewModel) non creano mai i propri Repository o Service internamente, ma dichiarano le dipendenze nel proprio costruttore. È il Service Locator a farsi carico di "iniettare" l'istanza corretta al momento della creazione, prelevandola dal registro globale.

3.2.6.2 Ragioni per l'utilizzo:

L'integrazione di `get_it` e della DI offre tre vantaggi fondamentali per l'applicazione:

- **Gestione del ciclo di vita:** Permette di differenziare tra *Singletons* (istanze uniche come `DeadManRepository` o `ApiClient`), che mantengono lo stato per tutta la durata dell'app, e *Factories* (come i ViewModel), che vengono creati ex-novo ogni volta che si accede a una schermata per garantire uno stato pulito.
- **Lazy Loading:** Grazie alla registrazione di tipo *Lazy*, alcune risorse (es. `MaterialService`) vengono istanziate solo nel momento del primo effettivo utilizzo, ottimizzando il consumo di memoria e i tempi di avvio del software.
- **Testabilità e Mocking:** Centralizzare la creazione dei componenti permette di sostituire istantaneamente le implementazioni reali con versioni fittizie durante i



test, semplicemente modificando la registrazione nel locator senza toccare il codice della View o la logica di business.

3.2.6.3 Riferimenti nel progetto:

Il pattern è applicato diffusamente in tutta l'applicazione. Di seguito alcune classi che integrano attivamente questo approccio:

- Autenticazione → **AuthViewModel**
- Chatbot → **ChatbotViewModel**
- Contatti Fidati → **TrustedContactViewModel**

3.2.7 Adapter

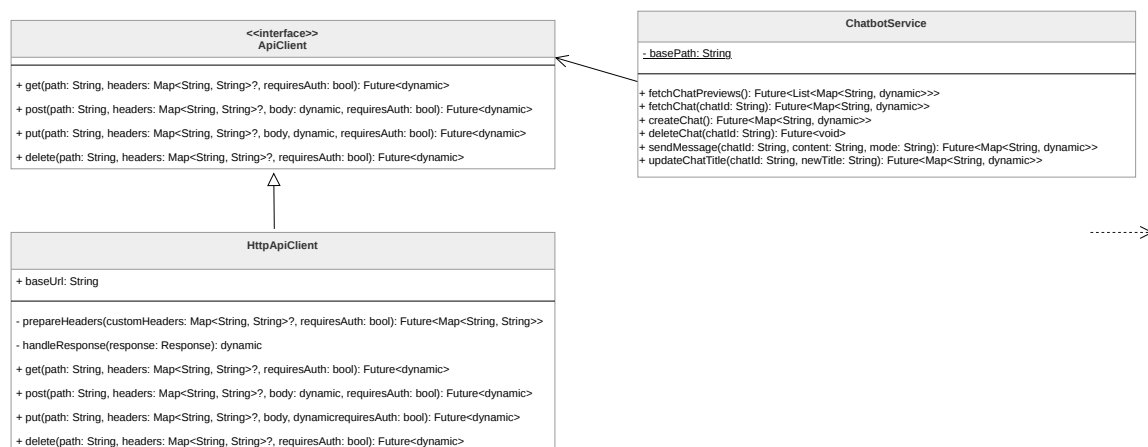


Figura 7: Rappresentazione del pattern Adapter all'ApiClient

3.2.7.1 Descrizione del pattern:

Il pattern Adapter è un design pattern strutturale che permette a interfacce incompatibili di lavorare insieme. Agisce come un convertitore che traduce le chiamate del client nel formato compreso da una classe o libreria esterna. Nell'Architettura del sistema, questo pattern è fondamentale per implementare i principi dell'Architettura Esagonale, dove il "core" dell'applicazione definisce dei contratti (Porte) e gli Adapter forniscono l'implementazione concreta per interfacciarsi con il mondo esterno.

3.2.7.2 Ragioni per l'utilizzo:

L'integrazione di questo pattern permette di isolare il codice sorgente dalle dipendenze di terze parti. Utilizzando un Adapter per la comunicazione di rete o per i servizi di sistema, si evita di "inquinare" i Service o i Repository con dettagli implementativi di librerie specifiche. Se in futuro si decidesse di cambiare il client HTTP (ad esempio passando dalla libreria **http** a **dio**) o il fornitore di un servizio Cloud, basterebbe creare un nuovo



Adapter che rispetti l'interfaccia predefinita, senza dover modificare una singola riga di logica di business.

3.2.7.3 Riferimenti nel progetto:

Il pattern è applicato in modo sistematico nella gestione dell'infrastruttura di rete e dei servizi hardware del dispositivo, oltre che nelle AWS Lambda che utilizzano l'Architettura esagonale:

- NetworkAdapter → **ApiClient**
- Location Service → **LocationService**

3.3. Architettura di deployment

3.3.1 Architettura a microservizi

Per l'architettura logica viene adottato il pattern architetturale **microservizi**. In questo modello l'applicazione è realizzata da componenti indipendenti, ciascuno responsabile di un sottoinsieme ben definito di funzionalità. Ogni componente è deployabile, aggiornabile e scalabile in modo autonomo rispetto agli altri.

Nel progetto ogni microservizio corrisponde a una singola funzione AWS Lambda, deployata tramite AWS SAM. Questa corrispondenza elimina la necessità dell'implementazione delle API per la comunicazione tra Lambda diverse che si introdurrebbe invocando Lambda distinte per ogni operazione, e riduce i costi di esecuzione in quanto AWS deve andare ad inizializzare una sola Lambda per microservizio. Ogni microservizio gestisce inoltre un proprio layer di persistenza dedicato, senza condivisione di istanze di database tra servizi distinti.

Per evitare l'accoppiamento tra microservizi, ogni Lambda interagisce esclusivamente con le proprie istanze dedicate di DynamoDB o S3, senza condivisione di risorse di persistenza tra servizi distinti. Questa scelta è visibile nei diagrammi dell'Architettura di ciascun microservizio (si veda [Sezione 3.6](#)).

Il ridimensionamento è gestito in modo automatico da AWS Lambda in risposta al volume di richieste, senza intervento manuale e senza costi per capacità inutilizzata.

3.4. Infrastruttura

L'infrastruttura del sistema è ospitata interamente su **Amazon Web Services (AWS)**. In questo modello il provisioning, il bilanciamento del carico, la scalabilità e la gestione dell'hardware sottostante sono demandati al Cloud provider, riducendo la necessità di



gestione da parte del team di sviluppo.

Questa scelta risulta coerente con l'Architettura three-tier descritta in precedenza e con l'adozione del pattern a microservizi, in quanto consente di distribuire componenti indipendenti che scalano automaticamente in funzione del carico. L'infrastruttura Serverless introduce inoltre i seguenti vantaggi:

- **Scalabilità dinamica:** le risorse computazionali vengono allocate automaticamente da AWS in base al numero di richieste ricevute riducendo i costi dovuti a capacità inutilizzata;
- **Isolamento dei servizi:** ogni microservizio viene eseguito in modo indipendente tramite una funzione AWS Lambda dedicata, con risorse di persistenza e permessi separati rispetto agli altri servizi;
- **Sicurezza:** l'accesso alle risorse AWS è regolato tramite policy IAM, limitando ogni Lambda esclusivamente alle operazioni necessarie al proprio dominio applicativo;
- **Riduzione dell'overhead operativo (Zero Ops):** la gestione dell'infrastruttura fisica, degli aggiornamenti di sicurezza, della disponibilità dei servizi e della replica dei dati è demandata ad AWS.

L'infrastruttura applicativa è organizzata attraverso i seguenti componenti principali:

- **Amazon API Gateway:** costituisce il punto di ingresso del sistema lato Backend ed espone gli endpoint HTTPS consumati dal client Flutter. Il servizio si occupa di ricevere le richieste HTTP, validarle e instradarle verso la funzione AWS Lambda associata;
- **Amazon Cognito:** gestisce l'autenticazione e l'autorizzazione degli utenti attraverso token JWT validati da API Gateway prima dell'invocazione delle Lambda protette;
- **AWS Lambda:** implementa il logic tier dell'applicazione secondo il pattern a microservizi. Ogni Lambda contiene la Business Logic relativa a uno specifico dominio funzionale ed esegue il codice in ambienti di esecuzione creati dinamicamente da AWS;
- **Amazon DynamoDB:** fornisce la persistenza dei dati strutturati tramite database NoSQL;
- **Amazon S3:** viene utilizzato per l'archiviazione dei contenuti multimediali, quali immagini e file audio.



3.4.1 Infrastructure as Code (IaC)

L'infrastruttura Cloud viene gestita tramite il paradigma dell'Infrastructure as Code (IaC). Tutte le risorse AWS necessarie al funzionamento del sistema vengono descritte tramite file YAML versionati all'interno del Repository del progetto.

Per la gestione dell'infrastruttura viene utilizzato **AWS SAM (Serverless Application Model)**, Framework sviluppato da AWS e basato su **AWS CloudFormation**. AWS SAM introduce una sintassi semplificata specifica per applicazioni Serverless, consentendo di definire in modo compatto risorse quali:

- funzioni AWS Lambda;
- endpoint API Gateway;
- bucket Amazon S3;
- tabelle Amazon DynamoDB;
- variabili d'ambiente e configurazioni di Deployment.

3.5. Architettura Front End

3.5.1 Autenticazione

La funzionalità di autenticazione è stata implementata seguendo il pattern architetturale MVVM raccomandato dalle linee guida di Flutter, adattato al flusso OAuth 2.0 con OpenID Connect descritto nelle specifiche. La struttura separa nettamente le responsabilità tra i layer: il Service gestisce la comunicazione con gli endpoint di AWS Cognito e il flusso OAuth, il repository mantiene lo stato dell'Utente autenticato e orchestra la traduzione dei dati grezzi in oggetti di dominio tramite i DTO, il ViewModel espone lo stato e le azioni alla View tramite i Command del pattern MVVM. Per la modellazione di un Utente viene utilizzato il domain model **User**.

L'istanza unica dell'Utente autenticato viene gestita tramite il Provider di Flutter a livello di app per mantenere una sola istanza attiva per tutta la durata della sessione.

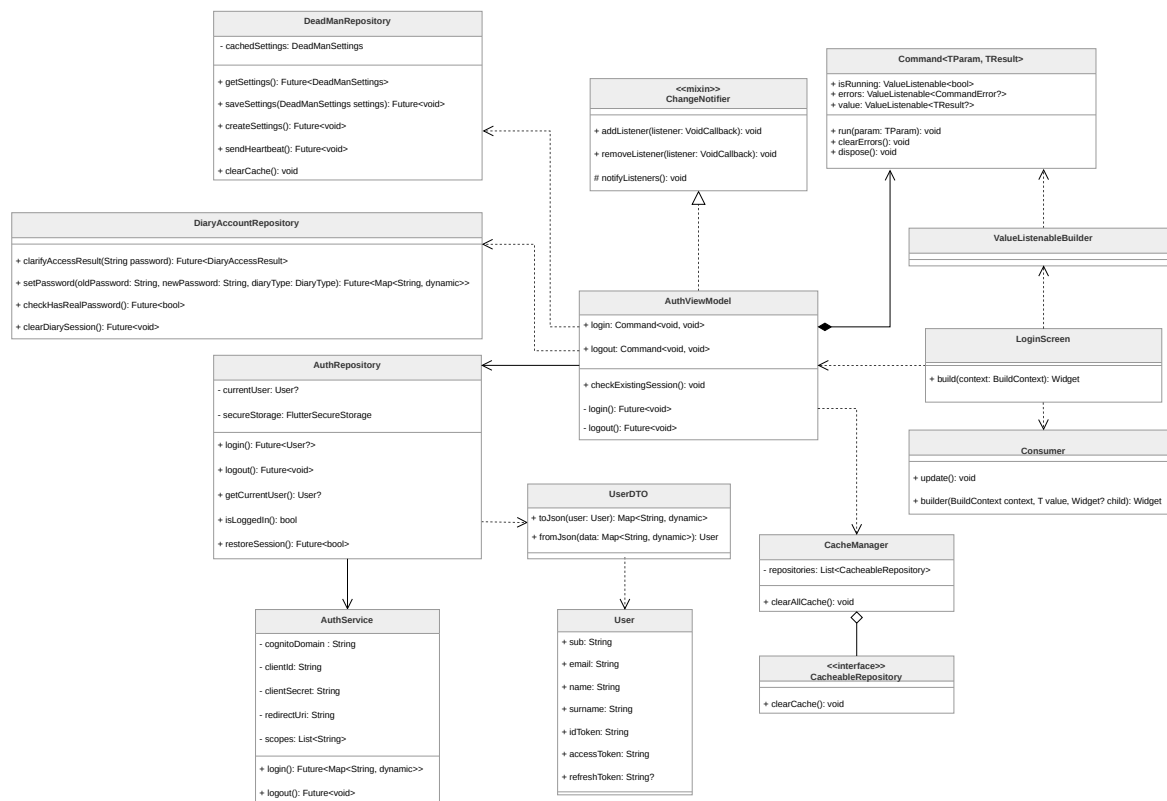


Figura 8: Diagramma delle classi relativo all'autenticazione

3.5.1.1 LoginScreen

Widget principale che rappresenta la schermata di autenticazione dell'applicazione. Nel contesto del pattern MVVM, svolge il ruolo di View, occupandosi esclusivamente della costruzione dell'interfaccia Utente tramite il metodo `build(BuildContext context)`. Mantiene un riferimento al `AuthViewModel`, dal quale osserva lo stato dell'autenticazione (avvalendosi di componenti come `ValueListenableBuilder` e `Consumer`) e a cui delega le azioni dell'Utente.

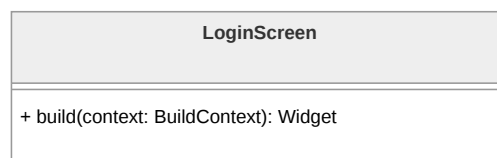
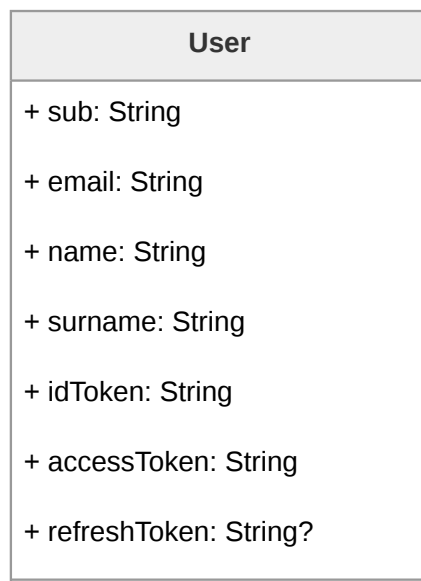


Figura 9: Diagramma della classe `LoginScreen`

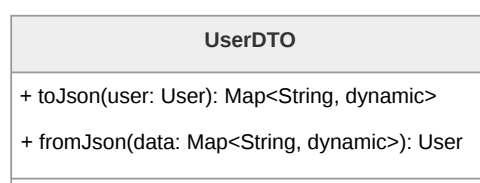
3.5.1.2 User

Rappresenta il modello di dominio dell'Utente autenticato all'interno dell'applicazione. Contiene le informazioni essenziali legate all'identità e alla sessione, come l'identificativo univoco e i token di accesso, ed è utilizzata dal resto del sistema per gestire lo stato dell'autenticazione in modo indipendente dal formato dei dati provenienti dall'esterno.

Figura 10: Diagramma della classe **User**

3.5.1.3 UserDTO

Ha il compito di gestire la conversione tra il modello di dominio **User** e la rappresentazione serializzata dei dati. Attraverso i metodi `toJson` e `fromJson`, consente rispettivamente di trasformare un oggetto **User** in formato JSON e di ricostruirlo a partire da dati serializzati.

Figura 11: Diagramma della classe **UserDTO**

3.5.1.4 AuthViewModel

Gestisce lo stato dell'autenticazione e coordina le operazioni richieste dalla View interagendo con il **AuthRepository**. Le funzionalità principali, come il login e il logout, sono esposte tramite il Command pattern. La logica concreta è incapsulata nei metodi che vengono passati come azioni ai rispettivi Command, mentre il metodo `checkExistingSession()` consente di verificare la presenza di una sessione già attiva al momento dell'avvio. Inoltre, funge da orchestratore: al login si interfaccia con il **DeadManRepository** per inizializza-



re le impostazioni di sicurezza, mentre al logout coordina la pulizia profonda dei dati interagendo con **CacheManager** e **DiaryAccountRepository**.

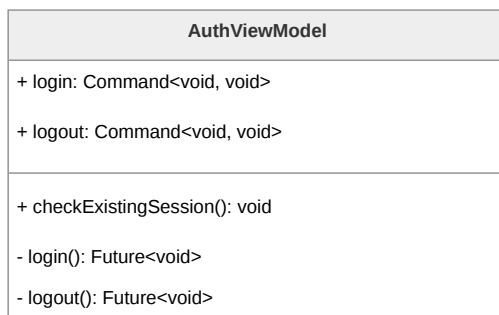


Figura 12: Diagramma della classe **AuthViewModel**

3.5.1.5 AuthRepository

Funge da intermediario tra il **AuthViewModel** e il livello di accesso ai dati, incapsulando la logica necessaria per gestire l'autenticazione. Mantiene lo stato dell'Utente corrente tramite **currentUser**, gestisce la persistenza sicura dei token tramite **FlutterSecureStorage** e delega le operazioni di comunicazione remota all'**AuthService**. Il Repository espone metodi per effettuare il login, il logout, recuperare l'Utente corrente e verificare o ripristinare una sessione attiva.

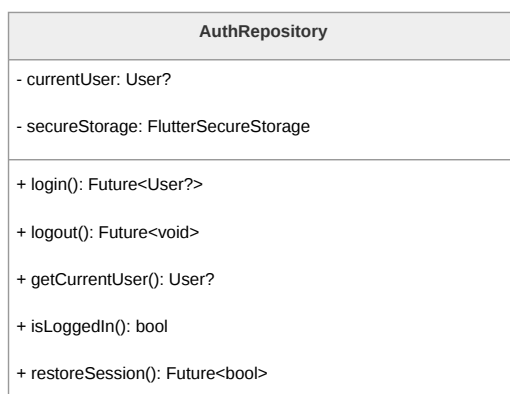
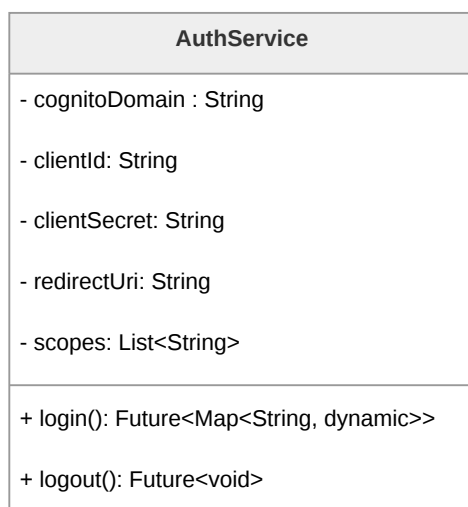


Figura 13: Diagramma della classe **AuthRepository**

3.5.1.6 AuthService

È responsabile della comunicazione con il sistema di autenticazione esterno AWS Cognito. Contiene le informazioni di configurazione necessarie, tra cui dominio, credenziali del client, URI di redirect e scope richiesti. Si occupa di eseguire le chiamate di rete per il login e il logout, delegando l'interazione e l'apertura del browser di sistema al pacchetto **flutter_web_auth_2**.

Figura 14: Diagramma della classe **AuthService**

3.5.2 Chatbot

La funzionalità del chatbot permette all'utente di comunicare con un LLM e di gestire lo storico delle proprie conversazioni. Il sistema è sviluppato seguendo il pattern **MVVM**, che assicura una chiara separazione tra la logica di stato e la visualizzazione dei widget. Per la gestione e l'ottimizzazione del caricamento della cronologia è stato integrato il pattern strutturale **Proxy**. Il sistema distingue nettamente un Data Layer (Service, Repository, API Client), un Domain Layer (interfaccia **Chat**, **ProxyChat**, **LocalChat**, **ChatMessage**) e un UI Layer (View e ViewModel).

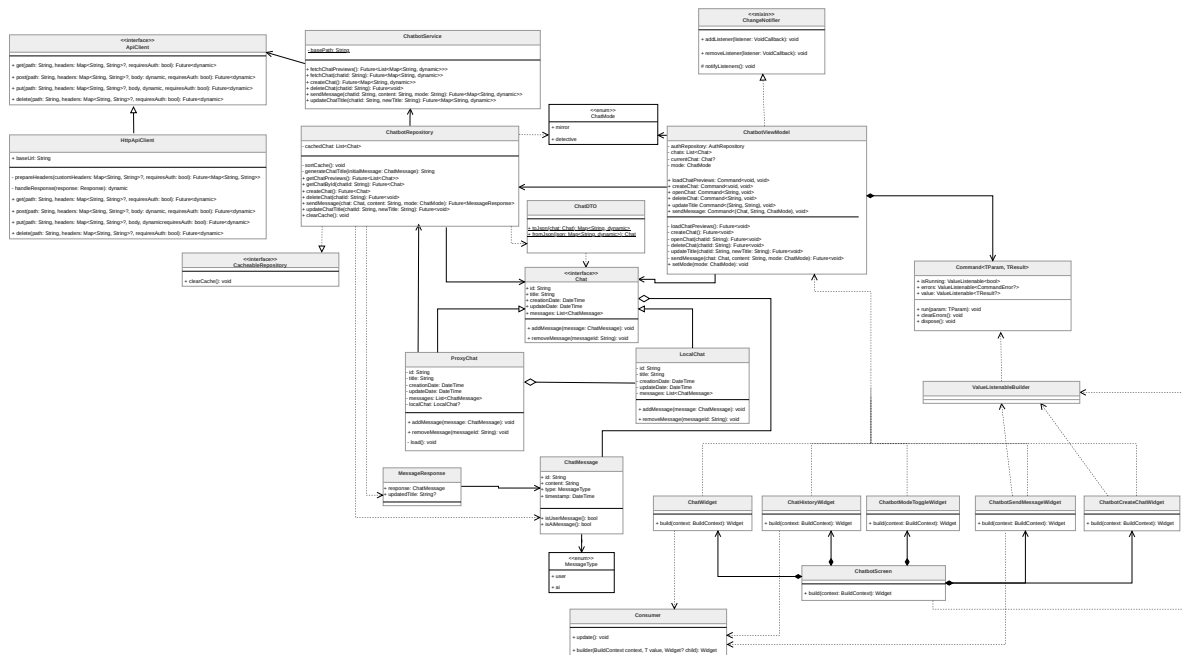


Figura 15: Diagramma delle classi relativo alle funzionalità del Chatbot

3.5.2.1 ChatbotScreen

Rappresenta la schermata principale del Chatbot all'interno del *Presentation Layer*. Funge da orchestratore visivo, componendo widget specializzati come **ChatWidget**, **ChatHistoryWidget** e **ChatbotSendMessageWidget**. Contiene esclusivamente la logica di rendering necessaria per disaccoppiare la presentazione dalla logica di business gestita dal ViewModel.

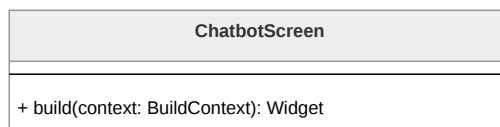


Figura 16: Diagramma della classe **ChatbotScreen**

3.5.2.2 ChatWidget

La classe **ChatWidget** è responsabile della visualizzazione della conversazione attiva. Utilizza internamente il widget **Consumer** per osservare il **ChatbotViewModel** e reagire ai cambiamenti dello stato applicativo, garantendo l'aggiornamento automatico della lista dei messaggi e dei feedback di caricamento.

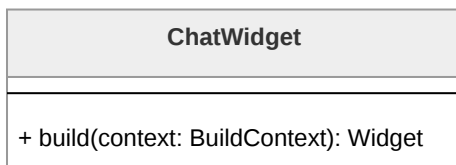


Figura 17: Diagramma della classe `ChatWidget`

3.5.2.3 ChatHistoryWidget

`ChatHistoryWidget` mostra l'elenco delle anteprime delle conversazioni passate. Grazie all'utilizzo del pattern **Proxy**, il widget è in grado di visualizzare i metadati delle chat (titolo, data) senza dover caricare l'intera cronologia dei messaggi. Tramite un `Consumer`, aggiorna la vista ogni volta che la lista delle anteprime nel ViewModel viene aggiornata.

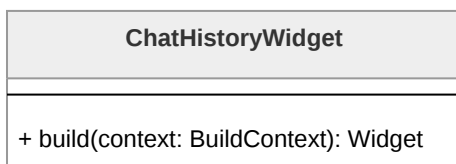


Figura 18: Diagramma della classe `ChatHistoryWidget`

3.5.2.4 ChatbotModeToggleWidget

Consente all'Utente di selezionare il contesto operativo del chatbot (`ChatMode`). Le modalità disponibili, "Specchio" e "Detective", influenzano il *system prompting* applicato lato Backend durante la generazione della risposta da parte di AWS Bedrock.



Figura 19: Diagramma della classe `ChatbotModeToggleWidget`

3.5.2.5 ChatbotSendMessageWidget

Gestisce l'input testuale e l'invio dei messaggi. Delega l'azione al ViewModel invocando il relativo Command. Il widget utilizza `ValueListenableBuilder` per monitorare lo stato di esecuzione dell'invio e mostrare un indicatore di caricamento (rotellina) durante l'attesa della risposta sincrona dal LLM.

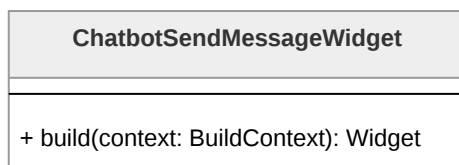


Figura 20: Diagramma della classe `ChatbotSendMessageWidget`

3.5.2.6 ChatbotCreateChatWidget

Widget dedicato all'avvio di nuove sessioni di chat. Comunica l'intenzione al ViewModel, che prepara lo stato per una nuova conversazione gestendo la creazione in modo *lazy*: la chat viene effettivamente creata sul server solo all'invio del primo messaggio.

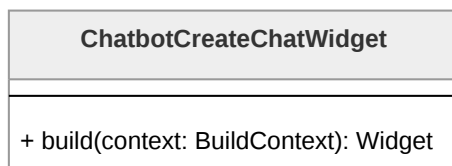


Figura 21: Diagramma della classe `ChatbotCreateChatWidget`

3.5.2.7 ChatbotViewModel

`ChatbotViewModel` è il nucleo della logica di presentazione. Orchestrando i vari **Command** (`sendMessage`, `openChat`, `loadChatPreviews`, `deleteChat`), gestisce la reattività della View e mantiene in memoria la modalità operativa (`ChatMode`). Implementa il mixin `ChangeNotifier` per notificare i widget osservatori ad ogni aggiornamento dello stato.

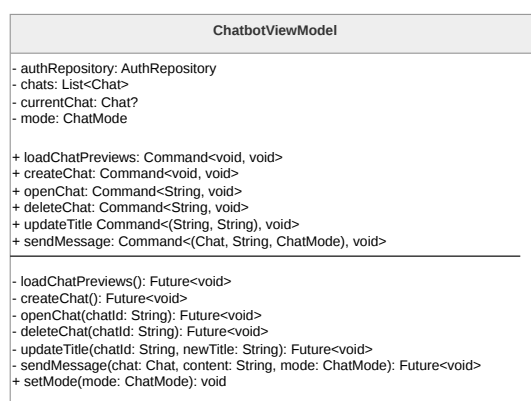


Figura 22: Diagramma della classe `ChatbotViewModel`

3.5.2.8 Chat



L'interfaccia **Chat** definisce il contratto per una conversazione tra Utente e il chatbot. Insieme a **ProxyChat** e **LocalChat**, è utilizzata nell'implementazione del pattern Proxy.

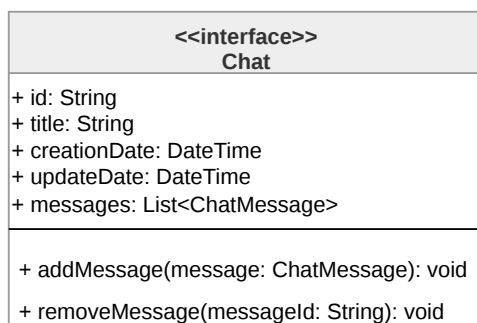


Figura 23: Diagramma della classe **Chat**

3.5.2.9 ProxyChat

Implementazione del pattern Proxy che agisce come segnaposto per una conversazione storica. Contiene solo i metadati essenziali e implementa il metodo **load()**, che interagisce con il Repository per istanziare una **LocalChat** e caricarne i messaggi solo quando l'Utente decide di aprire effettivamente la conversazione (*Lazy Loading*).

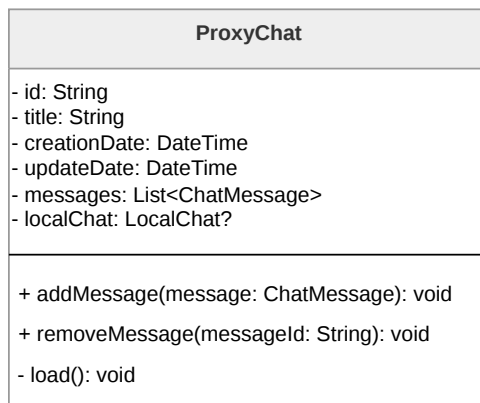


Figura 24: Diagramma della classe **ProxyChat**

3.5.2.10 LocalChat

Rappresenta l'oggetto reale nel pattern Proxy. Mantiene l'elenco completo dei **ChatMessage** scambiati e fornisce i metodi per la manipolazione della cronologia in memoria RAM durante la sessione attiva.

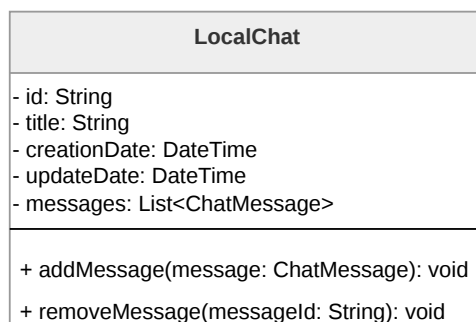


Figura 25: Diagramma della classe **LocalChat**

3.5.2.11 ChatMessage

Classe di dominio che modella il singolo messaggio, specificando il contenuto, il timestamp e il ruolo dell'emittente tramite l'enumerazione **MessageType**.

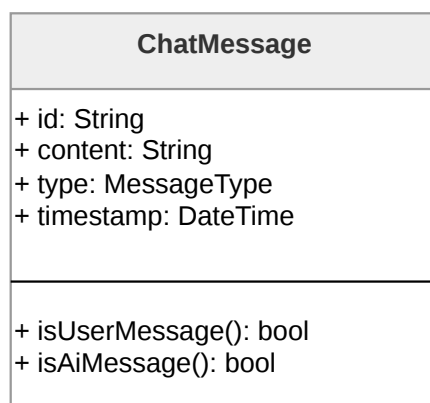


Figura 26: Diagramma della classe **ChatMessage**

3.5.2.12 ChatMode

ChatMode è un'enumerazione che rappresenta le due diverse modalità operative del chatbot. Viene utilizzato da **ChatbotViewModel**, che lo inoltra a **ChatbotRepository**, il quale lo utilizza per invocare i metodi del **ChatbotService** relativi all'invio dei messaggi in chat. Sarà poi il Backend a interpretarlo e utilizzarlo per generare la risposta al messaggio inviato dall'Utente.

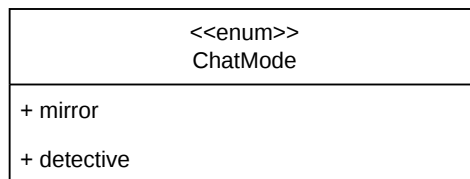


Figura 27: Diagramma della classe **ChatMode**

3.5.2.13 MessageType

3.5.2.14 MessageResponse

Incapsula la risposta prodotta dall'LLM lato server. Contiene il corpo del messaggio e i metadati necessari per aggiornare la conversazione corrente nel ViewModel.

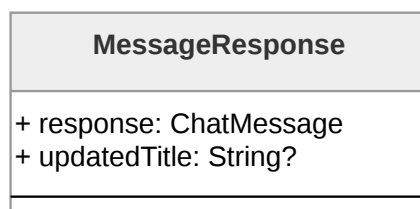


Figura 28: Diagramma della classe **MessageResponse**

3.5.2.15 ChatbotRepository

Agisce come orchestratore dei dati del chatbot, implementando una cache **in-memory** (**cachedChats**) per ottimizzare l'accesso alle sessioni di conversazione e garantire la sicurezza dei dati, evitando la persistenza locale di informazioni sensibili. La classe gestisce il recupero delle informazioni tramite il **ChatbotService** e utilizza **ChatDTO** per la mappatura e la trasformazione dei dati grezzi provenienti dal backend nei modelli di dominio utilizzati dall'applicazione.

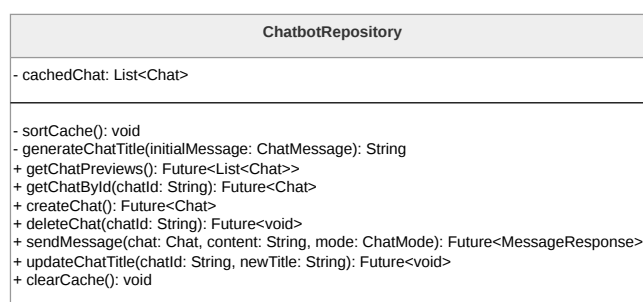


Figura 29: Diagramma della classe **ChatbotRepository**



3.5.2.16 ChatbotService

Responsabile della comunicazione con le API di AWS. Si occupa di invocare gli endpoint per il recupero delle chat, l'invio dei messaggi e la gestione della cronologia, restituendo risposte grezze al Repository.

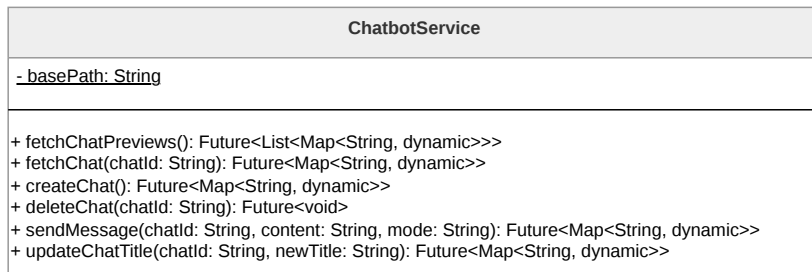


Figura 30: Diagramma della classe **ChatbotService**

3.5.2.17 ApiClient

Interfaccia astratta che definisce il Contratto per le operazioni HTTP (**get**, **post**, **put**, **delete**). Astrae il client di rete permettendo una facile sostituzione o simulazione (mocking) del livello di comunicazione.

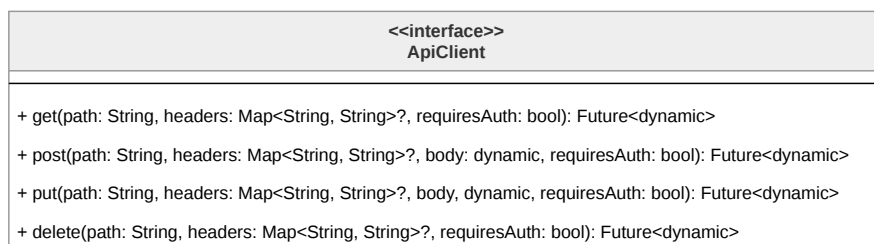


Figura 31: Diagramma dell'interfaccia **ApiClient**

3.5.2.18 HttpApiClient

Implementazione concreta dell'interfaccia **ApiClient**. Gestisce la comunicazione effettiva tramite il pacchetto **http**, occupandosi dell'inserimento dei token di autenticazione negli header di ogni richiesta e della gestione globale delle eccezioni di rete.

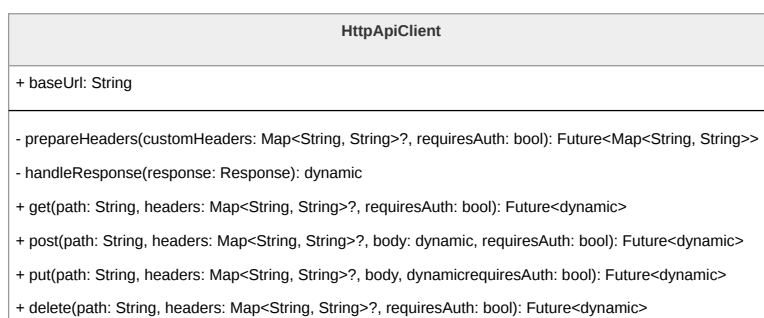


Figura 32: Diagramma della classe **HttpApiClient**



3.5.2.19 ChatDTO

Gestisce la traduzione tra i modelli di dominio e le rappresentazioni JSON scambiate tramite l'API. Isola il resto del sistema dai dettagli del formato di serializzazione esterno.

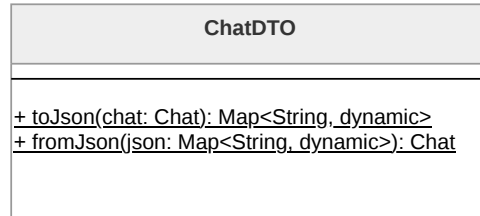


Figura 33: Diagramma della classe ChatDTO

3.5.3 Diari

La funzionalità di diario criptato e diario fittizio permette all'Utente di utilizzare due archivi separati in cui memorizzare note, contenenti testo, immagini e/o contenuti audio, protetti da password. L'implementazione della funzionalità segue il pattern MVVM e le linee guida del Framework Flutter, risultanti nella suddivisione tra un Data Layer, composto da classi Service e repository, responsabili per la *Business Logic* e per la *Persistence Logic* rispettivamente, ed un UI Layer, composto da classi View e Viewmodel. Gli oggetti principali della funzionalità sono le classi Note (ProxyNote e LocalNote), NoteElement e DiarySession.

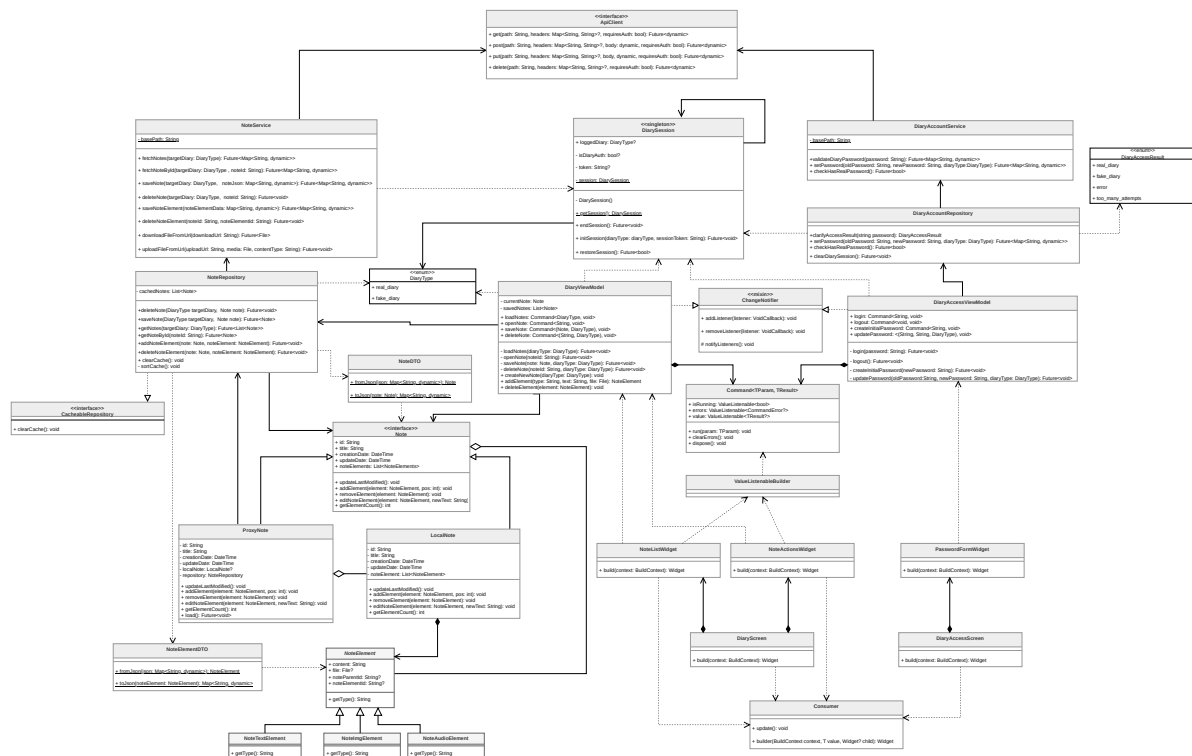


Figura 34: Diagramma delle classi relativo alle funzionalità dei Diari



3.5.3.1 DiaryAccessScreen

Gestisce la rappresentazione visiva della schermata di accesso ai diari nel *Presentation Layer*. Contiene la logica minima di rendering e compone widget specializzati come **PasswordFormWidget**. La gestione dello stato di autenticazione è delegata a **DiaryAccessViewModel**.

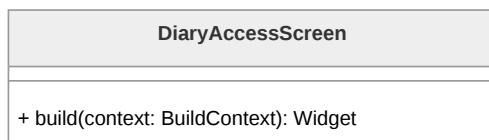


Figura 35: Diagramma della classe **DiaryAccessScreen**

3.5.3.2 PasswordFormWidget

Responsabile della visualizzazione dei campi di input per la password. In quanto widget di tipo **Consumer**, osserva il **DiaryAccessViewModel** per reagire a tentativi di accesso falliti, errori di rete o stati di blocco temporaneo dovuto al superamento del limite di tentativi.

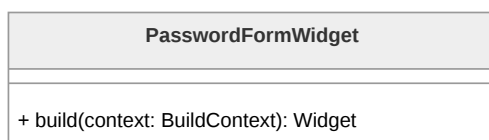


Figura 36: Diagramma della classe **PasswordFormWidget**

3.5.3.3 DiaryAccessViewModel

Coordina le operazioni di autenticazione tra UI e Data Layer. Implementa il Command **login** per validare le credenziali tramite il Repository. Una volta ottenuto l'accesso, comunica al sistema i dettagli relativi alla sessione attiva, tra cui la tipologia di diario aperto e il token di sessione. Essendo un **ChangeNotifier**, notifica i widget interessati ad ogni cambiamento di stato.

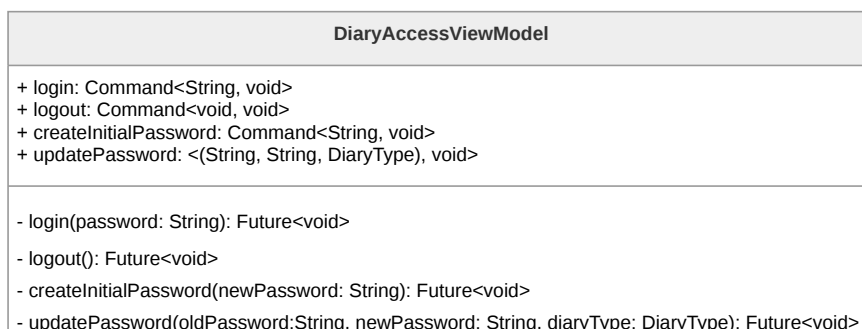


Figura 37: Diagramma della classe **DiaryAccessViewModel**



3.5.3.4 DiaryAccountRepository

Funge da strato di astrazione per la gestione dell'account del diario. Si occupa di invocare il metodo `clarifyAccessResult` per validare la password tramite il Service e trasforma le risposte grezze in oggetti `DiaryAccessResult`. È responsabile dell'inizializzazione della `DiarySession` globale in caso di successo.

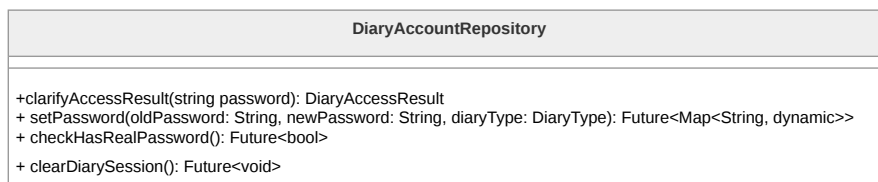


Figura 38: Diagramma della classe `DiaryAccountRepository`

3.5.3.5 DiaryAccountService

Rappresenta lo strato infrastrutturale responsabile della comunicazione con le API remote per la Validazione delle password e la gestione dei parametri di sicurezza del diario. Restituisce dati grezzi al Repository per la successiva elaborazione.

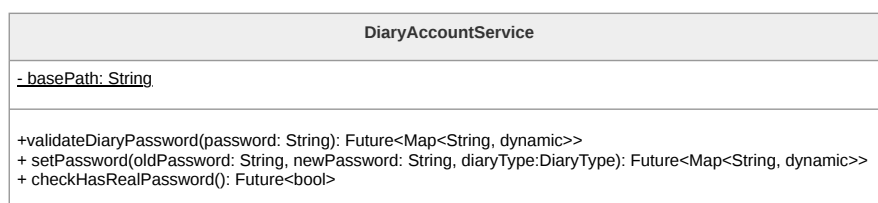


Figura 39: Diagramma della classe `DiaryAccountService`

3.5.3.6 DiaryAccessResult

L'enumerazione `DiaryAccessResult` permette la classificazione della risposta ad un tentativo di accesso ai diari da parte dell'Utente, distinguendo tra accesso avvenuto con successo ad uno dei due tipi di diario e accesso fallito con errore.

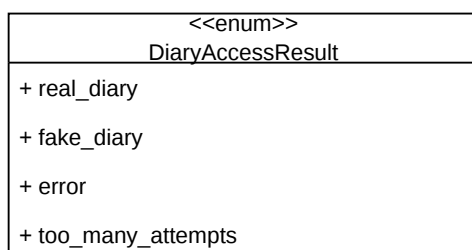


Figura 40: Diagramma della classe `DiaryAccessResult`



3.5.3.7 DiaryScreen

Gestisce l'interfaccia principale del diario sbloccato. Per garantire la sicurezza dell'Utente, la struttura visiva è identica per entrambi i tipi di diario, includendo i widget **NoteListWidget** e **NoteActionsWidget**. La gestione dello stato operativo è delegata a **DiaryViewModel**.

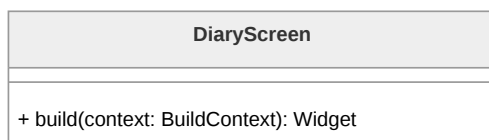


Figura 41: Diagramma della classe **DiaryScreen**

3.5.3.8 NoteListWidget

Visualizza l'elenco delle note caricate per la sessione corrente. Utilizza il **Consumer** per osservare il **DiaryViewModel** e aggiornare la lista in tempo reale a seguito di aggiunte o eliminazioni. Supporta il caricamento lazy delle note grazie al pattern Proxy.

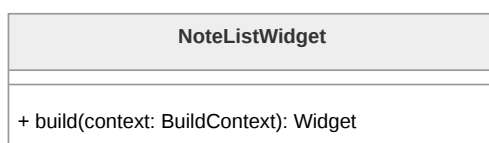


Figura 42: Diagramma della classe **NoteListWidget**

3.5.3.9 NoteActionsWidget

Espone le interazioni per la creazione di nuove note o l'attivazione della modalità di editing. Delega l'esecuzione dei comandi al ViewModel, mantenendo il disaccoppiamento tra UI e dominio.

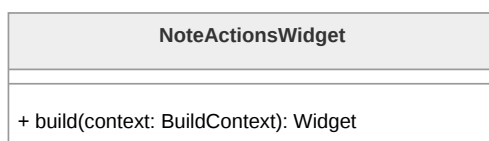


Figura 43: Diagramma della classe **NoteActionsWidget**

3.5.3.10 NoteEditorWidget

Componente responsabile dell'interfaccia di visualizzazione e modifica di una nota. Permette la manipolazione granulare dei singoli **NoteElement** (testo, audio e immagini) attraverso l'integrazione di widget specializzati per ogni tipologia di contenuto. Il widget interagisce con il ViewModel per gestire la sincronizzazione dei dati, assicurando che ogni



modifica apportata dall'utente venga correttamente processata e riflessa nell'interfaccia grafica in modo fluido.

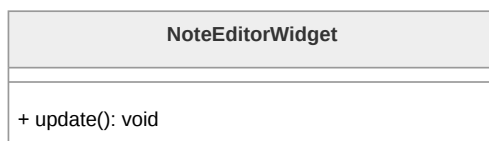


Figura 44: Diagramma della classe **NoteEditorWidget**

3.5.3.11 DiaryViewModel

Nucleo della logica di presentazione per la gestione del diario. Mantiene in memoria la lista delle note correnti ed espone i comandi CRUD (**addNote**, **updateNote**, **deleteNote**). Utilizza il **NoteRepository** per la persistenza dei dati e dei file multimediali, notificando la UI tramite **ChangeNotifier**.

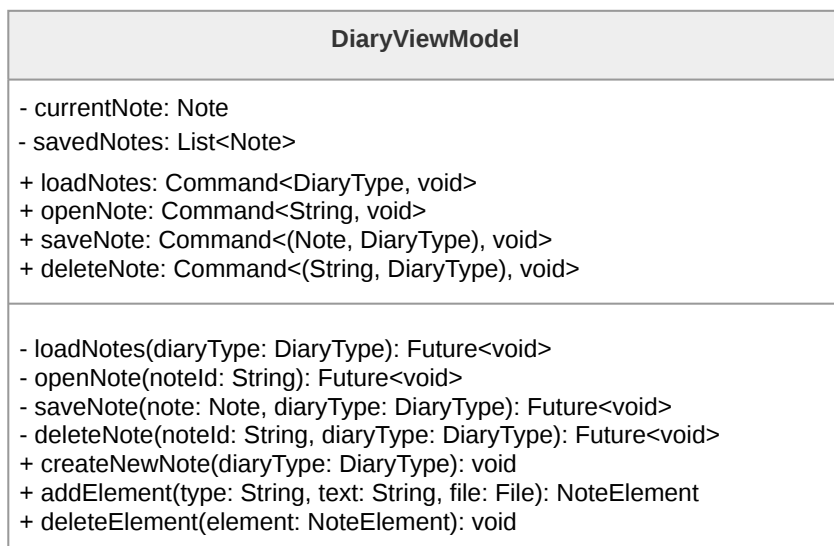


Figura 45: Diagramma della classe **DiaryViewModel**

3.5.3.12 DiarySession

Classe Singleton responsabile del mantenimento dello stato della sessione attiva. Tiene traccia del **DiaryType** e del token di accesso, garantendo che le informazioni sensibili vengano perse alla chiusura dell'applicazione per massimizzare la sicurezza.

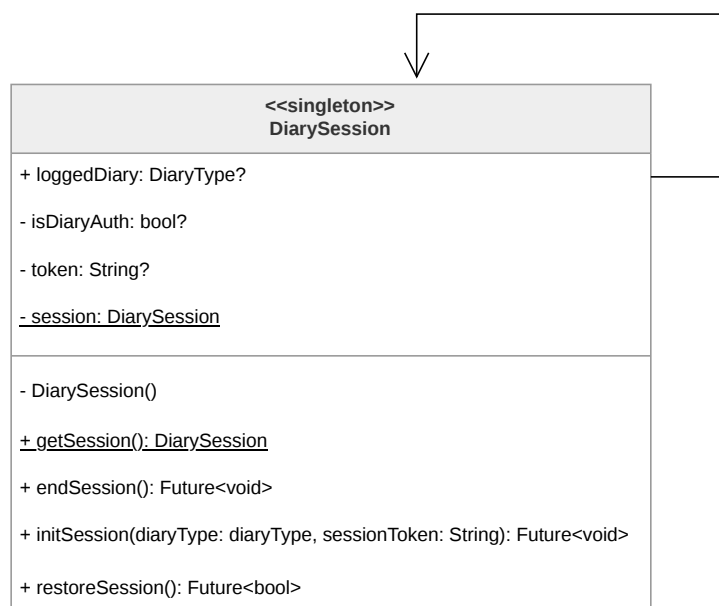


Figura 46: Diagramma della classe **DiarySession**

3.5.3.13 DiaryType

Enumerazione che definisce le due tipologie di archivio disponibili: **fake_diary** e **real_diary**. È un parametro fondamentale utilizzato per instradare correttamente le richieste verso gli endpoint del server.

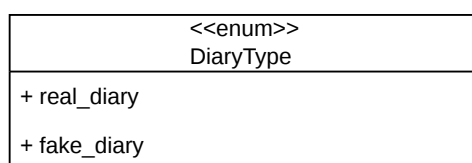


Figura 47: Diagramma della classe **DiaryType**

3.5.3.14 Note

Interfaccia che definisce il Contratto per una nota di diario. Supporta il pattern Proxy per ottimizzare le risorse di sistema durante la visualizzazione di elenchi di note voluminosi.

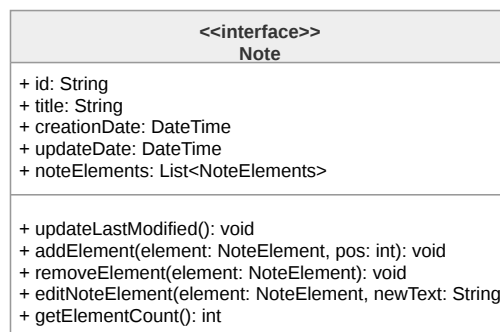


Figura 48: Diagramma della classe **Note**

3.5.3.15 ProxyNote

Realizza il pattern Proxy ritardando il caricamento completo della cronologia degli elementi di una nota. Mantiene i metadati di base e delega le operazioni a un oggetto **LocalNote** solo in caso di accesso effettivo ai contenuti.

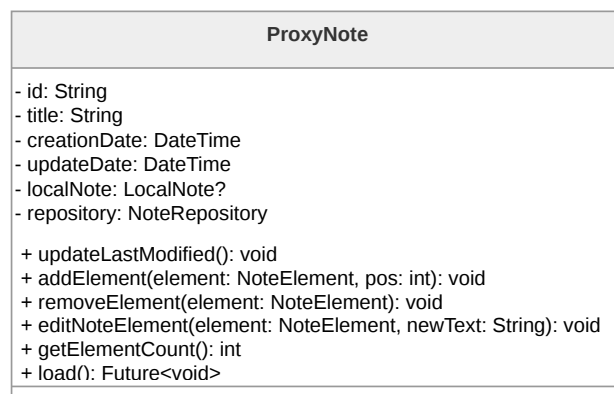


Figura 49: Diagramma della classe **ProxyNote**

3.5.3.16 LocalNote

Implementazione reale dell'interfaccia **Note**. Rappresenta l'oggetto caricato in memoria che contiene la lista polimorfica di **NoteElement** che compongono il corpo della nota.

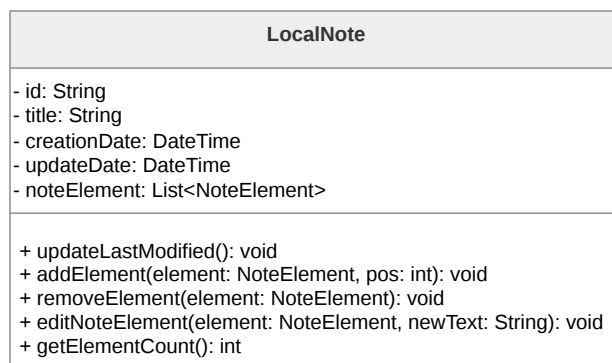


Figura 50: Diagramma della classe **LocalNote**

3.5.3.17 NoteElement

Classe base astratta che modella un blocco di contenuto all'interno della nota. Definisce il metodo **getType()** fondamentale per il rendering e la serializzazione polimorfica degli elementi.

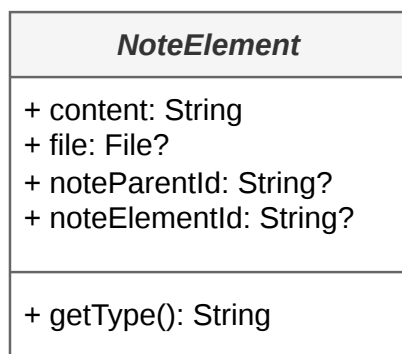


Figura 51: Diagramma della classe **NoteElement**

3.5.3.18 NoteTextElement

Specializzazione di **NoteElement** per la gestione di contenuti testuali all'interno dell'editor a blocchi.

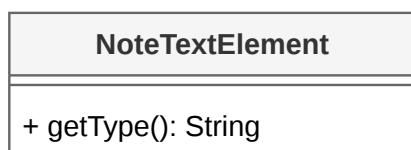


Figura 52: Diagramma della classe **NoteTextElement**



3.5.3.19 NoteImageElement

Specializzazione di **NoteElement** dedicata ai contenuti grafici. La classe gestisce il riferimento al file locale presente sul dispositivo per consentire la visualizzazione istantanea dell'immagine dopo la selezione; allo stesso tempo, mantiene il collegamento alla risorsa remota tramite l'URL generato a seguito del caricamento sul server.

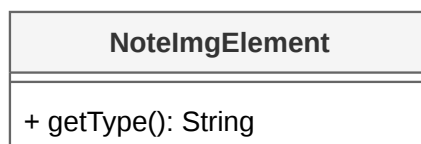


Figura 53: Diagramma della classe **NoteImageElement**

3.5.3.20 NoteAudioElement

Specializzazione di **NoteElement** per i messaggi vocali e le registrazioni audio, contenente i metadati necessari alla riproduzione.

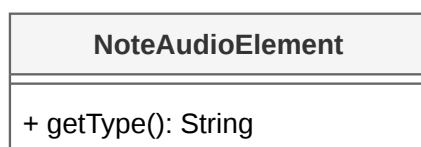
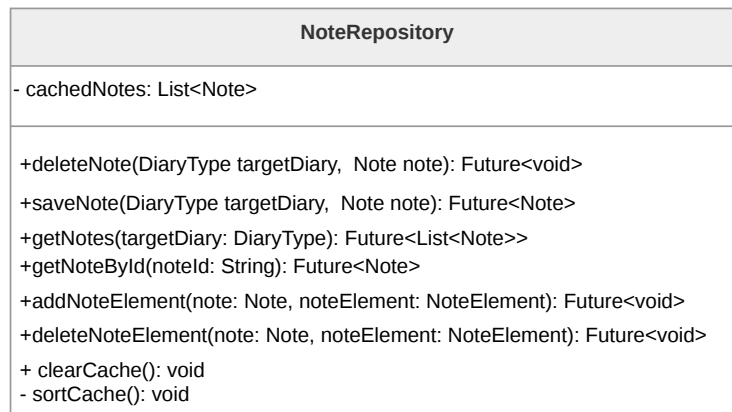


Figura 54: Diagramma della classe **NoteAudioElement**

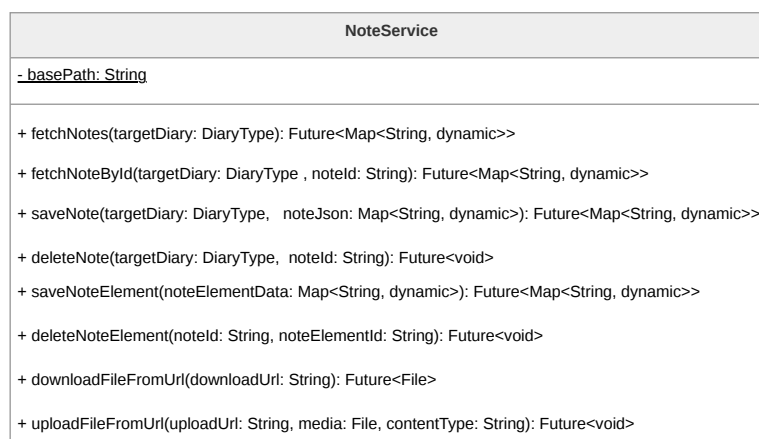
3.5.3.21 NoteRepository

Orchestra l'accesso ai dati delle note e dei file multimediali. Implementa logiche di cache in-memory (**cachedNotes**) e gestisce l'upload immediato dei media (**uploadFile**) tramite il Service, mantenendo lo stato locale aggiornato prima del completamento dell'operazione remota.

Figura 55: Diagramma della classe **NoteRepository**

3.5.3.22 NoteService

Livello infrastrutturale per la comunicazione con gli endpoint delle note. Gestisce operazioni CRUD remote e l'invio/ricezione di byte per i file multimediali. Restituisce i dati grezzi al Repository per la trasformazione in modelli di dominio.

Figura 56: Diagramma della classe **NoteService**

3.5.3.23 NoteDTO

La classe **NoteDTO** è un Data Transfer Object responsabile della serializzazione e deserializzazione delle note in formato JSON. Per la gestione del corpo della nota, delega la trasformazione della lista di elementi a **NoteElementDTO**, ricomponendo l'oggetto **Note** completo per il Domain Layer.

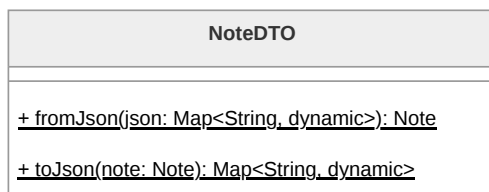


Figura 57: Diagramma della classe **NoteDTO**

3.5.3.24 NoteElementDTO

La classe **NoteElementDTO** implementa la logica di deserializzazione polimorfica per i blocchi multimediali. Funge da Factory: analizzando il campo identificativo del tipo nel JSON, esegue uno *switch* per istanziare la sottoclasse corretta di **NoteElement** (**NoteTextElement**, **NoteAudioElement**, **NoteImageElement** o **NoteFileElement**).

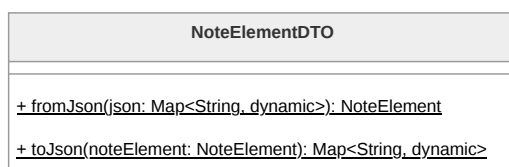


Figura 58: Diagramma della classe **NoteElementDTO**

3.5.4 Contatti Fidati

La funzionalità dei Contatti Fidati permette all'Utente di gestire una lista di persone di fiducia da avvisare in caso di inattività tramite il sistema di *Dead Man's Switch*. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un Data Layer, composto da Service e repository, e un UI Layer, composto da View e ViewModel.

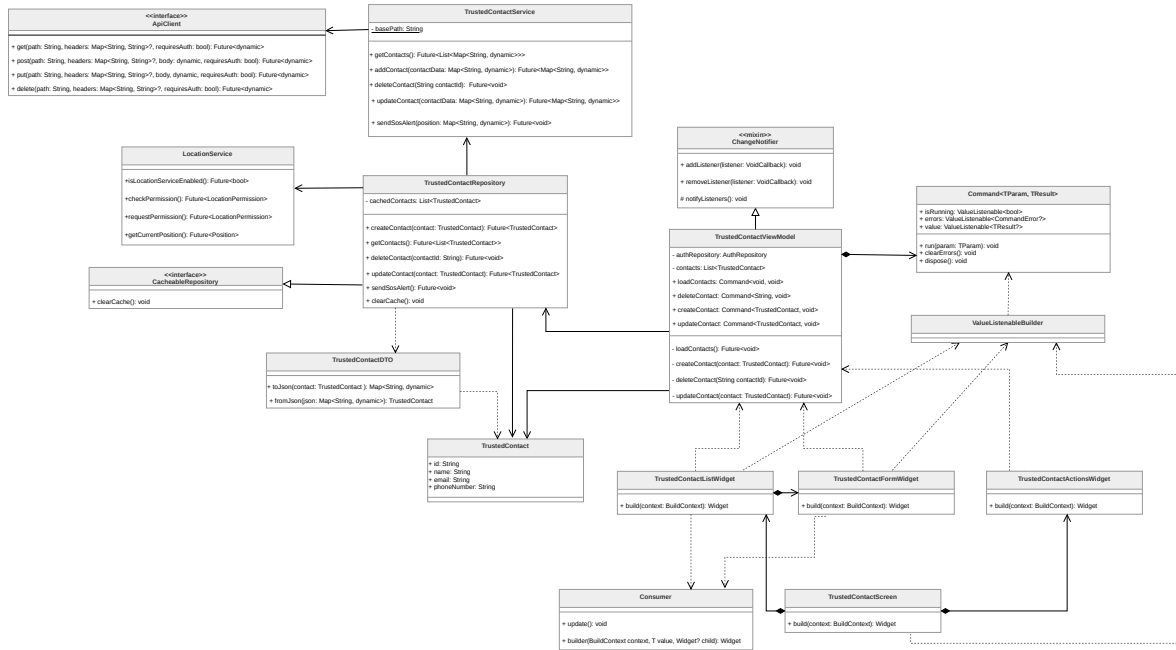


Figura 59: Diagramma delle classi relativo alle funzionalità dei Contatti Fidati

3.5.4.1 TrustedContactScreen

La classe **TrustedContactScreen** rappresenta la schermata principale dedicata alla gestione dei contatti fidati all'interno del *Presentation Layer*. Essa funge da compositore, unendo i vari widget presenti e ordinandoli secondo il layout stabilito, delegando la logica di stato ai componenti sottostanti.

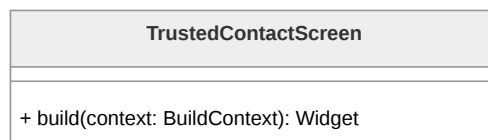


Figura 60: Diagramma della classe **TrustedContactScreen**

3.5.4.2 TrustedContactListWidget

La classe **TrustedContactListWidget** è responsabile della visualizzazione dell'elenco dei contatti attualmente salvati. Attraverso l'utilizzo del componente **Consumer**, osserva il ViewModel per reagire in modo reattivo ai cambiamenti della lista dei contatti e aggiornare dinamicamente l'interfaccia grafica senza ricostruire l'intera schermata.

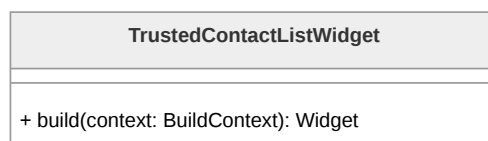


Figura 61: Diagramma della classe **TrustedContactListWidget**



3.5.4.3 TrustedContactFormWidget

TrustedContactFormWidget gestisce i campi di input necessari per l'inserimento o la modifica dei dati di un contatto fidato. Il widget integra componenti di osservazione come **Consumer** per monitorare lo stato di validazione e gli errori provenienti dal ViewModel, e **ValueListenableBuilder** per gestire lo stato di esecuzione dei Command, garantendo un feedback visivo durante le operazioni asincrone.

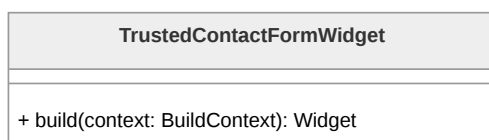


Figura 62: Diagramma della classe **TrustedContactFormWidget**

3.5.4.4 TrustedContactActionsWidget

Questo widget raggruppa i bottoni e le azioni specifiche per la gestione dei contatti fidati, come l'aggiunta di nuovi elementi o l'attivazione di procedure di modifica. Comunica le intenzioni dell'Utente al ViewModel invocando i relativi Command e reagisce ai cambiamenti di stato della View per gestire correttamente la disponibilità delle interazioni durante i processi di caricamento.



Figura 63: Diagramma della classe **TrustedContactActionsWidget**

3.5.4.5 TrustedContactViewModel

TrustedContactViewModel rappresenta il componente centrale per la gestione dello stato della funzionalità. Esso mantiene la lista dei contatti (**contacts**) e orchestra le operazioni asincrone attraverso l'uso del pattern Command (**loadContacts**, **createContact**, **updateContact**, **deleteContact**). La gestione degli stati di caricamento e degli eventuali fallimenti è incapsulata direttamente all'interno degli oggetti Command stessi, che espongono tali informazioni alla View in modo reattivo. Il ViewModel notifica i cambiamenti generali ai widget in ascolto tramite **ChangeNotifier**.

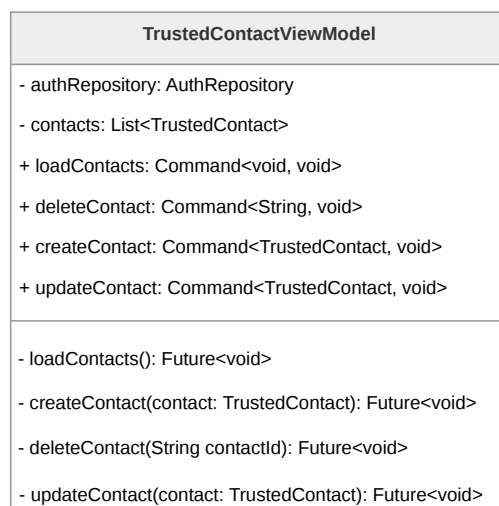


Figura 64: Diagramma della classe **TrustedContactViewModel**

3.5.4.6 TrustedContact

TrustedContact è la classe di dominio che modella il singolo contatto fidato. Contiene le informazioni anagrafiche e di recapito essenziali: identificativo unico (**id**), **name**, **email** e **phoneNumber**.



Figura 65: Diagramma della classe **TrustedContact**

3.5.4.7 TrustedContactRepository

TrustedContactRepository astrae la sorgente dei dati e implementa la gestione della cache locale tramite **cachedContacts** per migliorare la reattività dell'applicazione. La classe gestisce l'eliminazione dei contatti assicurando la sincronizzazione tra lo stato locale e il backend, con meccanismi di ripristino in caso di fallimento della richiesta. Inoltre, orchestra l'invio del segnale SOS tramite il metodo **sendSosAlert()**, coordinando l'acquisizione della posizione geografica tramite il **LocationService** prima di inoltrare la richiesta di emergenza al Service.

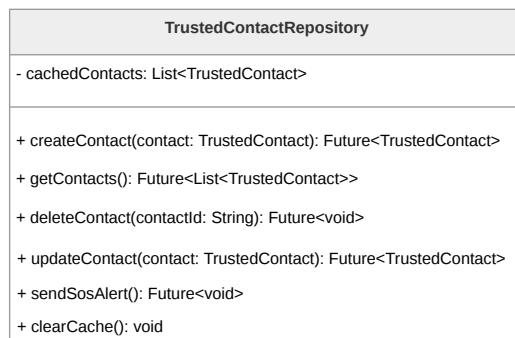


Figura 66: Diagramma della classe **TrustedContactRepository**

3.5.4.8 TrustedContactService

TrustedContactService si occupa delle chiamate API verso il Backend. Oltre alle operazioni CRUD standard (**getContacts**, **addContact**, **deleteContact**, **updateContact**), implementa il metodo **sendSosAlert** per inoltrare al backend la richiesta di invio dei messaggi di emergenza.

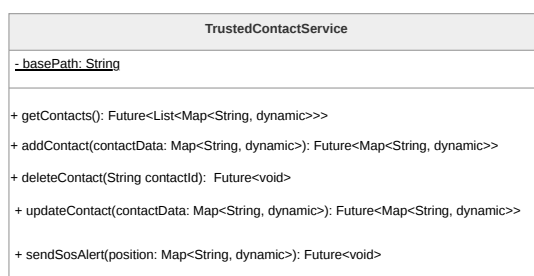


Figura 67: Diagramma della classe **TrustedContactService**

3.5.4.9 TrustedContactDTO

TrustedContactDTO gestisce la serializzazione e deserializzazione dei dati tramite i metodi **toJson** e **fromJson**. Garantisce che i dati scambiati con le API siano correttamente mappati da e verso gli oggetti di dominio **TrustedContact**.

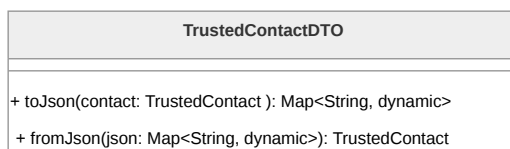


Figura 68: Diagramma della classe **TrustedContactDTO**

3.5.5 Luoghi Sicuri

La funzionalità dei Luoghi Sicuri permette all'Utente di visualizzare geograficamente, tramite una mappa dinamica, i punti di interesse rilevanti per la propria sicurezza personale

```
classDiagram
    class ApiClient {
        <<interface>>
        +get(path: String, headers: Map<String, String>?, requiresAuth: bool): Future<dynamic>*
        +post(path: String, headers: Map<String, String>?, body: dynamic, requiresAuth: bool): Future<dynamic>*
        +put(path: String, headers: Map<String, String>?, body: dynamic, requiresAuth: bool): Future<dynamic>*
        +delete(path: String, headers: Map<String, String>?, requiresAuth: bool): Future<dynamic>*
    }

    class SafePlaceService {
        -basePath: String
        +fetchSafePlaces(): Future<Map<String, dynamic>>*
    }

    class SafePlaceRepository {
        -cachedPlaces: List<SafePlace>
        -cachedMapState: (double, double, double)?
        +getPlaces(): Future<List<SafePlace>>*
        +clearCache(): void
    }

    class SafePlaceDTO {
        +fromJson(json: Map<String, dynamic>): SafePlace
        +toJson(place: SafePlace): Map<String, dynamic>
    }

    class SafePlaceCategory {
        <<enum>>
        +hospital
        +hospital
        +police
        +pharmacy
        +emergencyShelter
        +other
    }

    class SafePlace {
        +id: String
        +name: String
        +address: String
        +latitude: double
        +longitude: double
        +category: SafePlaceCategory
    }

    class ChangeNotifier {
        <<mixins>>
        +addListener(listener: VoidCallback): void
        +removeListener(listener: VoidCallback): void
        #notifyListeners(): void
    }

    class LocationService {
        +isLocationServiceEnabled(): Future<bool>*
        +checkPermission(): Future<LocationPermission>*
        +requestPermission(): Future<LocationPermission>*
        +getCurrentPosition(): Future<Position>*
    }

    class CommandTPParamTResult {
        +isRunning: ValueListenable<bool>*
        +error: ValueListenable<CommandError?>*
        +value: ValueListenable<TResult?>*
        +run(param: TParam): void
        +clearError(): void
        +dispose(): void
    }

    class ValueListenableBuilder {
    }

    class SafeMapViewModel {
        -selectedPlace: SafePlace?
        -userPosition: Position?
        +loadPlaces: Command<void, void>*
        +getUserLocation: Command<void, void>*
        -loadPlaces(): Future<void>*
        -getUserLocation(): Future<void>*
        +selectPlace(place: SafePlace): void
    }

    class CacheableRepository {
        <<interface>>
        +clearCache(): void
    }

    class SafePlaceMapWidget {
        +mapController: MapController
        +build(context: BuildContext): Widget
    }

    class SafePlaceMapScreen {
        +build(context: BuildContext): Widget
    }

    class Consumer {
        <<interface>>
        +update(): void
        +builder(BuildContext context, T value, Widget? child): Widget
    }

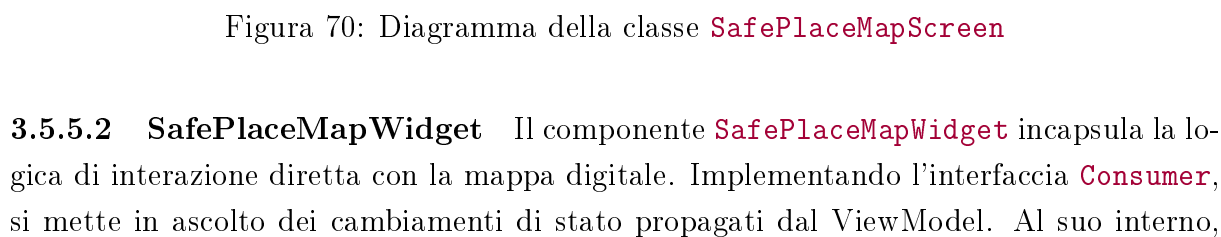
    ApiClient <|-- SafePlaceService
    SafePlaceService --> SafePlaceRepository
    SafePlaceRepository ..> CacheableRepository
    SafePlaceRepository ..> SafePlaceDTO
    SafePlaceRepository ..> SafePlace
    SafePlaceRepository ..> SafePlaceCategory
    SafePlaceRepository ..> SafePlaceMapWidget
    SafePlaceRepository ..> SafePlaceMapScreen
    SafePlaceRepository ..> Consumer
    SafePlaceRepository ..> ChangeNotifier
    SafePlaceRepository ..> LocationService
    SafePlaceRepository ..> CommandTPParamTResult
    SafePlaceRepository ..> ValueListenableBuilder
```

The UML class diagram illustrates the architecture of the SafePlace application. It features several key classes and interfaces:

- ApiClient**: An interface defining methods for API calls like `+ get(path: String, headers: Map<String, String>?, requiresAuth: bool): Future<dynamic>`.
- SafePlaceService**: Implements `ApiClient`, managing the base path and fetching safe places via `+ fetchSafePlaces(): Future<Map<String, dynamic>>`.
- SafePlaceRepository**: Manages cached places and map state. It implements `+ getPlaces(): Future<List<SafePlace>>` and `+ clearCache(): void`. It has dependencies on `CacheableRepository`, `SafePlaceDTO`, `SafePlace`, `SafePlaceCategory`, `SafePlaceMapWidget`, `SafePlaceMapScreen`, `Consumer`, `ChangeNotifier`, `LocationService`, and `CommandTPParamTResult`.
- SafePlaceDTO**: Handles JSON serialization/deserialization for `SafePlace` objects.
- SafePlace**: A data model containing fields like `+ id: String`, `+ name: String`, `+ address: String`, `+ latitude: double`, `+ longitude: double`, and `+ category: SafePlaceCategory`.
- SafePlaceCategory**: An enum representing categories such as hospital, police, pharmacy, emergency shelter, and other.
- ChangeNotifier**: A mixin class for managing listeners and notifications.
- LocationService**: Provides location-related functionality, including checking permissions and getting current position.
- CommandTPParamTResult**: A utility class for handling asynchronous commands and results.
- ValueListenableBuilder**: A builder for creating `ValueListenable` objects.
- SafeMapViewModel**: Manages the selected place and user position, providing methods like `+ loadPlaces: Command<void, void>` and `+ selectPlace(place: SafePlace): void`.
- CacheableRepository**: An interface for cache management.
- SafePlaceMapWidget** and **SafePlaceMapScreen**: UI components for displaying maps, implementing `+ build(context: BuildContext): Widget`.
- Consumer**: An interface for building widgets based on context and values.

3.5.5.1 SafePlaceMapScreen

La classe `SafePlaceMapScreen` rappresenta la schermata principale nel *Presentation Layer*. Funge da orchestratore (compositore), organizzando il layout della pagina e delegando il rendering cartografico al widget figlio specializzato. Si occupa inoltre della gestione dell'interfaccia passiva di dettaglio (tramite *BottomSheet* testuali, per evitare *context switch* non sicuri) e di coordinare l'ascolto reattivo granulare (tramite `ValueListenableBuilder`) degli stati asincroni, garantendo la *graceful degradation* (es. mostrando un avviso se il recupero della posizione GPS fallisce, senza bloccare la mappa).





utilizza i componenti forniti da `flutter_map` (`TileLayer` e `MarkerLayer`) per renderizzare la mappa e generare `Marker` visivi a partire da istanze di `SafePlace`. Inoltre, si occupa di intercettare gli eventi di interazione (pan e zoom) per salvare costantemente lo stato della navigazione nel `ViewModel`.

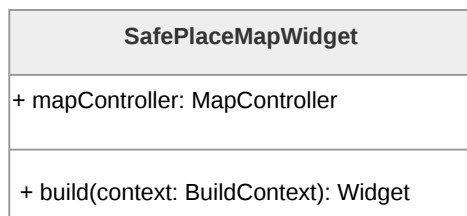


Figura 71: Diagramma della classe `SafePlaceMapWidget`

3.5.5.3 SafePlaceViewModel Il `SafePlaceViewModel` gestisce lo stato della View relativa ai luoghi sicuri. Eredita da `ChangeNotifier` per notificare i widget di cambiamenti di stato. Adotta il pattern Command esponendo i comandi `loadPlaces` e `getUserLocation`, che incapsulano internamente i `ValueListenable` per gli stati di caricamento (`isRunning`) ed errore (`errors`), garantendo una reattività granulare della View senza causare *rebuild* dell'intera schermata.

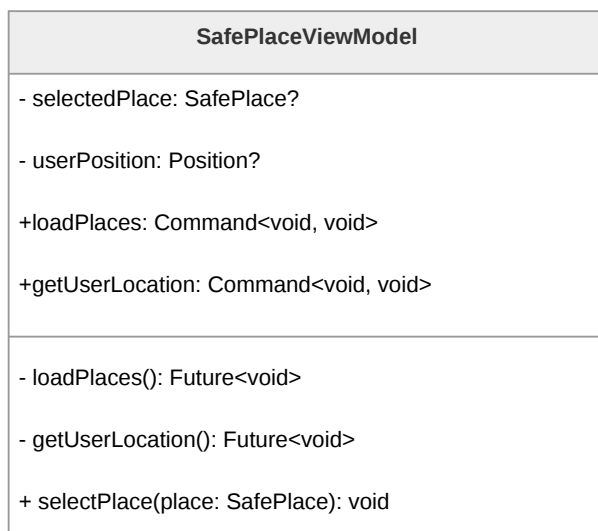


Figura 72: Diagramma della classe `SafePlaceViewModel`

`SafePlace` rappresenta l'entità di dominio fondamentale che modella un punto di interesse all'interno del sistema. La classe incapsula gli attributi identificativi del luogo, tra cui il nome e l'indirizzo testuale, oltre alle proprietà numeriche `latitude` e `longitude`. Queste ultime sono essenziali per il posizionamento spaziale dei `Marker` sulla superficie carto-



grafica gestita da `flutter_map`. La classe mantiene inoltre un riferimento alla propria categoria di appartenenza, delegando ad essa la semantica tipologica del luogo.

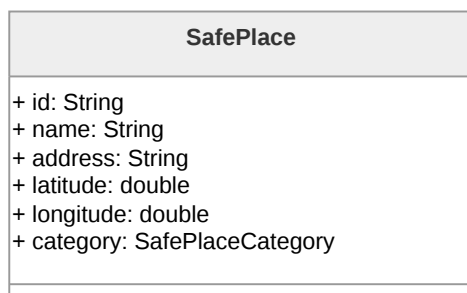


Figura 73: Diagramma della classe `SafePlace`

3.5.5.4 SafePlaceCategory L'enumerazione `SafePlaceCategory` definisce la classificazione tipologica dei luoghi sicuri gestiti dall'applicazione.

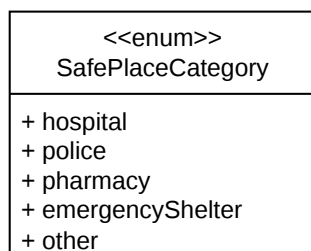


Figura 74: Diagramma della classe `SafePlaceCategory`

3.5.5.5 SafePlaceRepository Il `SafePlaceRepository` funge da *Single Source of Truth* per la sessione corrente, estendendo l'interfaccia `CacheableRepository`. Oltre a convertire i dati grezzi in oggetti di dominio tramite il DTO, incapsula una logica di memorizzazione in cache (`cachedPlaces`). Salva inoltre le coordinate e lo zoom correnti (`cachedMapState`) per ripristinare il contesto geografico tra diverse navigazioni nella schermata, ottimizzando le prestazioni e abbattendo i costi associati alle chiamate di rete superflue verso il Backend Serverless.

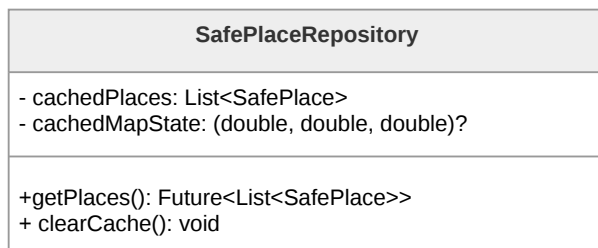


Figura 75: Diagramma della classe **SafePlaceRepository**

3.5.5.6 LocationService Appartenente al livello infrastrutturale, **LocationService** astrae l'interazione con l'hardware GPS del dispositivo mobile. Offre i metodi essenziali per controllare l'abilitazione del sensore (**isLocationServiceEnabled**) e per richiedere in modo sicuro i permessi all'Utente (**checkPermission**, **requestPermission**), permettendo al ViewModel di gestire preventivamente gli accessi negati.

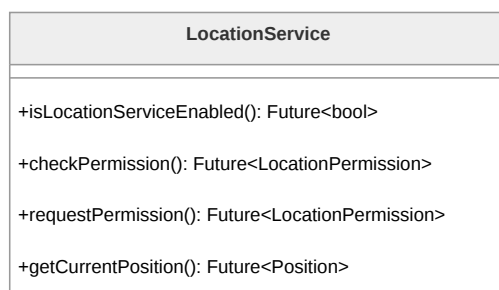


Figura 76: Diagramma della classe **LocationService**

3.5.5.7 SafePlaceService **SafePlaceService** è incaricato delle comunicazioni HTTP verso gli endpoint del Backend per il recupero dei luoghi sicuri. Sfruttando **ApiClient**, inoltra le richieste per ottenere il dataset dei luoghi sicuri, contenente le coordinate geografiche di ogni singolo luogo e i relativi metadati.

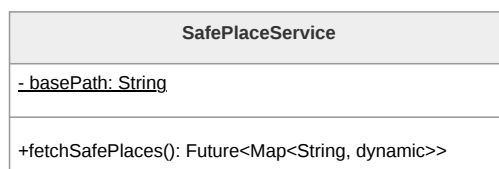


Figura 77: Diagramma della classe **SafePlaceService**

3.5.5.8 SafePlaceDTO La classe **SafePlaceDTO** (*Data Transfer Object*) isola la logica di serializzazione e deserializzazione (**fromJson**, **toJson**). Mappa le chiavi del Payload JSON scaricata dal Backend nei tipi primitivi Dart attesi dal dominio, disaccoppiando la persistenza esterna dalla *Business Logic* interna.

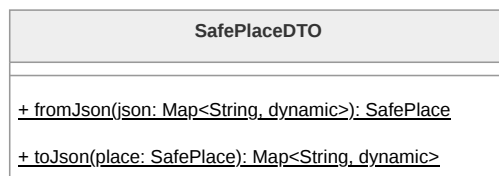


Figura 78: Diagramma della classe `SafePlaceDTO`

3.5.6 Dead Man's Switch

La funzionalità del *Dead Man's Switch* rappresenta un sistema di sicurezza critico che invia avvisi automatici in caso di inattività prolungata dell'Utente. L'Architettura frontend è focalizzata sulla gestione sicura e reattiva della configurazione di tale sistema. Il frontend non interagisce direttamente con servizi di posta elettronica o code di schedulazione, ma demanda la gestione dei timer e l'invio delle email al Backend, garantendo l'affidabilità del servizio anche ad applicazione chiusa. Il conteggio dell'inattività viene azzerato tramite un segnale di *heartbeat*, il cui innesco primario è delegato a componenti esterne (come l'avvenuto login). Analogamente al resto dell'applicazione, la funzionalità implementa il pattern MVVM.

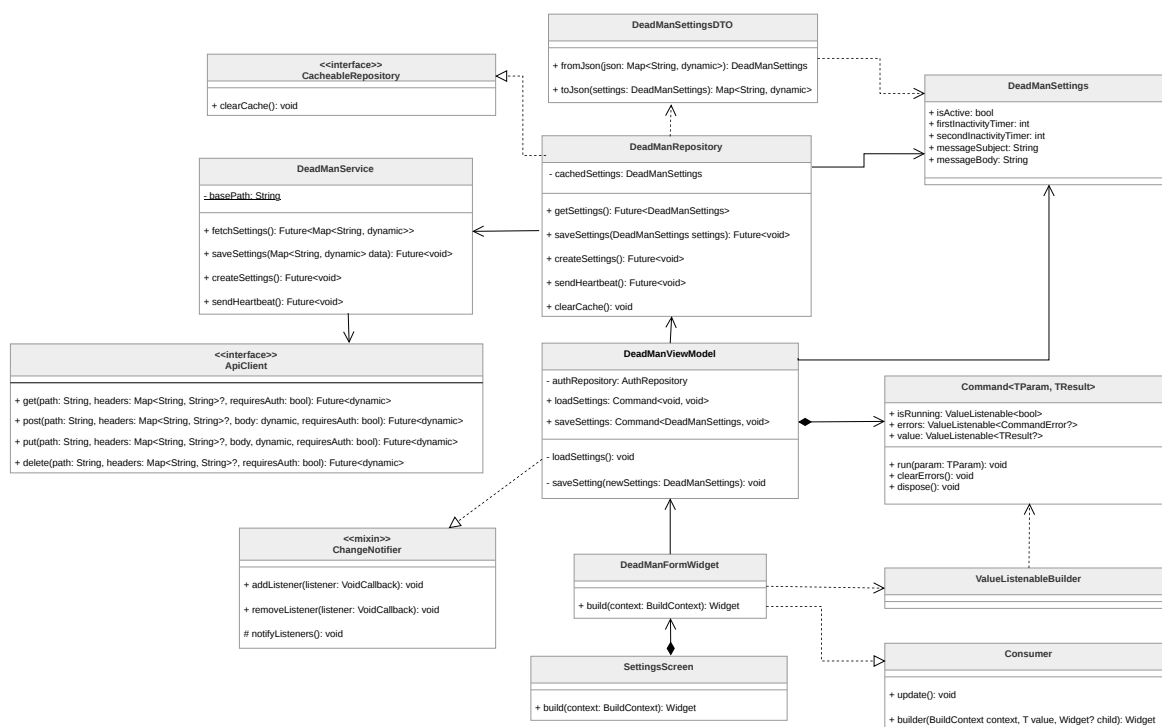


Figura 79: Diagramma delle classi relativo alla funzionalità del Dead Man's Switch

3.5.6.1 SettingsScreen

La classe `SettingsScreen` rappresenta la schermata principale che ospita il pannello di controllo per la sicurezza. Funge da compositore, strutturando



do il layout della pagina e integrando i form interattivi, delegando al **DeadManViewModel** la reattività agli stati asincroni e la Persistence Logic.

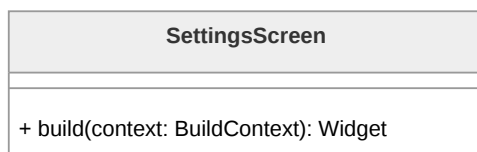


Figura 80: Diagramma della classe **SettingsScreen**

3.5.6.2 DeadManFormWidget Questo componente incapsula gli elementi interattivi dell'interfaccia, come lo switch per l'attivazione del servizio, i campi di input per i timer e le aree di testo per i messaggi di emergenza. È implementato come widget con stato locale (*StatefulWidget*) per gestire in modo efficiente lo stato transitorio della form durante la digitazione, evitando onerosi *rebuild* del ViewModel. Si occupa inoltre di disabilitare visivamente e bloccare i campi di input quando l'Utente sceglie di disattivare il servizio.



Figura 81: Diagramma della classe **DeadManFormWidget**

3.5.6.3 DeadManViewModel **DeadManViewModel** orchestra la logica di business relativa alla configurazione del servizio. Abbandonando la gestione manuale di flag di caricamento generici, adotta il pattern Command esponendo le proprietà **loadSettings** e **saveSettings**. Questi comandi incapsulano autonomamente gli stati asincroni dell'operazione, permettendo alla View di reagirvi in modo granulare. Riceve i dati dalla form solo al momento del salvataggio, delegando al Repository la comunicazione dei nuovi parametri temporali e testuali.

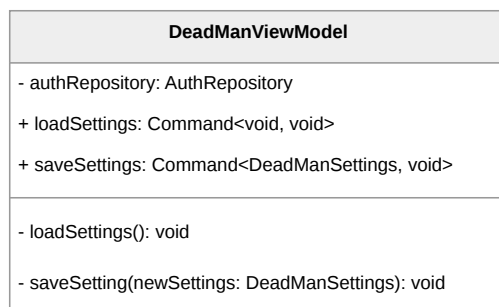


Figura 82: Diagramma della classe **DeadManViewModel**

3.5.6.4 DeadManSettings **DeadManSettings** è l'entità di dominio che modella le preferenze del *Dead Man's Switch*. Contiene gli attributi necessari a configurare la logica di allarme: uno stato booleano di attivazione, i valori temporali che definiscono le due distinte soglie di inattività, nonché l'oggetto e il corpo del messaggio testuale.

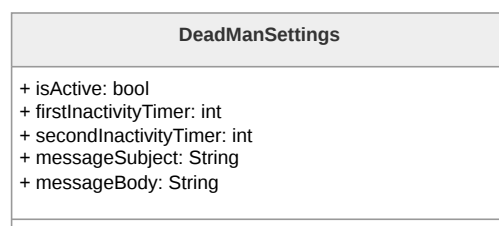


Figura 83: Diagramma della classe **DeadManSettings**

3.5.6.5 DeadManRepository Il **DeadManRepository** astrae la persistenza e il recupero delle configurazioni, agendo come *Single Source of Truth* temporanea in quanto implementazione di **CacheableRepository**. Utilizza **DeadManService** per recuperare o inviare le preferenze e per instradare richieste accessorie come la creazione iniziale (**createSettings**) o la segnalazione di apertura dell'app (**sendHeartbeat**). Si occupa inoltre di convertire le risposte grezze di rete in oggetti di dominio sicuri tramite l'apposito DTO.

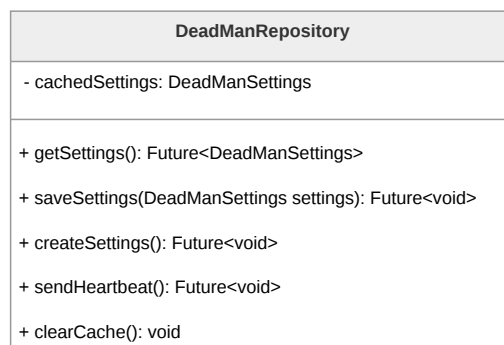


Figura 84: Diagramma della classe **DeadManRepository**

3.5.6.6 DeadManService Appartenente al Data Layer, **DeadManService** si occupa della comunicazione HTTP con le API del Backend. Fornisce i metodi necessari per effettuare le operazioni CRUD sulle configurazioni (**fetchSettings**, **saveSettings**, **createSettings**) e per l'invio effettivo del segnale vitale dell'Utente verso i server AWS (**sendHeartbeat**).

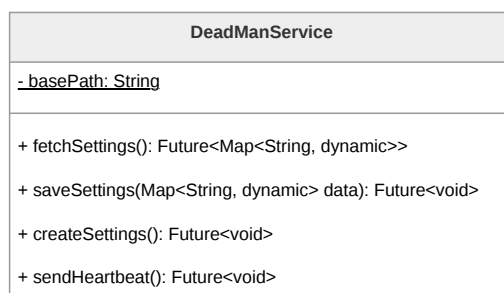


Figura 85: Diagramma della classe **DeadManService**

3.5.6.7 DeadManSettingsDTO Il **DeadManSettingsDTO** (*Data Transfer Object*) gestisce la logica di serializzazione bidirezionale. Consente di mappare coerentemente i campi del Payload JSON in istanze del dominio, assicurando che la comunicazione tra l'app Flutter e il Backend rispetti un rigido Contratto dati per i nuovi parametri di configurazione temporali e testuali.

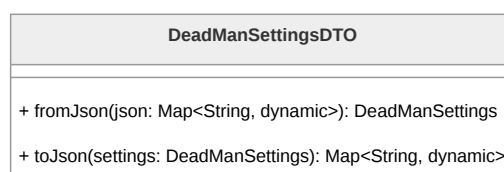


Figura 86: Diagramma della classe **DeadManSettingsDTO**



3.5.7 Materiale Informativo

La funzionalità del Materiale Informativo consente all'Utente di consultare risorse educative, normative di legge e contatti utili per la sicurezza personale. A livello architetturale, questa sezione condivide la medesima impostazione ibrida e ottimizzata della Mappa dei Luoghi Sicuri. Aderisce al pattern MVVM.

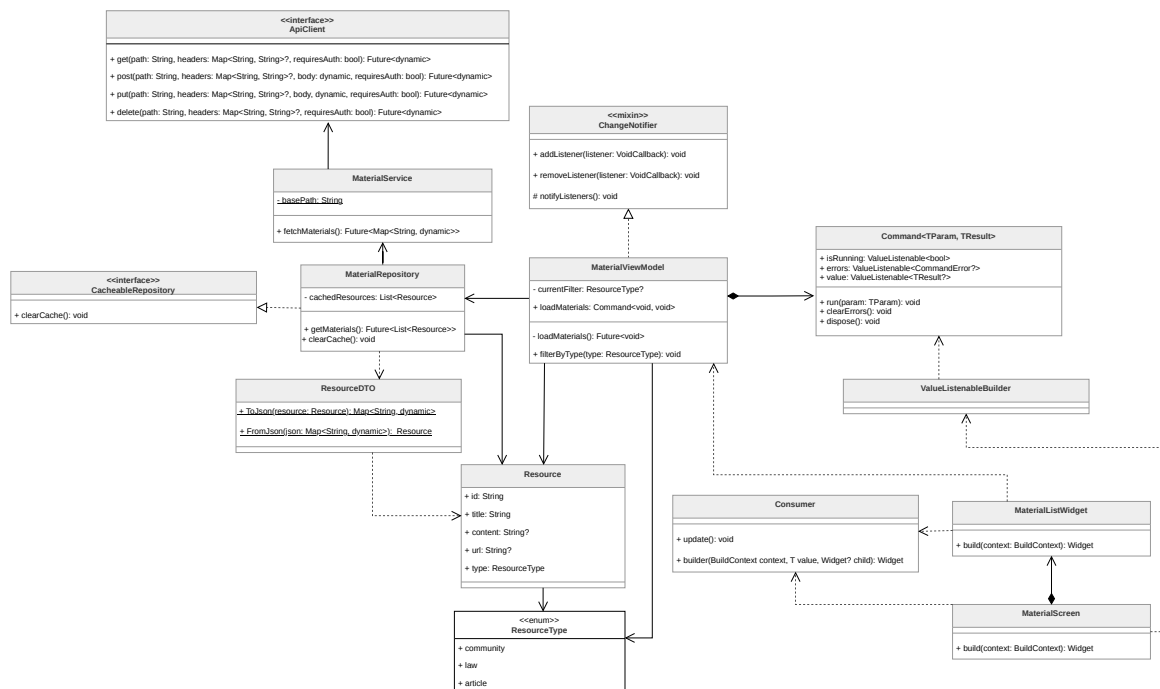


Figura 87: Diagramma delle classi complessivo della funzionalità Materiale Informativo

3.5.7.1 MaterialScreen La classe **MaterialScreen** rappresenta il punto d'ingresso per la fruizione delle risorse all'interno del *Presentation Layer*. Agisce come compositore del layout principale: organizza la barra di navigazione, delega la visualizzazione degli stati asincroni (caricamento, errore) e ospita i componenti di interazione rapida, che permettono all'Utente di selezionare agilmente le categorie di interesse senza bloccare l'interfaccia.

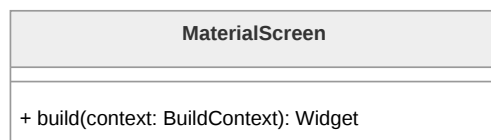


Figura 88: Diagramma della classe **MaterialScreen**

3.5.7.2 MaterialListWidget Il **MaterialListWidget** è incaricato della renderizzazione della lista di risorse. Sfruttando un approccio ibrido, si mette in ascolto tramite **Consumer** della lista filtrata fornita dal ViewModel. La visualizzazione adotta strategie UX differenziate in base alla natura del dato: per i contenuti testuali interni (**content**)



utilizza il componente **ExpansionTile**, permettendo la lettura inline senza perdere il contesto della lista; per i riferimenti esterni (**url**) utilizza il pacchetto nativo **url_launcher**, delegando l'apertura del link al browser di sistema per garantire la massima sicurezza ed evitare *context switch* non controllati all'interno dell'app.

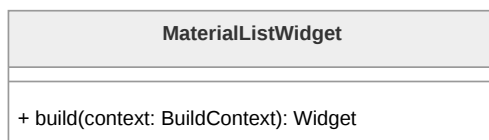


Figura 89: Diagramma della classe **MaterialListWidget**

3.5.7.3 MaterialViewModel Il **MaterialViewModel** gestisce la Presentation Logic e lo stato dell'interfaccia. Espone il comando asincrono **loadMaterials** (basato sul pattern Command) per governare il primo fetch dei dati tramite **ValueListenable**. La caratteristica peculiare di questo ViewModel è la gestione totalmente client-side del filtraggio tramite la variabile **currentFilter** e il metodo sincrono **filterByType**: operando sulla lista già mantenuta in memoria, le transizioni di interfaccia risultano istantanee, azzerando la latenza di rete e ottimizzando i costi infrastrutturali.

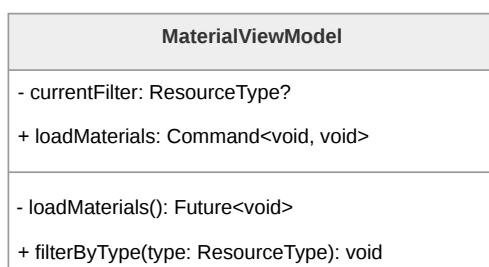
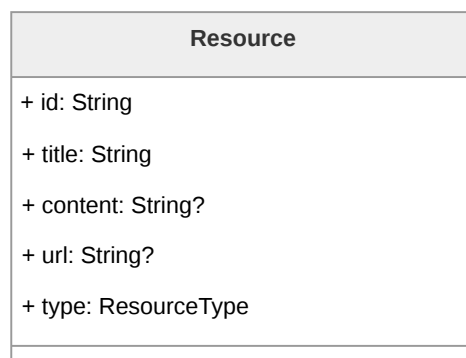
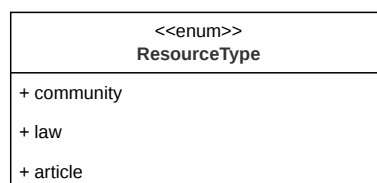


Figura 90: Diagramma della classe **MaterialViewModel**

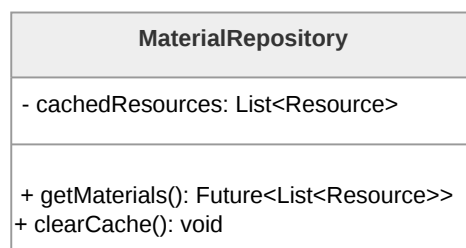
3.5.7.4 Resource **Resource** modella la singola entità di dominio del materiale informativo. È progettata per essere flessibile, contenendo campi obbligatori di identificazione e campi opzionali mutualmente esclusivi nella logica applicativa: **content** (per brevi testi informativi) e **url** (per link che rimandano a leggi ufficiali, siti istituzionali, ecc..).

Figura 91: Diagramma della classe **Resource**

3.5.7.5 ResourceType **ResourceType** è l'enumerazione di dominio che definisce le possibili nature della risorsa (es. **community**, **law**, **article**). Questa astrazione tipizzata è cruciale non solo per la *Business Logic*, ma anche per il ViewModel, che la utilizza come chiave discriminante per l'applicazione dei filtri locali.

Figura 92: Diagramma della classe **ResourceType**

3.5.7.6 MaterialRepository Il **MaterialRepository** assolve al ruolo di *Single Source of Truth* per l'accesso ai documenti. Implementando l'interfaccia **CacheableRepository**, mantiene in memoria la lista completa dei materiali recuperati (**cachedResources**). Questa scelta architetturale disaccoppia le operazioni di filtraggio visivo dalla rete, permettendo all'applicazione di fornire una reattività immediata ai cambi di categoria operati dall'Utente.

Figura 93: Diagramma della classe **MaterialRepository**



3.5.7.7 MaterialService Appartenente al livello infrastrutturale, il **MaterialService** gestisce le invocazioni HTTP tramite **ApiClient**. Il metodo **fetchMaterials** inoltra la richiesta al Backend AWS Serverless, il quale si occupa di esporre e recapitare il Payload JSON statico custodito all'interno di un Bucket S3, configurazione ideale per minimizzare il caricamento iniziale e mantenere i costi scalabili.

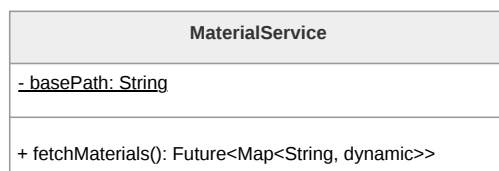


Figura 94: Diagramma della classe **MaterialService**

3.5.7.8 ResourceDTO La classe **ResourceDTO** (*Data Transfer Object*) gestisce il confine di serializzazione. Mappa in modo difensivo le chiavi del JSON scaricato dal Backend (gestendo correttamente la presenza o l'assenza dei campi opzionali **content** o **url**) e restituisce al Repository oggetti **Resource** perfettamente istanziati e tipizzati per il dominio interno.

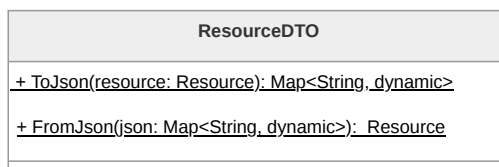


Figura 95: Diagramma della classe **ResourceDTO**

3.5.8 Main App, Home e Sistema SOS

Il nucleo orchestrale dell'applicazione è definito dall'integrazione tra la radice del sistema, la dashboard di controllo e il modulo di emergenza. Questa sezione governa il Ciclo di vita della sessione, il routing globale e i meccanismi di sicurezza immediata, garantendo che le funzionalità critiche siano sempre accessibili all'Utente.

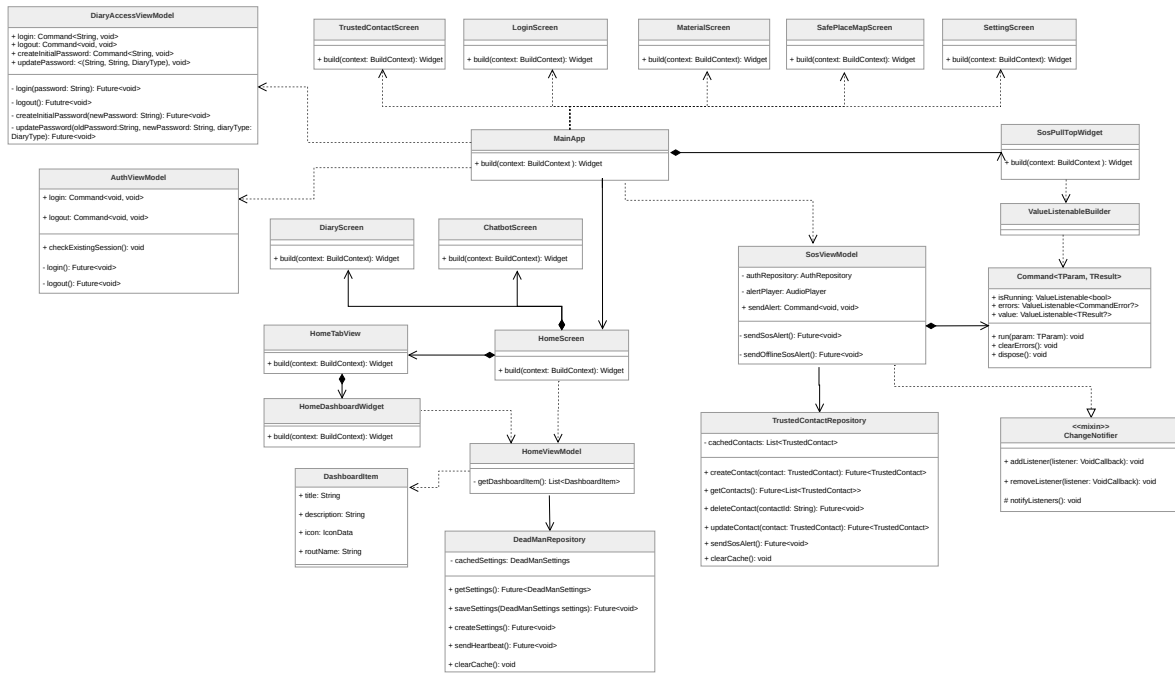


Figura 96: Diagramma delle classi complessivo di Main App, Home e SOS

3.5.8.1 MainAppWidget La classe **MainAppWidget** costituisce l'*Entry Point* dell'applicazione Flutter. Si occupa dell'inizializzazione dei provider globali (**AuthViewModel**, **SosViewModel**, **DiaryAccessViewModel**) tramite un **MultiProvider**. Architetturealmente, definisce una strategia di navigazione ibrida: utilizza rotte nominate per le pagine verticali (es. `/login`, `/settings`) e sfrutta il parametro **builder** di **MaterialApp** per iniettare il **SosPullTopWidget** in uno stack globale. Questa scelta assicura che il trigger di emergenza risieda in uno strato superiore dell'interfaccia, rendendolo disponibile sopra ogni altra schermata.

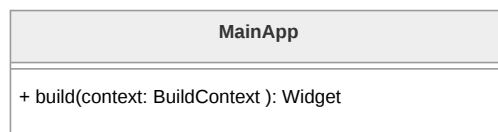
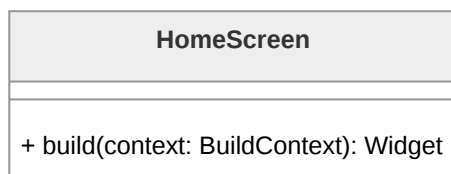
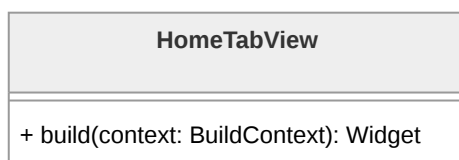


Figura 97: Diagramma della classe **MainAppWidget**

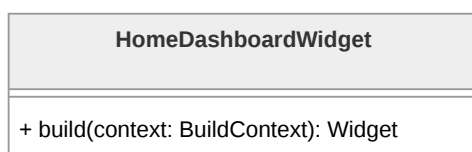
3.5.8.2 HomeScreen La **HomeScreen** implementa la navigazione principale dell'applicativo tramite un pattern a schede (*Tab Navigation*). Coordina un widget **PageView** per lo scorrimento delle sezioni (Chatbot, Home, Diario) e una **NavigationBar** per la selezione rapida. Per preservare l'integrità dei dati sensibili, la classe integra logiche di controllo dell'autenticazione: se l'Utente non è loggato, le tab protette vengono sostituite dinamicamente da un **AuthPlaceholderScreen**, impedendo accessi non autorizzati senza interrompere il flusso di navigazione.

Figura 98: Diagramma della classe **HomeScreen**

3.5.8.3 HomeTabView La **HomeTabView** rappresenta il contenuto specifico della tab centrale dell'applicazione. Agisce come un contenitore specializzato che definisce il layout della dashboard, includendo l'*AppBar* con i punti di accesso al profilo e alle impostazioni, e i messaggi di benvenuto personalizzati. Al suo interno, delega il caricamento e la visualizzazione dei moduli interattivi al widget figlio **HomeDashboardWidget**.

Figura 99: Diagramma della classe **HomeTabView**

3.5.8.4 HomeDashboardWidget Il **HomeDashboardWidget** è incaricato della renderizzazione visiva dei punti di accesso ai vari moduli dell'app (Materiali, Mappa, Contatti). Ogni modulo viene presentato tramite una card interattiva che incapsula la logica di navigazione verso la relativa rotta nominata.

Figura 100: Diagramma della classe **HomeDashboardWidget**

3.5.8.5 HomeViewModel Il **HomeViewModel** centralizza la Presentation Logic della dashboard. Attraverso il metodo `getDashboardItem()`, popola staticamente una lista di oggetti di dominio **DashboardItem**, definendone l'iconografia, il titolo descrittivo e la rotta di destinazione. Questo approccio permette di gestire l'Architettura della homepage in modo centralizzato, facilitando l'aggiunta o la rimozione di moduli funzionali senza modificare la struttura dei widget.

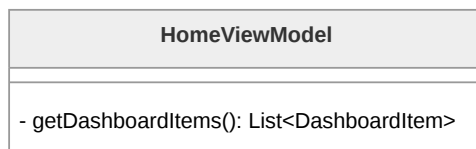


Figura 101: Diagramma della classe **HomeViewModel**

3.5.8.6 SosPullTopWidget Il **SosPullTopWidget** rappresenta l'interfaccia di attivazione globale del sistema di emergenza. Essendo iniettato alla radice dell'applicazione, rimane persistente e accessibile tramite un'interazione di trascinamento (*pull*) o pressione. Comunica direttamente con il **SosViewModel** per innescare immediatamente le procedure di soccorso indipendentemente dalla schermata in cui si trova l'Utente.



Figura 102: Diagramma della classe **SosPullTopWidget**

3.5.8.7 SosViewModel Il **SosViewModel** orchestra la logica critica di invio delle segnalazioni. Implementando il pattern Command, espone il metodo **sendAlert** che avvia una strategia di *fallback* basata sulla connettività del dispositivo. In modalità online, inoltra una richiesta al Backend per l'invio di email tramite AWS SES; in modalità offline, attiva un segnale acustico ad alta intensità tramite il componente **AudioPlayer**, garantendo una forma di allerta locale anche in assenza di rete.

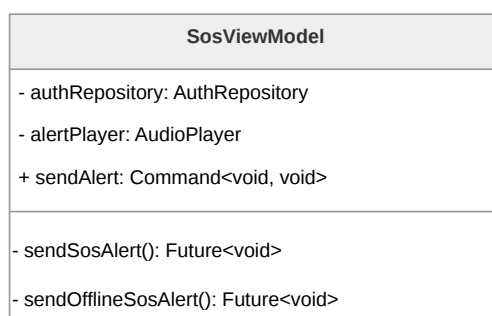


Figura 103: Diagramma della classe **SosViewModel**



3.6. Architettura Back End

3.6.1 Autenticazione

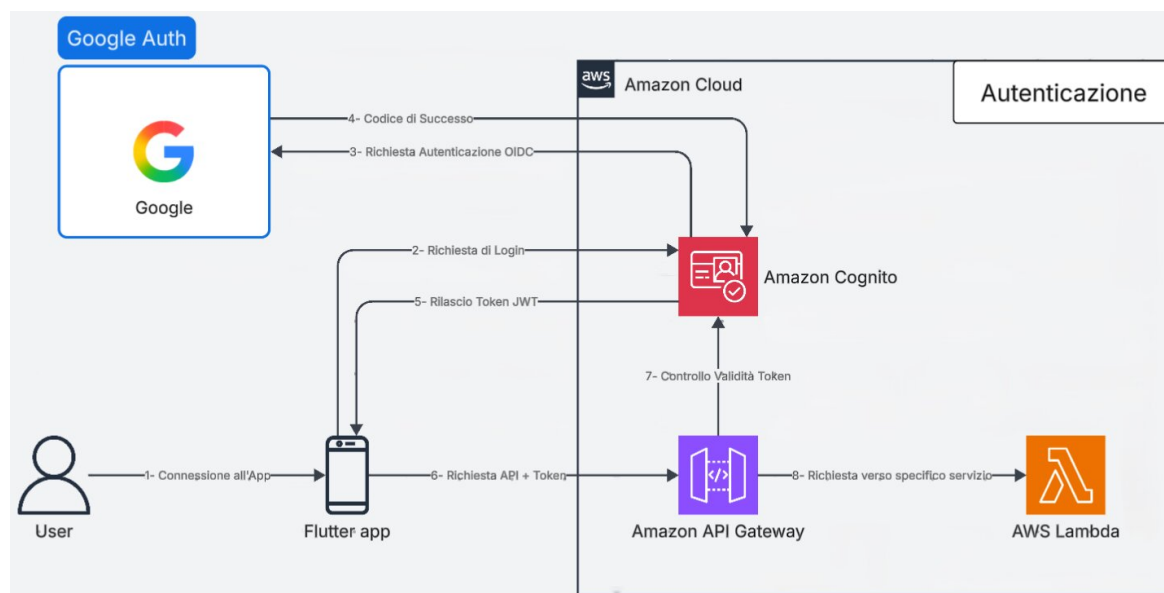


Figura 104: Architettura e Flusso di Autenticazione

3.6.1.1 Architettura e Flusso di Autenticazione: Modulo Federated Login

La presente sezione descrive l'Architettura logica e il flusso dei dati relativi al modulo di autenticazione e autorizzazione dell'applicazione. Il sistema adotta un modello di federated login basato su Amazon Cognito integrato con Google come Identity Provider (IdP), progettato per garantire un accesso sicuro e standardizzato alle risorse di Backend senza demandare la gestione crittografica delle credenziali all'applicazione client.

3.6.1.1.1 Inizializzazione del Flusso e Autenticazione Federata

Il processo di autenticazione ha inizio quando l'Utente interagisce con l'applicazione client (Flutter) richiedendo l'accesso al sistema. L'applicazione delega interamente la gestione dell'identità inoltrando una richiesta di login ad Amazon Cognito. Cognito, fungendo da Service Provider, intercetta la richiesta e avvia una procedura di autenticazione basata sullo standard OpenID Connect (OIDC) delegandola ai server di Google. L'Utente completa la fase di riconoscimento interfacciandosi direttamente con l'infrastruttura del



provider esterno. A seguito della corretta verifica delle credenziali, Google restituisce ad Amazon Cognito un codice di autorizzazione attestante il successo dell'operazione e l'identità dell'Utente.

3.6.1.1.2 Emissione dei Token di Sessione

Acquisita la conferma dal provider federato, Amazon Cognito processa il codice di autorizzazione e genera un set di token crittografici aderenti allo standard JWT (JSON Web Token). Questi token vengono trasmessi in modo sicuro all'applicazione mobile, concludendo la fase di autenticazione. Da questo momento, il client dispone di una prova di identità verificata e utilizzerà l'Access Token JWT per autorizzare le successive chiamate di rete dirette ai servizi di Backend.

3.6.1.1.3 Validazione Perimetrale e Instradamento Sicuro

Per accedere alle logiche applicative, l'app Flutter compone una richiesta HTTPS indirizzata all'Amazon API Gateway, includendo il token JWT nelle intestazioni di autorizzazione. L'API Gateway assume il ruolo di strato di sicurezza perimetrale (edge security). Prima di instradare il traffico, il Gateway utilizza un Cognito Authorizer nativo per eseguire un rigoroso controllo di validità sul token ricevuto. Tale procedura verifica l'autenticità della firma crittografica, l'integrità del Payload e la validità temporale della sessione direttamente contro le configurazioni dello User Pool di Cognito.

Questa specifica impostazione architetturale assicura che qualsiasi richiesta malevola, non autorizzata o provvista di credenziali scadute venga respinta al livello di rete. Solamente le richieste che superano con successo la validazione del token vengono instradate dall'API Gateway verso la specifica funzione AWS Lambda designata, garantendo che le risorse computazionali elaborino esclusivamente traffico certificato e sicuro.

3.6.2 Luoghi sicuri

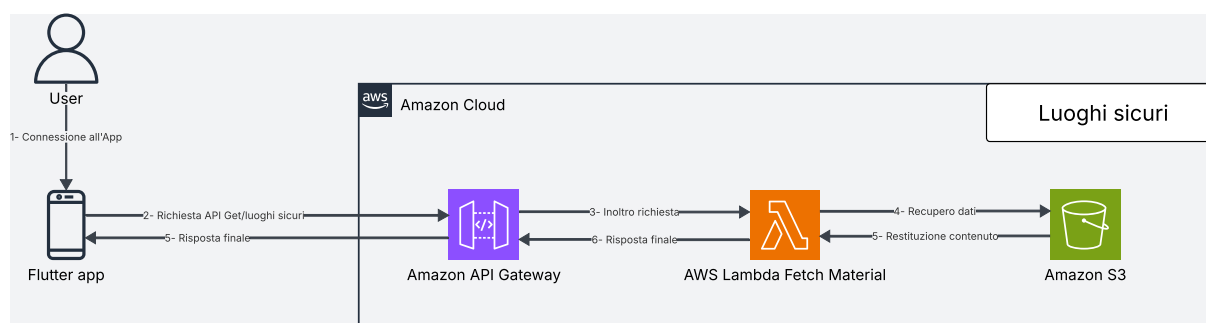


Figura 105: Architettura moduli Luoghi Sicuri



3.6.2.1 Architettura e Flusso di Elaborazione: Modulo Luoghi Sicuri

Il modulo "Luoghi Sicuri" è stato ingegnerizzato adottando una rigorosa separazione delle responsabilità (*Separation of Concerns*) tra l'elaborazione lato client (Frontend) e l'infrastruttura Cloud (Backend). L'obiettivo primario è stato quello di minimizzare il carico computazionale server-side ed eliminare del tutto i costi legati ai calcoli geospaziali. Il flusso di elaborazione si articola nei seguenti passaggi:

- **Ingestione della Richiesta:** L'Utente accede alla sezione della mappa tramite l'applicazione. Il client Flutter genera una richiesta HTTPS (metodo `GET /safe_locations`).
- **Instradamento verso Lambda:** API Gateway intercetta la richiesta, ne valida il contesto e la inoltra alla funzione AWS Lambda dedicata alla gestione dei luoghi sicuri.
- **Accesso ai dati e logica applicativa:** la Lambda esegue la logica applicativa necessaria e recupera i dati dal bucket S3 dedicato. Al suo interno sono infatti memorizzati sia i metadati descrittivi, sia le coordinate geospaziali (latitudine e longitudine) per ciascun punto di interesse.
- **Elaborazione e serializzazione:** la funzione Lambda elabora i dati estratti e li converte in un Payload JSON standardizzato e ottimizzato per il consumo lato client.
- **Risposta al client:** API Gateway inoltra la risposta HTTP 200 OK al client Flutter contenente l'elenco dei luoghi sicuri.

3.6.2.1.1 Delega della Logica Geospaziale al Livello Client

Al fine di ottimizzare ulteriormente i costi, l'intera logica di rendering visivo e di routing è delegata all'applicazione sul dispositivo dell'Utente. Il Backend AWS si configura esclusivamente come Data Provider di coordinate statiche. Alla ricezione del Payload JSON, l'applicativo Flutter si fa carico di istanziare la mappa nativa e posizionare i marker.

3.6.2.2 API associate

- **GET /safe-places/**
 - **Descrizione:** Recupera la lista dei luoghi sicuri disponibili, includendo informazioni geografiche e tipologia della struttura.
 - **Parametri:**
 - * Nessun parametro richiesto.
 - **Response:**



- * 200 OK: richiesta completata con successo.

- * Corpo della risposta:

```
[
  {
    "marker_id": "luogo-001",
    "name": "Ospedale Centrale",
    "address": "Via Roma 10, Milano",
    "latitude": 45.4642,
    "longitude": 9.1900,
    "category": "ospedale"
  },
  {
    "marker_id": "luogo-002",
    "name": "Centro Antiviolenza Milano",
    "address": "Via Torino 25, Milano",
    "latitude": 45.4658,
    "longitude": 9.1859,
    "category": "centro_antiviolenza"
  },
  {
    "marker_id": "luogo-003",
    "name": "Comando Carabinieri Milano Centro",
    "address": "Piazza Duomo 1, Milano",
    "latitude": 45.4641,
    "longitude": 9.1919,
    "category": "carabinieri"
  }
]
```

– **Errori:**

- * 405 Method Not Allowed: metodo HTTP non consentito per l'endpoint.

- * 500 Internal Server Error: errore interno del server.

Nota di sicurezza: L'integrazione di un Authorizer Cognito per la verifica delle autorizzazioni è stata deliberatamente omessa per questa route. L'assenza dell'Authorizer in quest'area è difatti una scelta architetturale mirata a rimuovere complessità infrastrutturale non necessaria e ad abbattere i costi operativi associati al recupero di questi specifici dati.



3.6.2.3 Architettura logica: Modulo Luoghi Sicuri

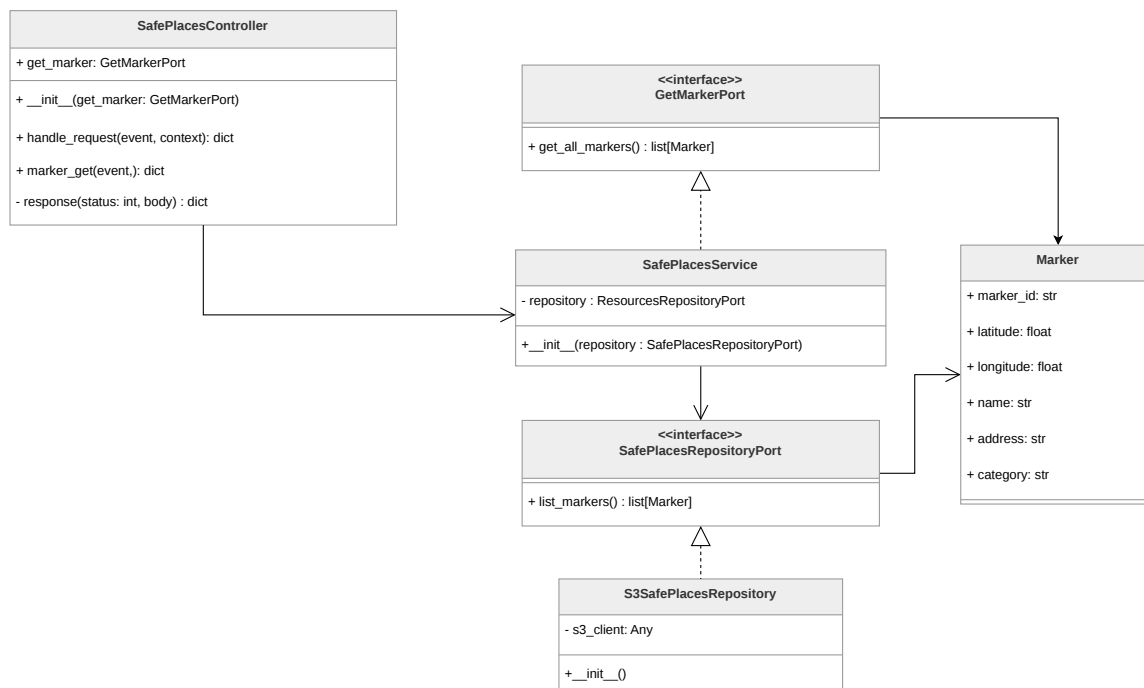


Figura 106: Componenti del microservizio luoghi sicuri

3.6.2.3.1 Marker

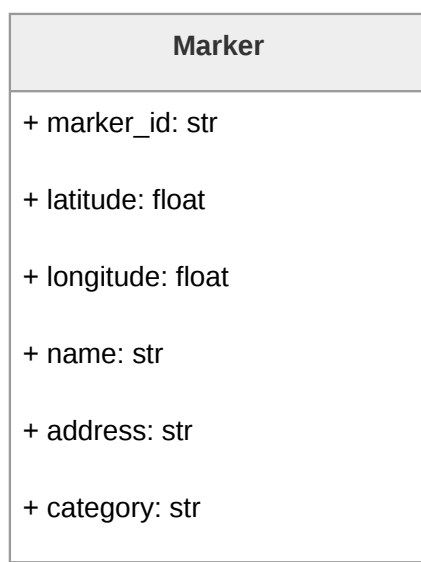


Figura 107: Diagramma della classe **Marker**



Rappresenta un punto geografico sicuro visualizzabile sulla mappa nel frontend dell'applicazione. Descrizione attributi:

- **marker_id**, che identifica univocamente il marker;
- **latitude** e **longitude** definiscono la posizione geografica;
- **name** indica il nome del marker;
- **address** indica l'indirizzo;
- **category** indica la categoria del marker;

3.6.2.3.2 SafePlacesController

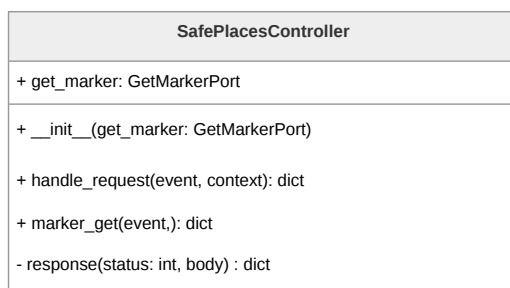


Figura 108: Diagramma della classe **SafePlacesController**

Controller del pattern esagonale. Riceve la richiesta grezza da API Gateway, la instrada verso la inbound port e costruisce il body per la risposta HTTP da restituire al client. Descrizione metodi:

- **marker_get(event)** esegue il parsing della richiesta e chiama la funzione appropriata della specifica port primaria;
- **handle_request(event)** chiama la funzione appropriata del controller in base alla route della richiesta;
- **response(status, body)** costruisce il dizionario di risposta nel formato atteso da API Gateway.

3.6.2.3.3 GetMarkerPort

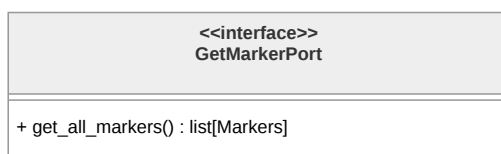


Figura 109: Diagramma dell'interfaccia **GetMarkerPort**



È la port primaria del pattern esagonale.

Il metodo `get_all_markers()` restituisce la lista completa dei marker disponibili senza esporre dettagli implementativi su come vengono recuperati.

3.6.2.3.4 SafePlacesService

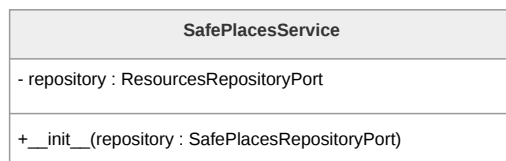


Figura 110: Diagramma della classe `SafePlacesService`

Corrisponde alla domain logic del microservizio. Contiene la logica per il recupero dei luoghi sicuri e orchestra la comunicazione con il layer di persistenza tramite la outbound port `SafePlacesRepositoryPort`.

3.6.2.3.5 SafePlacesRepositoryPort

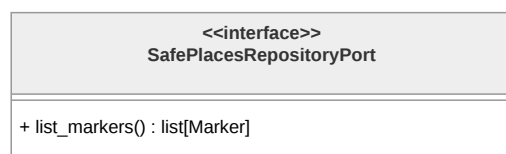


Figura 111: Diagramma dell'interfaccia `SafePlacesRepositoryPort`

Corrisponde alla outbound port del pattern esagonale. Definisce il metodo per il fetch della lista di tutti i marker disponibili, senza esporre dettagli implementativi di S3 o qualsiasi altro servizio sottostante.

3.6.2.3.6 S3SafePlacesRepository



Figura 112: Diagramma della classe `S3SafePlacesRepository`

Corrisponde all'autbound che implementa `SafePlacesRepositoryPort`. Contiene la logica per il fetch dei marker dall'istanza del bucket S3. Il costruttore `__init__()` inizializza il client S3.



3.6.2.3.7 Materiale informativo

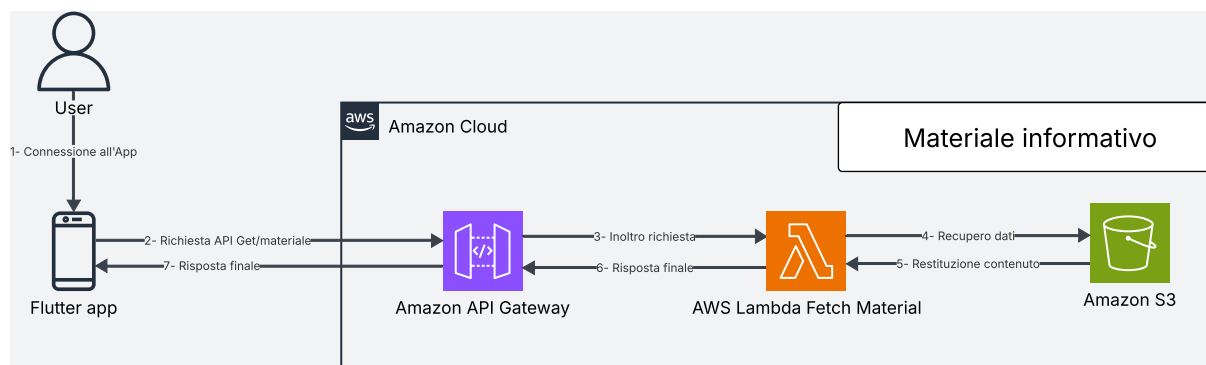


Figura 113: Architettura moduli Materiale Informativo

3.6.2.4 Architettura e Flusso di Elaborazione: Modulo Materiale Informativo

La sezione dedicata al "Materiale Informativo" è stata progettata privilegiando la massima efficienza e l'ottimizzazione dei costi operativi. Trattandosi di un modulo a prevalenza di lettura (Read-Heavy) e privo di logiche di business complesse per la manipolazione dei dati, l'Architettura prevede una funzione Lambda molto leggera, che si interfaccia con S3 esclusivamente per il fetch dei dati.

Il ciclo di elaborazione si innesca quando l'applicazione client necessita di recuperare il catalogo delle informazioni, generando una richiesta HTTPS (metodo **GET**) verso l'endpoint dedicato. L'API Gateway intercetta la chiamata e la inoltra alla funzione AWS Lambda responsabile del modulo.

La Lambda si occupa quindi di:

- recuperare i record relativi ai materiali informativi dal bucket S3;
- decodificare il contenuto per validarne l'integrità strutturale;
- serializzare il risultato aggiungendo gli header CORS necessari per renderlo consumabile dal frontend.

Il risultato elaborato viene restituito all'API Gateway, che lo inoltra al client Flutter tramite una risposta HTTP. Il ciclo sincrono si conclude con la consegna del catalogo informativo all'applicazione.

3.6.2.5 API associate

- **GET /materials/**
 - **Descrizione:** Recupera la lista delle risorse informative disponibili, tra cui riferimenti normativi, comunità di supporto e articoli informativi relativi alla violenza domestica e di genere.



– **Parametri:**

- * Nessun parametro richiesto.

– **Response:**

- * 200 OK: richiesta completata con successo.

- * Corpo della risposta:

```
[
  {
    "resource_id": "resource-123",
    "title": "Centro Antiviolenza Roma",
    "content": "Servizio di supporto per vittime.",
    "url": "https://www.centroantiviolenza.it/",
    "type": "COMMUNITY"
  },
  {
    "resource_id": "resource-456",
    "title": "Legge tutela vittime",
    "content": "Normativa nazionale violenza domestica.",
    "url": null,
    "type": "LAW"
  },
  {
    "resource_id": "resource-789",
    "title": "Riconoscere i segnali di abuso",
    "content": "Guida informativa.",
    "url": null,
    "type": "ARTICLE"
  }
]
```

– **Errori:**

- * 500 Internal Server Error: errore interno del server.

Nota di sicurezza: In maniera analoga a quanto previsto per i luoghi sicuri, anche per l'accesso al materiale informativo si è optato per non implementare l'Authorizer Cognito. Questa decisione è dettata dalla volontà di snellire l'Architettura, riducendo le complessità superflue ed evitando di incorrere nei costi operativi aggiuntivi dell'infrastruttura di autenticazione.



3.6.2.6 Architettura logica: Modulo Materiale Informativo

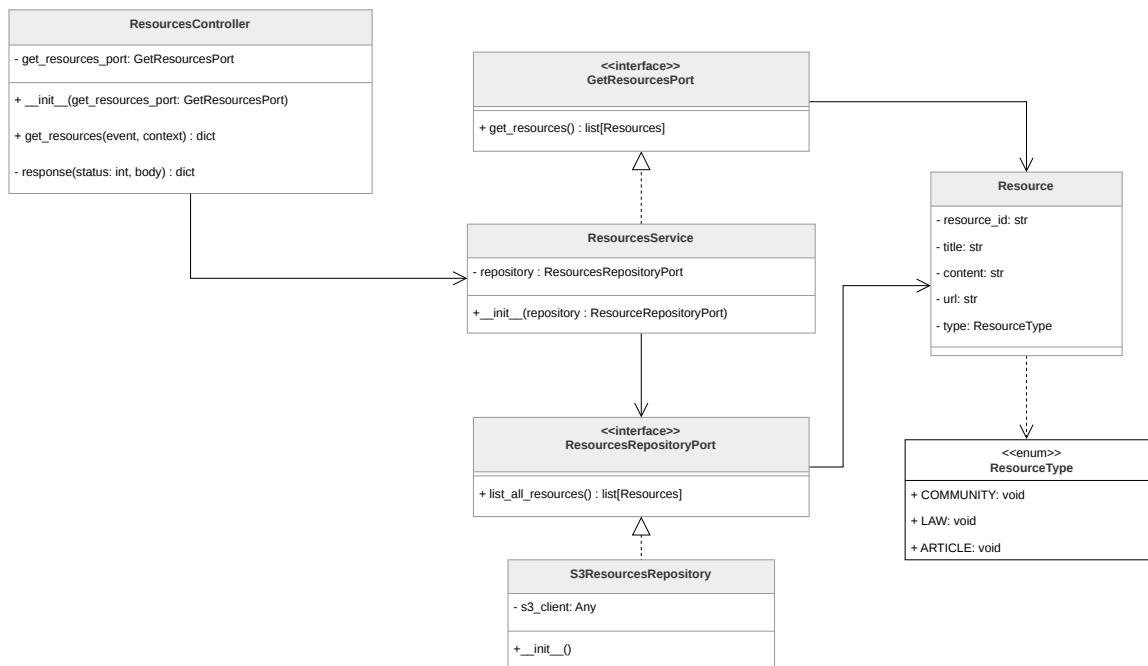


Figura 114: Componenti del microservizio materiale informativo



3.6.2.6.1 Resource

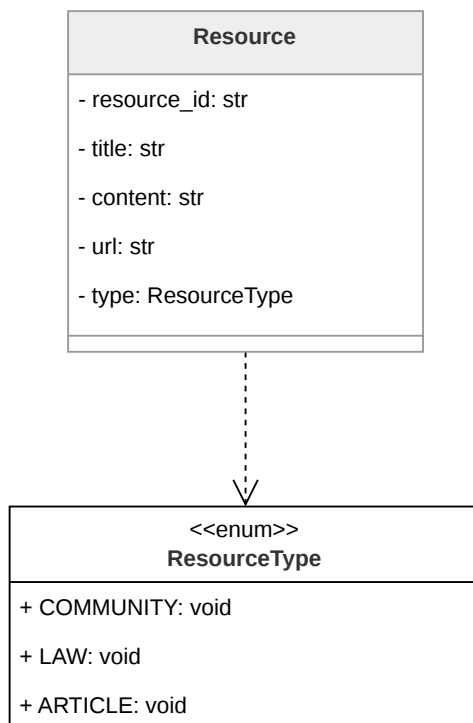


Figura 115: Diagramma della classe **Resource**

Rappresenta una risorsa informativa disponibile nell'applicazione.

Descrizione attributi:

- **resource_id**, identificativo della singola risorsa;
- **title**, nome della singola risorsa;
- **content**, testo descrittivo della singola risorsa;
- **url**, collegamento alla fonte esterna;
- **type**, corrisponde al tipo **ResourceType** e categorizza la risorsa secondo una delle tipologie previste dal dominio.

L'enumerazione **ResourceType** definisce le tre categorie disponibili: **COMMUNITY**, **LAW** e **ARTICLE**.



3.6.2.6.2 ResourcesController

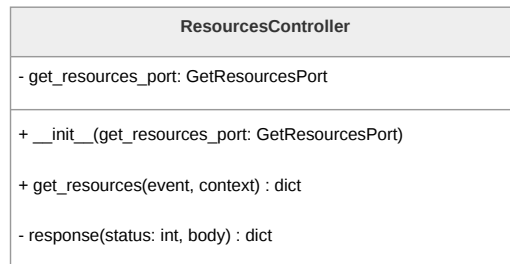


Figura 116: Diagramma della classe **ResourcesController**

Corrisponde al controller del pattern esagonale. Riceve la richiesta grezza di API Gateway, la instrada verso la inbound port e costruisce il body della risposta HTTP da restituire al client.

Descrizione metodi:

- **get_resources(event)**, costituisce il punto di ingresso del controller e riceve i parametri nativi della Lambda;
- **response(status, body)**, metodo di utilità che costruisce il dizionario di risposta nel formato atteso da API Gateway.

3.6.2.6.3 GetResourcesPort

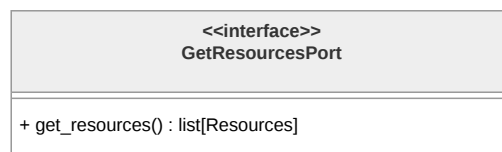


Figura 117: Diagramma della classe **GetResourcesPort**

È la inbound port del pattern esagonale. Descrive l'interfaccia per il fetch della lista completa delle risorse informative disponibili.

3.6.2.6.4 ResourcesService

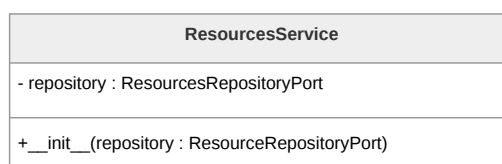


Figura 118: Diagramma della classe **ResourcesService**



Costituisce la domain logic del microservizio, implementa la inbound port **GetResourcesPort**. Contiene la logica per il recupero del materiale informativo e orchestra la comunicazione con il persistence layer tramite la outbound port **ResourcesRepositoryPort**.

3.6.2.6.5 ResourcesRepositoryPort

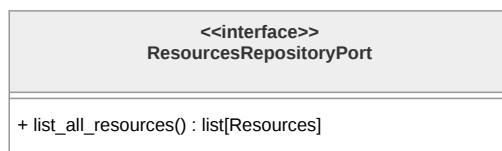


Figura 119: Diagramma della classe **ResourcesRepositoryPort**

Corrisponde all'outbound port del pattern esagonale. Definisce l'interfaccia per la restituzione della lista completa delle risorse disponibili, senza esporre dettagli implementativi relativi al servizio S3.

3.6.2.6.6 S3ResourcesRepository

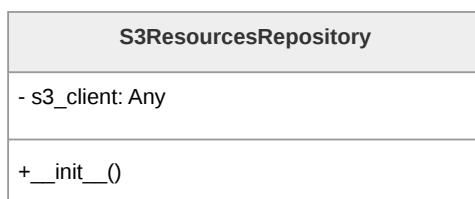


Figura 120: Diagramma della classe **S3ResourcesRepository**

Corrisponde all'outbound adapter, implementa **ResourcesRepositoryPort**. Contiene tutta la logica specifica per il recupero delle risorse informative da un bucket S3.



3.6.3 Diario criptato e fittizio

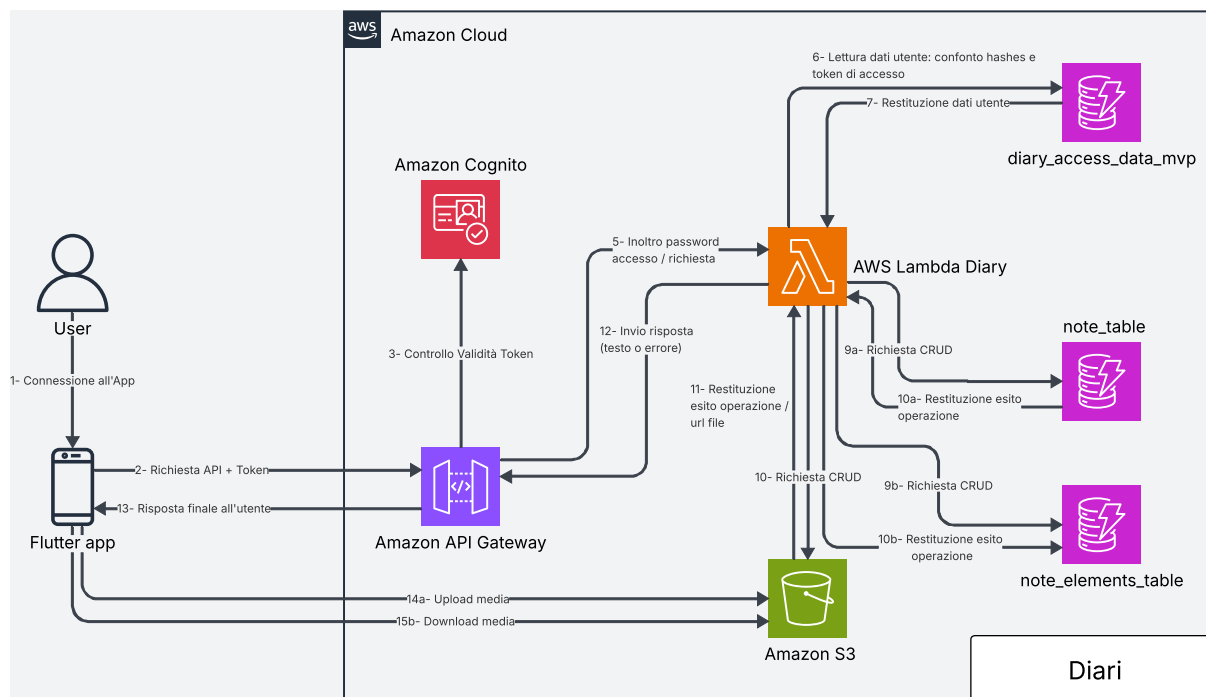


Figura 121: Architettura diari

3.6.3.1 Architettura e Flusso di Elaborazione: diario criptato e fittizio

La sezione dedicata al diario criptato e fittizio è progettata per garantire la sicurezza delle informazioni dell'Utente e la corretta gestione e archiviazione di diverse tipologie di dati, quali contenuti testuali, immagini e tracce audio. A differenza delle altre funzionalità, il diario è l'unica componente protetta da una password aggiuntiva, distinta da quella utilizzata per l'account principale.

Questo approccio consente all'applicazione di fornire accesso a risorse differenti in base alla password inserita, dato che i due diari sono protetti da credenziali separate. Poiché Amazon Cognito non supporta direttamente flussi di autenticazione di questo tipo, la gestione dell'accesso al diario avviene tramite la memorizzazione in DynamoDB degli hash delle due password. Quando l'Utente tenta l'accesso, viene generato l'hash della password inserita tramite HMAC-SHA256. Questo hash viene quindi confrontato con quello salvato nella tabella utenti. In caso di corrispondenza, l'accesso al diario viene garantito generando un token di accesso casuale, anch'esso salvato in DynamoDB. Tale token, comunicato al client in seguito al login, viene riportato nell'header di tutte le richieste a dati sensibili presenti nel diario.

L'archiviazione delle note avviene su due servizi AWS distinti. I contenuti testuali sono memorizzati in DynamoDB, che mantiene le informazioni per ciascun Utente e per ciascun diario. I file binari, come immagini e tracce audio, sono invece salvati in bucket S3. Il



recupero di tali risorse avviene tramite URL *presigned*, al fine di ridurre la quantità di dati trasferiti sulla rete. Poiché ogni nota è costituita da tanti blocchi informativi, le informazioni relative a tali blocchi sono memorizzate in una tabella DynamoDB separata rispetto a quella che tiene traccia dei metadati della nota che li contiene. Ciò garantisce maggiore modularità e rende più efficienti le operazioni di aggiornamento.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta di Autenticazione:** L'Utente inserisce la password del diario tramite app. Il client genera una richiesta HTTPS (metodo POST /diary/auth) includendo la password nel body della richiesta.
- **verifica delle Credenziali:** API Gateway verifica che l'Utente sia autenticato all'applicazione. Dopodiché inoltra la richiesta alla funzione Lambda, che calcola l'hash della password ricevuta e lo confronta con gli hash memorizzati in DynamoDB. In caso di corrispondenza, viene identificato il tipo di diario associato.
- **Generazione token di accesso:** dopo la validazione della password, un token di accesso casuale viene generato, salvato su DynamoDB e passato in risposta al client. Tale token dovrà essere fornito nell'header di ogni richiesta successiva relativa a operazioni CRUD sui contenuti del diario.
- **Recupero delle Note:** Quando l'Utente richiede di recuperare la lista di note di uno dei diari (metodo GET /notes), API Gateway invia la richiesta alla Lambda, che recupera le *preview* dal database DynamoDB e le restituisce al client.
- **Recupero di una Nota:** Per ottenere il contenuto completo di una nota (metodo GET /notes/{note_id}), la Lambda recupera i dati da DynamoDB e gli eventuali riferimenti a file su S3, costruendo l'oggetto finale che viene restituito al client.
- **Gestione dei Contenuti Binari:** In presenza di immagini o tracce audio, la Lambda genera URL *presigned* per gli oggetti memorizzati su S3, permettendo al client di accedere direttamente alle risorse senza transitare attraverso il Backend.
- **Operazioni di Scrittura:** Per la creazione o modifica di note, la Lambda memorizza in DynamoDB i metadati principali e la struttura complessiva della nota, salvando eventuali file binari su S3.
- **Eliminazione delle Note:** In caso di richiesta di eliminazione (metodo DELETE /notes/{note_id}), la Lambda rimuove i dati associati dal database e, se presenti, elimina anche i file binari corrispondenti dai bucket S3.
- **Aggiunta elemento ad una Not:** In caso di richiesta di aggiunta di un elemento, che sia composto da un'immagine, un audio o una stringa di testo, la Lambda crea una entry su DynamoDB per la memorizzazione dei metadati dell'elemento e tramite



il client di S3 crea un *URL presigned* per l'upload del file binario direttamente dal client.

- **Eliminazione elemento da una Nota:** In caso di richiesta di eliminazione di un'elemento di una nota, la Lambda rimuove i metadati da DynamoDB e, se presente, il file binario associato all'elemento presente nel bucket S3.
- **Logout:** quando l'Utente esce dal diario, il Backend viene notificato e la Lambda elimina la sessione precedentemente avviata.

3.6.3.2 API associate

Le API relative al diario criptato e fittizio sono suddivise in due categorie distinte:

- **API di autenticazione:** gestiscono login, logout, impostazione password e generazione e convalida dei token di accesso.
- **API per le operazioni CRUD:** gestiscono il recupero, la creazione, la modifica e l'eliminazione delle note. Le richieste agli endpoint che gestiscono queste operazioni devono avvenire specificando, nell'header della richiesta, il token di sessione ricevuto dopo il login al diario.
- **POST /diary/auth/login**
 - **Descrizione:** Autentica l'Utente rispetto a una tipologia di diario e restituisce un token di accesso se la password è corretta.
 - **Parametri:**
 - * **password:** password del diario.
 - **Response:**
 - * 200 OK: autenticazione completata con successo.
 - * Corpo della risposta:

```
{
  "diary_type": "REAL_DIARY",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9"
```
 - **Errori:**
 - * 400 Bad Request: JSON malformato o campo **password** mancante.
 - * 500 Internal Server Error: Errore interno al server.
- **POST /diary/auth/logout**
 - **Descrizione:** Termina la sessione dell'Utente autenticato.
 - **Parametri:**



- * Nessuno.
- **Response:**
 - * 200 OK: logout completato con successo.
 - * Corpo della risposta: vuoto
- **Errori:**
 - * 400 Bad Request: Utente non trovato.
 - * 500 Internal Server Error: Errore interno al server.
- **POST /diary/auth/set_password**
 - **Descrizione:** Imposta o aggiorna la password di un diario per l'Utente, distinguendo tra diario reale e fittizio.
 - **Parametri:**
 - * password: nuova password.
 - * previous_password: password precedente (obbligatoria dopo la prima impostazione).
 - * diary_type: tipo di diario (REAL_DIARY o FAKE_DIARY).
 - **Response:**
 - * 200 OK: password impostata correttamente.
 - * Corpo della risposta:

```
{  
  "message": "Password set successfully"  
}
```
 - **Errori:**
 - * 400 Bad Request: diary_type non valido, password non conforme ai requisiti di sicurezza, password mancante o precedente errata, JSON malformato o campi mancanti.
 - * 500 Internal Server Error: errore di persistenza della password.
- **POST /diary/auth/status**
 - **Descrizione:** Restituisce lo stato di configurazione delle password dei diari dell'Utente autenticato.
 - **Parametri:**
 - * Nessuno.
 - **Response:**
 - * 200 OK: richiesta completata con successo.



```
* Corpo della risposta:
{
  "has_real_password": true,
  "has_fake_password": false
}
```

– **Errori:**

- * 400 Bad Request: JSON malformato o Utente non trovato.
- * 500 Internal Server Error: errore interno al server.

● **PUT /notes**

– **Descrizione:** Crea una nuova nota associata all'Utente, composta da metadati e una lista di elementi.

– **Parametri:**

- * **user_id:** identificativo dell'Utente;
- * **title:** titolo della nota;
- * **created_at:** timestamp di creazione;
- * **last_modified_at:** timestamp dell'ultima modifica;
- * **diary_type:** tipo di diario (REAL_DIARY o FAKE_DIARY);
- * **elements:** lista di elementi della nota:
 - **type:** tipo dell'elemento (text, image, audio);
 - **content:** obbligatorio solo nel caso venga inserito del testo, identifica il contenuto testuale inserito dall'Utente.

– **Response:**

- * 200 OK: operazione completata con successo.
- * Corpo della risposta:

```
{
  "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
  "user_id": "user1",
  "title": "Titolo",
  "created_at": "2026-02-22T15:22:27.615449+00:00",
  "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
  "diary_type": "REAL_DIARY"
  "note_elements": [
    {
      "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
      "note_element_id": "01KQM5R2QQQ3GTV7026YGY7PD1",
      "type": "text",
```



```

        "content": "testo"
    },
    {
        "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
        "note_element_id": "01KQM5R2QRHD5Q38YEWSXNFJ2P",
        "type": "image",
        "content": "key_s3",
        "upload_url": "https://presigned-url"
    }
]
}

```

– **Errori:**

- * 400 Bad Request: Invalid diary_type (ValueError su enum DiaryType).
- * 500 Internal Server Error: errori di service.add_note, parsing JSON non valido, campi mancanti o errori runtime non gestiti.

• **GET /notes/{diary_type}**

– **Descrizione:** Recupera la lista delle note dell'Utente filtrate per tipo diario.

– **Parametri:**

- * user_id: identificativo dell'Utente;
- * diary_type: tipo di diario.

– **Response:**

- * 200 OK: liste delle note recuperata con successo.
- * Corpo della risposta:

```

{
    "notes": [
        {
            "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
            "user_id": "user1",
            "title": "Titolo",
            "created_at": "2026-02-22T15:22:27.615449+00:00",
            "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
            "diary_type": "REAL_DIARY"
        },
        {
            "note_id": "01KQM5R2QQQ3GTV7026YGY7PD1",
            "user_id": "user1",

```



```

        "title": "Titolo",
        "created_at": "2026-02-22T15:22:27.615449+00:00",
        "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
        "diary_type": "REAL_DIARY"
    }
]
}

```

– **Errori:**

- * 400 Bad Request: Invalid diary_type (ValueError su enum DiaryType).
- * 500 Internal Server Error: errore di service.list_notes.

• **GET /notes/{diary_type}/{note_id}**

– **Descrizione:** una nota completa con i suoi elementi recuperata con successo.

– **Parametri:**

- * user_id: identificativo dell'Utente;
- *
- * note_id: identificativo della nota;
- * diary_type: tipo di diario.

– **Response:**

- * 200 OK: : nota recuperata con successo.
- * Corpo della risposta:

```

{
    "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
    "user_id": "user1",
    "title": "Titolo",
    "created_at": "2026-02-22T15:22:27.615449+00:00",
    "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
    "diary_type": "REAL_DIARY"
    "note_elements": [
        {
            "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
            "note_element_id": "01KQM5R2QQQ3GTV7026YGY7PD1",
            "type": "text",
            "content": "testo"
        },
        {
            "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",

```



```

        "note_element_id": "01KQM5R2QRHD5Q38YEWSXNFJ2P",
        "type": "image",
        "content": "key_s3",
        "download_url": "https://presigned-url"
    }
]
}

```

– **Errori:**

- * 400 Bad Request: diary_type non valido.
- * 404 Not Found: nota non trovata.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /notes/{diary_type}/{note_id}**

– **Descrizione:** Elimina una nota e di tutti i suoi elementi che la compongono.

– **Parametri:**

- * note_id: identificativo della nota;
- * user_id: identificativo dell'Utente;
- * diary_type: tipo di diario.

– **Response:**

- * 200 OK: eliminazione completata con successo.
- * Corpo della risposta:


```

{
    "message": "Note deleted successfully"
}

```

– **Errori:**

- * 400 Bad Request: campi obbligatori mancanti.
- * 404 Not Found: nota non trovata.
- * 500 Internal Server Error: errore interno del server.

• **PUT /notes/note_element**

– **Descrizione:** Aggiunge un elemento ad una nota esistente.

– **Parametri:**

- * note_id: identificativo della nota a cui si vuole aggiungere l'elemento;
- * type: tipo del contenuto inserito (text, image, audio)
- * content: obbligatorio solo nel caso venga inserito del testo, identifica il contenuto testuale inserito dall'Utente.



– **Response:**

- * 200 OK: elemento aggiunto con successo alla nota.
- * Corpo della risposta:

```
{
  "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
  "note_element_id": "01KQM5R2QRHD5Q38YEWSXNFJ2P",
  "type": "image",
  "content": "key_s3",
  "upload_url": "https://presigned-url"
}
```

– **Errori:**

- * 400 Bad Request: campi mancanti o incoerenti.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /notes/note_element/{note_id}/{note_element_id}**

– **Descrizione:** Elimina un elemento da una nota (e il file associato se presente).

– **Parametri:**

- * **note_id:** identificativo della nota a cui si vuole eliminare l'elemento;
- * **note_element_id:** identificativo dell'elemento della note che si vuole eliminare;
- * **type:** tipo del contenuto che si vuole eliminare;
- * **content:** contenuto dell'elemento da eliminare.

– **Response:**

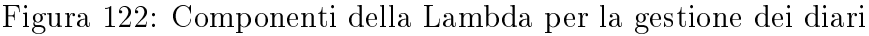
- * 200 OK:

```
{
  "message": "Note element deleted successfully"
}
```

– **Errori:**

- * 400 Bad Request: campi obbligatori mancanti.
- * 500 Internal Server Error: errore interno del server.

111





3.6.3.3.1 DiaryType

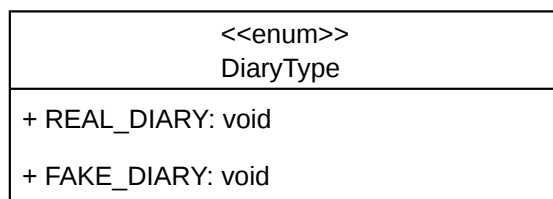


Figura 123: Diagramma dell'enumerazione **DiaryType**

L'enum **DiaryType** definisce i due tipi di diario supportati dal sistema: **REAL_DIARY** per il diario reale contenente i dati autentici dell'Utente e **FAKE_DIARY** per il diario fittizio.

3.6.3.3.2 Note

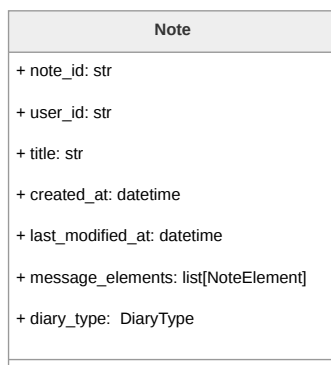


Figura 124: Diagramma della classe **Note**

La classe **Note** rappresenta l'entità di dominio principale del sistema, modellando una nota completa del diario. Include gli identificativi univoci (**note_id**, **user_id**), i metadati temporali (**created_at**, **last_modified_at**), il titolo della nota e il tipo di diario di appartenenza (**diary_type**). La nota è composta da una collezione di elementi di tipo **NoteElement**. La serializzazione della classe avviene tramite **NoteDTO**.



3.6.3.3.3 NoteElement

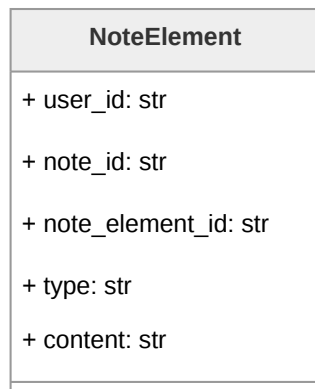


Figura 125: Diagramma della classe **NoteElement**

NoteElement modella un singolo blocco di contenuto all'interno di una nota. Ogni elemento è identificato univocamente tramite **note_element_id** e mantiene un riferimento alla nota di appartenenza (**note_id**) e all'Utente proprietario (**user_id**). Il campo **type** specifica la tipologia del contenuto (testo, immagine, audio), mentre **content** ne contiene il valore effettivo. Questa granularità permette di gestire note composite e strutturate, supportando l'aggiunta e rimozione incrementale di elementi. La classe viene serializzata tramite **NoteElementDTO**.

3.6.3.3.4 NoteElementDTO

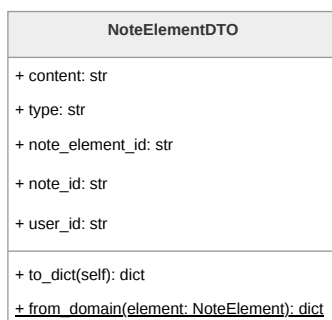


Figura 126: Diagramma della classe **NoteElementDTO**

La classe **NoteElementDTO** fornisce il mapping bidirezionale tra l'entità di dominio **NoteElement** e il formato di persistenza su DynamoDB.



3.6.3.3.5 NoteDTO

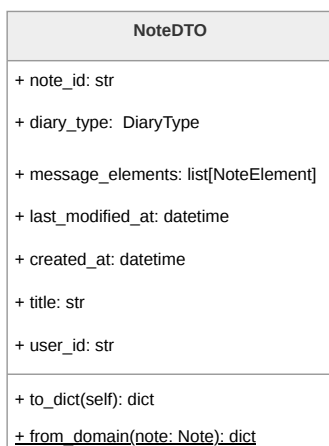


Figura 127: Diagramma della classe **NoteDTO**

La classe **NoteDTO** gestisce la conversione tra l'entità di dominio **Note** e il formato di persistenza su DynamoDB.

3.6.3.3.6 GetNoteCmd

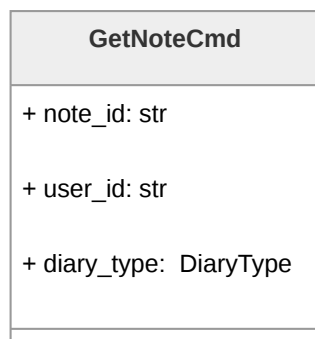


Figura 128: Diagramma della classe **GetNoteCmd**

Il command object **GetNoteCmd** incapsula i parametri necessari per richiedere il recupero di una singola nota. Contiene l'identificativo della nota (**note_id**), l'identificativo dell'Utente richiedente (**user_id**) e il tipo di diario da cui recuperare la nota (**diary_type**). Questo oggetto viene consumato dal metodo **get_note** dell'interfaccia **GetNotePort**, implementata da **NoteService**, che delega il recupero effettivo al repository di persistenza.



3.6.3.3.7 GetNotesCmd

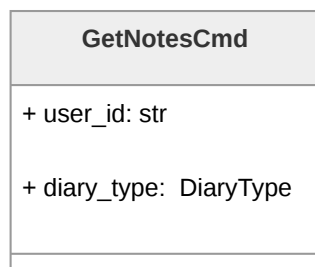


Figura 129: Diagramma della classe **GetNotesCmd**

GetNotesCmd rappresenta la richiesta dell'elenco delle note appartenenti a un Utente. Include l'identificativo dell'Utente (**user_id**) e il tipo di diario da interrogare (**diary_type**). Viene utilizzato dal metodo **list_notes** di **GetNotePort** per recuperare l'insieme completo delle note dell'Utente, mantenendo l'isolamento tra diario reale e fittizio.

3.6.3.3.8 AddElementNewNoteCmd

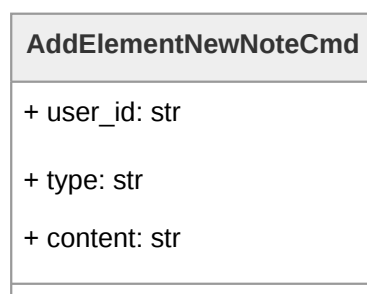


Figura 130: Diagramma della classe **AddElementNewNoteCmd**

Il command **AddElementNewNoteCmd** rappresenta la richiesta dell'aggiunta di un blocco di contenuto destinato a far parte di una nuova nota in fase di creazione. Contiene l'identificativo dell'Utente proprietario (**user_id**), la tipologia dell'elemento (**type**) e il suo contenuto (**content**). Questo oggetto viene utilizzato all'interno di **AddNoteCmd** come componente della lista **message_elements**, permettendo la creazione di note con elementi preimpostati.



3.6.3.3.9 AddNoteCmd

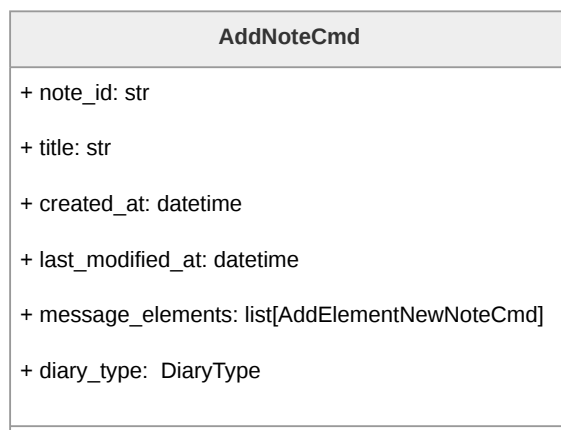


Figura 131: Diagramma della classe **AddNoteCmd**

AddNoteCmd incapsula tutti i dati necessari per la creazione di una nuova nota completa. Include l'identificativo della nota, il titolo, i timestamp di creazione e ultima modifica, il tipo di diario di destinazione e le richieste di creazione per i singoli elementi della nota. **AddNoteCmd** viene consumato dal metodo **add_note** dell'interfaccia **SetNotePort**, implementata da **NoteService**.

3.6.3.3.10 AddNoteElementCmd

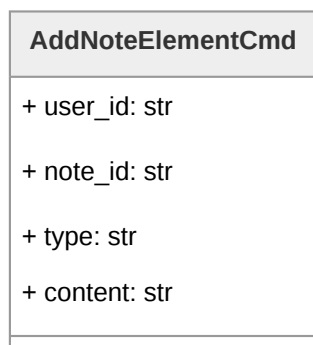


Figura 132: Diagramma della classe **AddNoteElementCmd**

Il command object **AddNoteElementCmd** rappresenta la richiesta di aggiunta di un singolo elemento a una nota esistente. Contiene l'identificativo dell'Utente e della nota target, la tipologia del nuovo elemento e il suo contenuto. Viene processato dal metodo **add_note_element** dell'interfaccia **SetNoteElementPort**, che delega a **NoteService** la creazione dell'entità **NoteElement** e la successiva persistenza tramite il repository.



3.6.3.3.11 DeleteNoteCmd

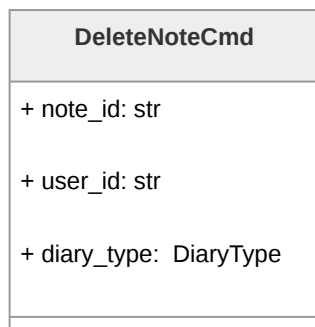


Figura 133: Diagramma della classe `DeleteNoteCmd`

`DeleteNoteCmd` incapsula i parametri per l'eliminazione di una nota. Include l'identificativo della nota da rimuovere, quello dell'Utente richiedente e il tipo di diario da cui eliminare la risorsa. Questo command viene consumato dal metodo `delete_note` di `DeleteNotePort`, che coordina sia la rimozione dal repository di persistenza che l'eventuale cleanup degli allegati file tramite `FileRepositoryPort`.

3.6.3.3.12 DeleteNoteElementCmd

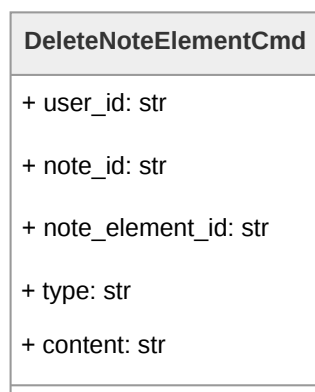


Figura 134: Diagramma della classe `DeleteNoteElementCmd`

Il command `DeleteNoteElementCmd` modella la richiesta di rimozione di un singolo elemento da una nota esistente. Contiene l'identificativo dell'Utente, della nota di riferimento e dell'elemento specifico (`note_element_id`). Viene processato dal metodo `delete_note_element` dell'interfaccia `DeleteNoteElementPort`, implementata da `NoteService`.



3.6.3.3.13 ValidatePasswordCmd

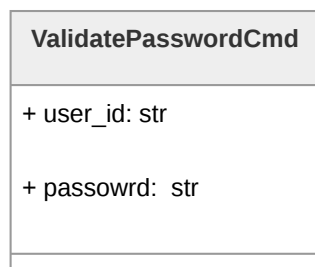


Figura 135: Diagramma della classe `ValidatePasswordCmd`

`ValidatePasswordCmd` rappresenta la richiesta di validazione delle credenziali di accesso al diario. Include l'identificativo dell'Utente e la password da verificare. Questo command viene consumato dal metodo `validate_password` dell'interfaccia `ValidationPort`, implementata da `DiaryAuthService`, che delega la verifica al repository di autenticazione e restituisce il tipo di diario corrispondente alla password fornita, permettendo così l'accesso selettivo al diario reale o fittizio.

3.6.3.3.14 ValidateTokenCmd

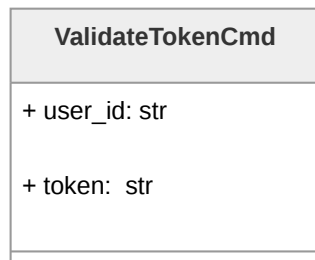


Figura 136: Diagramma della classe `ValidateTokenCmd`

`ValidateTokenCmd` rappresenta la richiesta di validazione del token associato alla sessione di accesso al diario corrente. Include l'identificativo dell'Utente e il token da verificare. Questo command viene consumato dal metodo `validate_token` dell'interfaccia `ValidationPort`, implementata da `DiaryAuthService`, che delega la verifica al repository di autenticazione e restituisce un booleano che indica se il token inserito corrisponde a quello presente nella tabella DynamoDB.



3.6.3.3.15 SetPasswordCmd

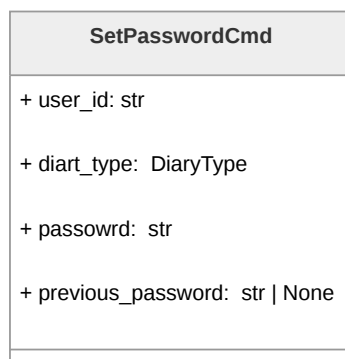


Figura 137: Diagramma della classe **SetPasswordCmd**

Il command object **SetPasswordCmd** incapsula i dati necessari per l'impostazione o l'aggiornamento delle password del diario criptato o fittizio. Include l'identificativo Utente, il tipo di diario di riferimento, la nuova password (**password**) e opzionalmente la password precedente (**previous_password**) per operazioni di modifica. Viene utilizzato dai metodi **set_real_password** e **set_fake_password** dell'interfaccia **SetPasswordPort**, permettendo la configurazione indipendente delle credenziali per ciascun tipo di diario.

3.6.3.3.16 CheckPasswordCmd

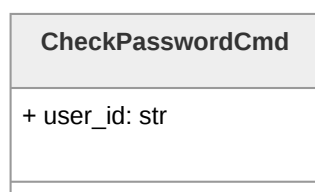


Figura 138: Diagramma della classe **CheckPasswordCmd**

CheckPasswordCmd modella la richiesta di verifica dello stato di configurazione delle password per un Utente. Contiene esclusivamente l'identificativo dell'Utente e viene consumato dal metodo **check_password_status** dell'interfaccia **CheckPasswordPort** per determinare quali credenziali di accesso sono state configurate. Ciò è necessario per permettere al frontend di mostrare una pagina di inserimento password dedicata nel caso in cui l'Utente non abbia mai impostato la password per il diario con cui vuole interagire.



3.6.3.3.17 GetNotePort

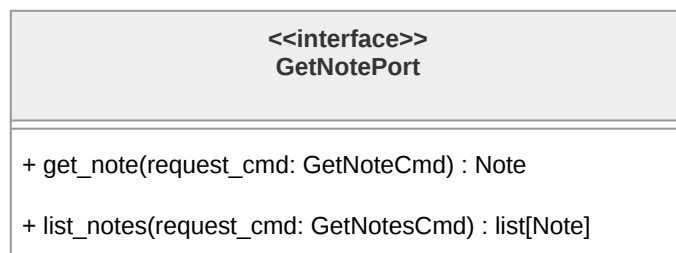


Figura 139: Diagramma dell'interfaccia **GetNotePort**

L'interfaccia **GetNotePort** definisce il contratto per le operazioni di lettura delle note. Tali operazioni di lettura includono il fetch di una singola nota e il fetch di tutte le note di un singolo Utente. **GetNotePort** viene implementata da **NoteService**, che orchestra il recupero dei dati tramite **NoteRepositoryPort**.

3.6.3.3.18 SetNotePort

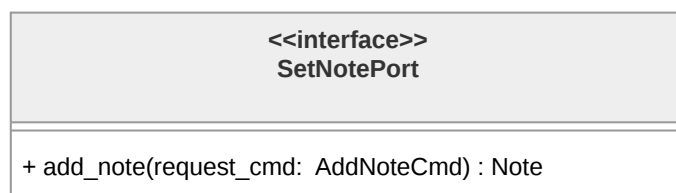


Figura 140: Diagramma dell'interfaccia **SetNotePort**

L'interfaccia **SetNotePort** definisce il contratto per la creazione di una nuova nota. È implementata da **NoteService**.

3.6.3.3.19 DeleteNotePort



Figura 141: Diagramma dell'interfaccia **DeleteNotePort**

L'interfaccia **DeleteNotePort** definisce il contratto per l'eliminazione di un'intera nota. È implementata da **NoteService**, che orchestra sia la rimozione della nota dal repository che l'eventuale cleanup dei file allegati attraverso **FileRepositoryPort**.



3.6.3.3.20 SetNoteElementPort

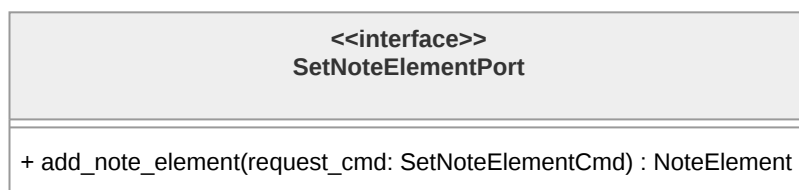


Figura 142: Diagramma dell'interfaccia **SetNoteElementPort**

L'interfaccia **SetNoteElementPort** definisce il contratto per l'aggiunta di un elemento di contenuto a una nota esistente. Il metodo **add_note_element** accetta un **AddNoteElementCmd** e restituisce l'entità **NoteElement** creata. È implementata da **NoteService**.

3.6.3.3.21 DeleteNoteElementPort



Figura 143: Diagramma dell'interfaccia **DeleteNoteElementPort**

L'interfaccia **DeleteNoteElementPort** definisce il contratto per la rimozione di un singolo elemento da una nota esistente. È implementata da **NoteService**.

3.6.3.3.22 SetPasswordPort

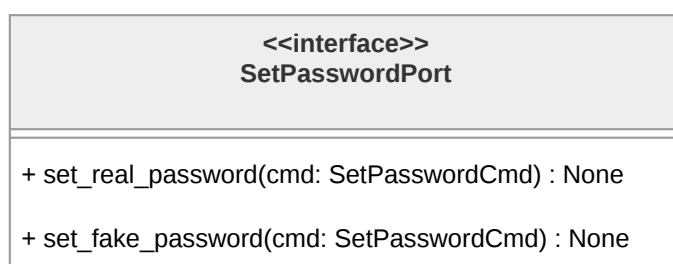


Figura 144: Diagramma dell'interfaccia **SetPasswordPort**

L'interfaccia **SetPasswordPort** definisce il contratto per la configurazione delle credenziali di accesso. Espone due metodi distinti: **set_real_password** per l'impostazione della password del diario reale e **set_fake_password** per quella del diario fittizio, entrambi accettando un **SetPasswordCmd**. È implementata da **DiaryAuthService**.



3.6.3.3.23 ValidationPort

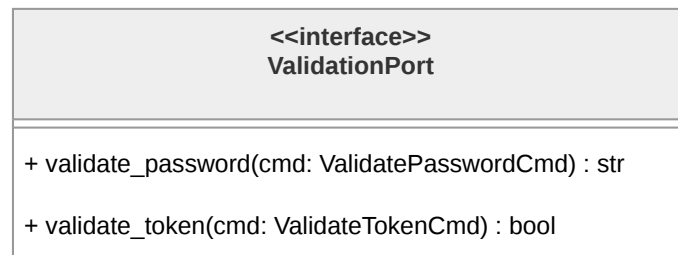


Figura 145: Diagramma dell'interfaccia **ValidationPort**

L'interfaccia **ValidationPort** definisce il contratto per la validazione delle credenziali di accesso al diario. Il metodo `validate_password` accetta un **ValidatePasswordCmd** e restituisce una stringa rappresentante il tipo di diario corrispondente alla password fornita se una corrispondenza viene effettivamente trovata. Il metodo `validate_token` accetta un **ValidateTokenCmd** e restituisce un booleano indicante la validità del token. È implementata da **DiaryAuthService**.

3.6.3.3.24 CheckPasswordPort

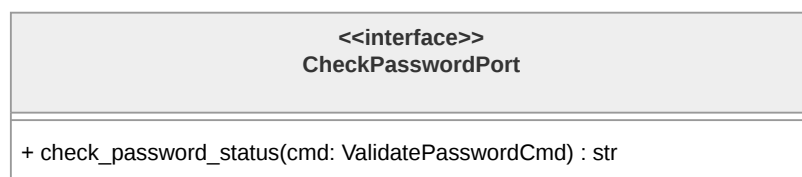


Figura 146: Diagramma dell'interfaccia **CheckPasswordPort**

L'interfaccia **CheckPasswordPort** definisce il contratto per la verifica dello stato di configurazione delle password. È implementata da **DiaryAuthService**.



3.6.3.3.25 NoteRepositoryPort

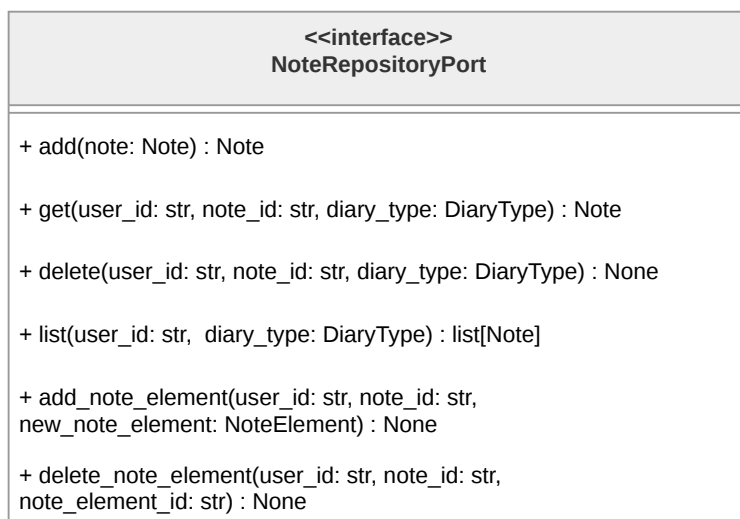


Figura 147: Diagramma dell'interfaccia **NoteRepositoryPort**

L'interfaccia **NoteRepositoryPort** definisce il contratto per la persistenza delle note e dei loro elementi. Espone i metodi **add** per la creazione di note, **get** per il recupero di una nota specifica, **delete** per la rimozione, **list** per l'elenco completo delle note di un Utente, **add_note_element** per l'inserimento di elementi in note esistenti e **delete_note_element** per la loro rimozione. Tutti i metodi accettano il parametro **diary_type** per garantire l'isolamento tra diario reale e fittizio. Questa porta di uscita viene implementata da **DynamoNoteAdapter**, che fornisce la persistenza concreta su DynamoDB.

3.6.3.3.26 FileRepositoryPort

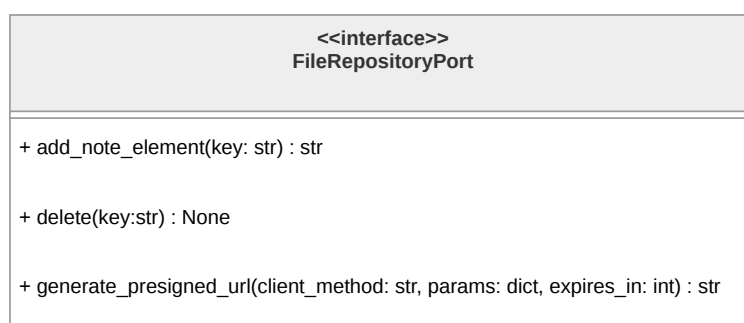


Figura 148: Diagramma dell'interfaccia **FileRepositoryPort**

Costituisce l'outbound adapter, definisce i metodi per la gestione degli file (immagini o audio) inseriti all'interno delle note del diario. Espone il metodo **add_note_element** che accetta una chiave e restituisce l'URL di accesso, **delete** per la rimozione di file tramite chiave e **generate_presigned_url** per la generazione di URL firmati con parametri e



scadenza configurabili. **FileRepositoryPort** è implementata da **S3NoteAdapter**, che fornisce l'integrazione concreta con Amazon S3.

3.6.3.3.27 DiaryAuthRepositoryPort

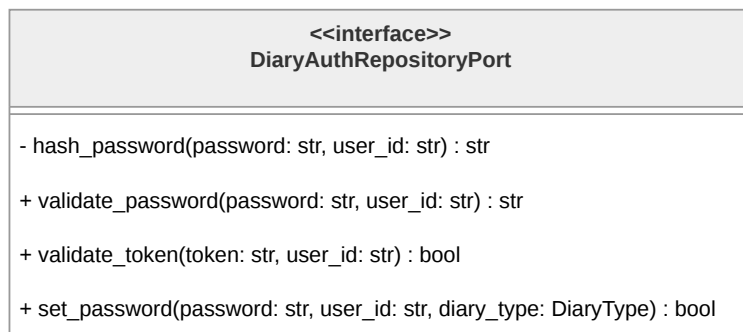


Figura 149: Diagramma dell'interfaccia **DiaryAuthRepositoryPort**

Costituisce l'outbound adapter, definisce i metodi per la persistenza e validazione delle credenziali di accesso. Espone il metodo **hash_password** per la generazione dell'hash di una password, **validate_password** per la verifica delle credenziali fornite restituendo il tipo di diario corrispondente, e **set_password** per la persistenza delle credenziali associate a uno specifico **diary_type**. Implementata da **DynamoAuthAdapter**, questa porta permette a **DiaryAuthService** di gestire l'autenticazione mantenendo separati gli hash delle password per diario reale e fittizio.

3.6.3.3.28 NoteService

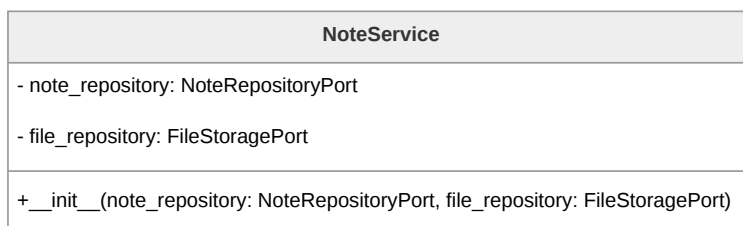


Figura 150: Diagramma della classe **NoteService**

Rappresenta la domain logic per la gestione delle note, implementando le interfacce **GetNotePort**, **SetNotePort**, **DeleteNotePort**, **SetNoteElementPort** e **DeleteNoteElementPort**. Coordina l'intera Business Logic delle note: gestisce le operazioni CRUD e la comunicazione con le porte per i servizi di persistenza. Tali porte vengono passate alla classe tramite dependency injection nel costruttore.



3.6.3.3.29 DiaryAuthService

DiaryAuthService
- auth_repository: AuthRepositoryPort
+ __init__(auth_repository: AuthRepositoryPort)

Figura 151: Diagramma della classe **DiaryAuthService**

Rappresenta la domain logic per la gestione dell'autenticazione al diario, che include l'impostazione della password di entrambi i diari e la loro verifica. Si occupa inoltre di verificare che la password impostata dall'Utente sia conforme a specifici criteri di sicurezza. **DiaryAuthService** riceve tramite costruttore la dipendenza **auth_repository** di tipo **DiaryAuthRepositoryPort** per l'orchestrazione delle operazioni relative alla persistenza dei dati di accesso degli utenti.

3.6.3.3.30 DynamoNoteAdapter

DynamoNoteAdapter
- dynamodb_client : Any
+ __init__()

Figura 152: Diagramma della classe **DynamoNoteAdapter**

Rappresenta l'outbound adapter, implementa l'interfaccia **NoteRepositoryPort** fornendo la persistenza concreta delle note su DynamoDB. È responsabile di effettuare le richieste concrete a DynamoDB e a ritornare i risultati ottenuti, implementando concretamente le varie operazioni CRUD. L'adapter utilizza **NoteDTO** e **NoteElementDTO** per la conversione bidirezionale tra le entità di dominio e il formato di persistenza su DynamoDB, isolando completamente il layer applicativo dai dettagli implementativi dello storage.



3.6.3.3.31 S3NoteAdapter

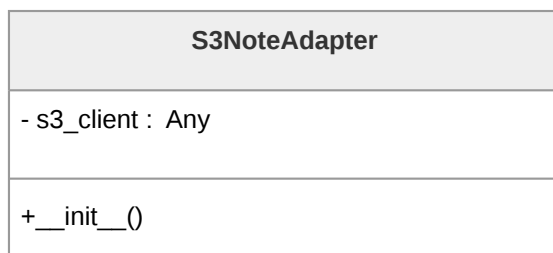


Figura 153: Diagramma della classe **S3NoteAdapter**

Rappresenta l'outbound adapter, implementa l'interfaccia **FileRepositoryPort** fornendo la gestione degli allegati file su Amazon S3. Gestisce la generazione dei `presigned_url` e le chiamate a S3 per l'archiviazione o il recupero dei file binari. Viene utilizzato da **NoteService** per gestire il ciclo di vita completo degli allegati associati alle note.

3.6.3.3.32 DynamoAuthAdapter

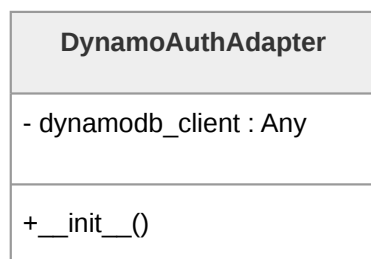


Figura 154: Diagramma della classe **DynamoAuthAdapter**

Rappresenta l'outbound adapter, implementa l'interfaccia **DiaryAuthRepositoryPort** fornendo la persistenza delle credenziali di autenticazione su DynamoDB. Mantiene un riferimento al client DynamoDB (`dynamodb_client`) inizializzato nel costruttore. Il metodo `hash_password` genera l'hash sicuro di una password per un Utente specifico tramite HMAC-SHA-256 applicato sulla concatenazione tra l'id dell'Utente e la password stessa. `validate_password` Verifica le credenziali fornite restituendo il tipo di diario corrispondente all'hash validato, e `set_password` memorizza o aggiorna le credenziali associate a uno specifico `diary_type`.



3.6.3.3.33 DiaryNoteController

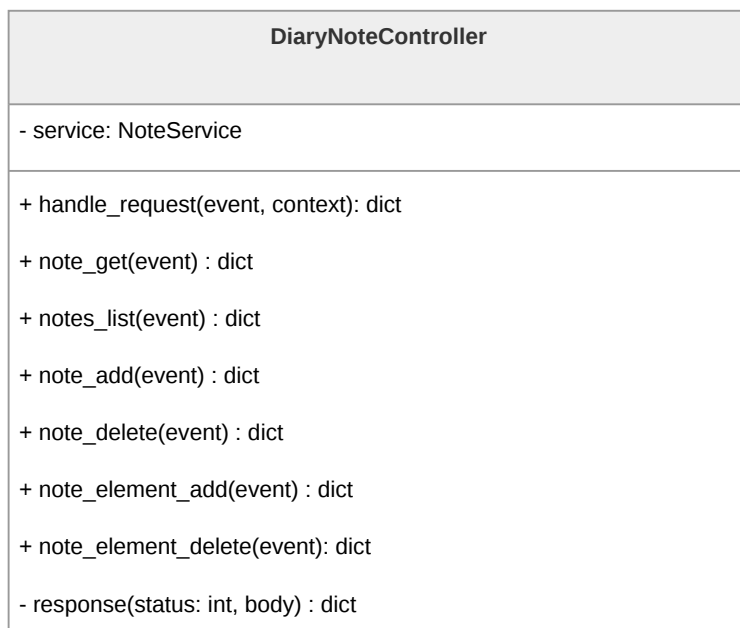


Figura 155: Diagramma della classe **DiaryNoteController**

La classe **DiaryNoteController** costituisce il punto di ingresso per le richieste relative alle operazioni sulle note. Mantiene un riferimento a **NoteService** che orchestra la Business Logic. Il metodo **handle_request** funge da router principale, interpretando la richiesta in arrivo dal Lambda handler e delegandola alle funzioni specifiche.



3.6.3.34 DiaryAccessController



Figura 156: Diagramma della classe **DiaryAccessController**

La classe **DiaryAccessController** costituisce il punto di ingresso per le operazioni di autenticazione. Mantiene un riferimento a **DiaryAuthService** che implementa la logica di gestione delle credenziali. Il metodo **handle_request** funge da router principale, interpretando la richiesta in arrivo dal Lambda handler e delegandola alle funzioni specifiche.

3.6.4 Chatbot

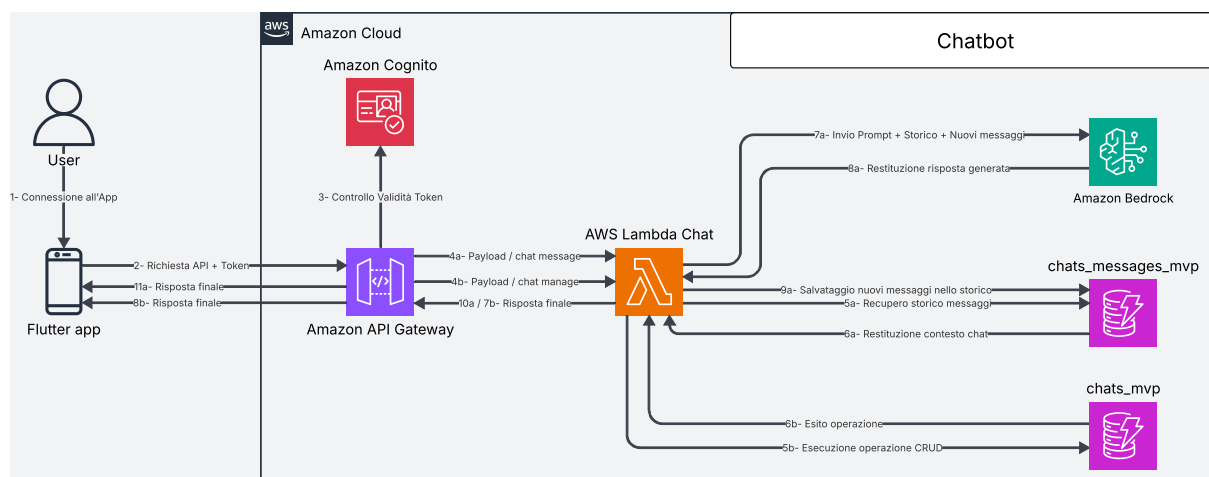


Figura 157: Architettura chatbot



3.6.4.1 Architettura e Flusso di Elaborazione: chatbot (detective delle relazioni e specchio intelligente)

L'Architettura di Backend dedicata al chatbot è progettata per gestire conversazioni persistenti tra Utente e modello linguistico, garantendo scalabilità, efficienza nell'accesso ai dati e qualità delle risposte generate.

API Gateway rappresenta il punto di ingresso per tutte le richieste client e si occupa della verifica dell'autenticazione dell'Utente. Una volta validata la richiesta, questa viene inoltrata a una funzione Lambda che implementa la logica applicativa del chatbot, inclusa la gestione delle chat e l'interazione con il modello linguistico.

Le conversazioni sono strutturate come entità persistenti (chat), ciascuna identificata da un `chat_id` univoco all'interno dell'intero sistema. Per garantire una maggiore modularità e facilitare il recupero delle informazioni, chat e messaggi sono memorizzati in tabelle DynamoDB separate. Ogni chat contiene una sequenza di messaggi (Utente e assistente artificiale), anch'essi memorizzati in DynamoDB. Per ottimizzare le prestazioni e ridurre il carico di rete, le operazioni di recupero dello storico restituiscono esclusivamente informazioni di sintesi (ad esempio titolo e metadata), mentre il contenuto completo dei messaggi viene fornito solo su richiesta esplicita.

Quando l'Utente invia un messaggio, il sistema non si limita a inoltrarlo direttamente al modello, ma recupera le informazioni più rilevanti dalla chat in corso. In particolare, la Lambda recupera il contesto rilevante della conversazione dalla base di dati (DynamoDB), costruendo un prompt arricchito che include la cronologia significativa dei messaggi. Per estrarre tali informazioni, la Lambda utilizza tecniche di ricerca lessicale basate su BM25, evitando di concatenare l'intera conversazione al testo inviato dall'Utente. Questo approccio consente di migliorare la coerenza e la pertinenza delle risposte generate dal modello.

Il prompt così costruito viene inviato a Bedrock, che restituisce la risposta generata dal LLM. La risposta, insieme al messaggio dell'Utente, viene quindi memorizzata in DynamoDB come parte della chat. Nel caso in cui il messaggio inviato sia il primo di una nuova chat, la Lambda imposta il titolo della chat alle prime tre parole del messaggio dell'Utente.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta:** L'Utente interagisce con il chatbot tramite app, generando una richiesta HTTPS verso uno degli endpoint disponibili (ad esempio `POST /chats/{chat_id}/messages`).
- **Autenticazione:** API Gateway verifica che l'Utente sia autenticato all'applicazione. In caso positivo, la richiesta viene inoltrata alla funzione Lambda responsabile della logica del chatbot.



- **Gestione della Chat:** La Lambda identifica la chat di riferimento tramite `chat_id`. Se necessario, recupera i metadati o verifica l'esistenza della chat in DynamoDB.
- **Recupero del Contesto (RAG):** Prima di inviare il messaggio al modello, la Lambda esegue una query su DynamoDB per ottenere i messaggi precedenti rilevanti della conversazione, costruendo il contesto da includere nel prompt.
- **Costruzione del Prompt:** Il messaggio dell'Utente viene combinato con il contesto recuperato, formando un prompt strutturato da inviare al modello linguistico.
- **Invocazione del Modello:** La Lambda invia il prompt a Bedrock, che elabora la richiesta e restituisce una risposta generata.
- **Persistenza dei Messaggi:** La Lambda salva sia il messaggio dell'Utente sia la risposta del modello in DynamoDB, associandoli alla chat corrente.
- **Aggiornamento dei Metadati:** Se necessario, la Lambda aggiorna le informazioni della chat (ad esempio titolo o timestamp dell'ultima attività).
- **Restituzione della Risposta:** La risposta generata dal modello, insieme ai metadati aggiornati, viene restituita al client.
- **Recupero dello Storico:** Quando l'Utente richiede la lista delle chat (metodo `GET /chats/`), la Lambda restituisce solo le *preview* delle chat, senza includere l'intero contenuto dei messaggi.
- **Recupero di una Chat:** Per ottenere il contenuto completo di una chat (metodo `GET /chats/{chat_id}`), la Lambda recupera tutti i messaggi associati da DynamoDB e li restituisce al client.
- **Operazioni CRUD:** Le operazioni di creazione, aggiornamento ed eliminazione delle chat vengono gestite direttamente dalla Lambda tramite interazioni con DynamoDB.

3.6.4.2 API associate

- **GET /chats/**
 - **Descrizione:** Recupera la lista delle chat associate all'Utente autenticato, restituendo solo le informazioni principali, senza i messaggi.
 - **Parametri:**
 - * `user_id`: identificativo dell'Utente.



– **Response:**

* 200 OK: richiesta completata con successo.

* Corpo della risposta:

```
{
  "chats": [
    {
      "chat_id": "01KPX72J3Y5MFM9982J3TFTH7H",
      "title": "Supporto emotivo",
      "created_at": "2026-02-22T15:22:27.615449+00:00",
      "updated_at": "2026-04-22T18:13:05.452786+00:00",
      "messages": []
    },
    {
      "chat_id": "01KPX8AB4Z9QTR672L8VYH3D2C",
      "title": "Violenza domestica",
      "created_at": "2026-03-05T09:11:42.123456+00:00",
      "updated_at": "2026-04-24T16:47:19.987654+00:00",
      "messages": []
    }
  ]
}
```

– **Errori:**

* 400 Bad Request: parametri mancanti o non validi.

* 401 Unauthorized: Utente non autenticato.

* 500 Internal Server Error: errore interno del server.

• **POST /chats/**

– **Descrizione:** Crea una nuova chat associata all'Utente autenticato, inizializzata senza messaggi.

– **Parametri:**

* title: titolo della chat.

– **Response:**

* 201 Created: chat creata con successo.

* Corpo della risposta:

```
{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KQ52JAEYD7ZDH894CN9HEH05",
  "title": "La mia nuovissima chat",
}
```



```

    "created_at": "2026-04-26T14:19:48.574747+00:00",
    "updated_at": "2026-04-26T14:19:48.574747+00:00",
    "messages": []
  }

```

– **Errori:**

- * 400 Bad Request: body mancante o non valido.
- * 500 Internal Server Error: errore interno del server.

• **GET /chats/{chat_id}**

– **Descrizione:** Recupera una chat specifica identificata da `chat_id`, includendo tutti i messaggi associati.

– **Parametri:**

- * `chat_id`: identificativo della chat.

– **Response:**

- * 200 OK: richiesta completata con successo.

* Corpo della risposta:

```

{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
  "title": "La mia chat",
  "created_at": "2026-04-22T15:22:27.615449+00:00",
  "updated_at": "2026-04-25T17:26:12.748959+00:00",
  "messages": [
    {
      "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
      "message_id": "01MSG001",
      "text": "Ciao, come ti chiami?",
      "sender": "user",
      "created_at": "2026-04-25T17:25:00.000000+00:00"
    },
    {
      "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
      "message_id": "01MSG002",
      "text": "Sono Chatbot, come posso aiutarti?",
      "sender": "ai",
      "created_at": "2026-04-25T17:25:01.000000+00:00"
    }
  ]
}

```



```
}
```

– **Errori:**

- * 404 Not Found: chat non trovata.
- * 400 Bad Request: parametri non validi.
- * 500 Internal Server Error: errore interno del server.

• **PUT /chats/{chat_id}**

– **Descrizione:** Aggiorna le informazioni di una chat esistente identificata da `chat_id`.

– **Parametri:**

- * `chat_id`: identificativo della chat.
- * `title`: nuovo titolo della chat.
- * `user_id`: identificativo dell'Utente.

– **Response:**

- * 200 OK: aggiornamento completato con successo.
- * Corpo della risposta:

```
{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
  "title": "Nuovo titolo aggiornato",
  "created_at": "2026-04-22T15:22:27.615449+00:00",
  "updated_at": "2026-04-26T14:26:38.684047+00:00",
  "messages": [...]
}
```

– **Errori:**

- * 400 Bad Request: body non valido o parametri mancanti.
- * 404 Not Found: chat non trovata.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /chats/{chat_id}**

– **Descrizione:** Elimina una chat esistente identificata da `chat_id` insieme ai relativi messaggi.

– **Parametri:**

- * `chat_id`: identificativo della chat.

– **Response:**



- * **204 No Content:** eliminazione completata con successo.
- * Corpo della risposta: vuoto.
- **Errori:**
 - * **404 Not Found:** chat non trovata.
 - * **400 Bad Request:** parametri non validi.
 - * **500 Internal Server Error:** errore interno del server.
- **POST /chats/{chat_id}/messages**
 - **Descrizione:** Inserisce un nuovo messaggio all'interno di una chat esistente e attiva la generazione della risposta da parte del modello LLM, secondo la modalità specificata.
 - **Parametri:**
 - * **chat_id:** identificativo della chat.
 - * **message:** contenuto del messaggio dell'Utente.
 - * **response_mode** (body, opzionale): modalità di generazione della risposta del chatbot: "mirror" o "detective".
 - **Response:**
 - * **200 OK:** messaggio elaborato con successo e risposta generata.
 - * Corpo della risposta:

```
{  
  "response": "questa è la risposta del chatbot",  
  "input_message_id": "01KQ536RE4QX5XSCZVC42XQJ49",  
  "response_message_id": "01KQ536RGM0Q86WPV8TNGZ5J1V"  
}
```
 - **Errori:**
 - * **400 Bad Request:** messaggio mancante o non valido.
 - * **404 Not Found:** chat non trovata.
 - * **500 Internal Server Error:** errore interno del servizio LLM o del Backend.

3.6.4.3 Architettura logica del chatbot

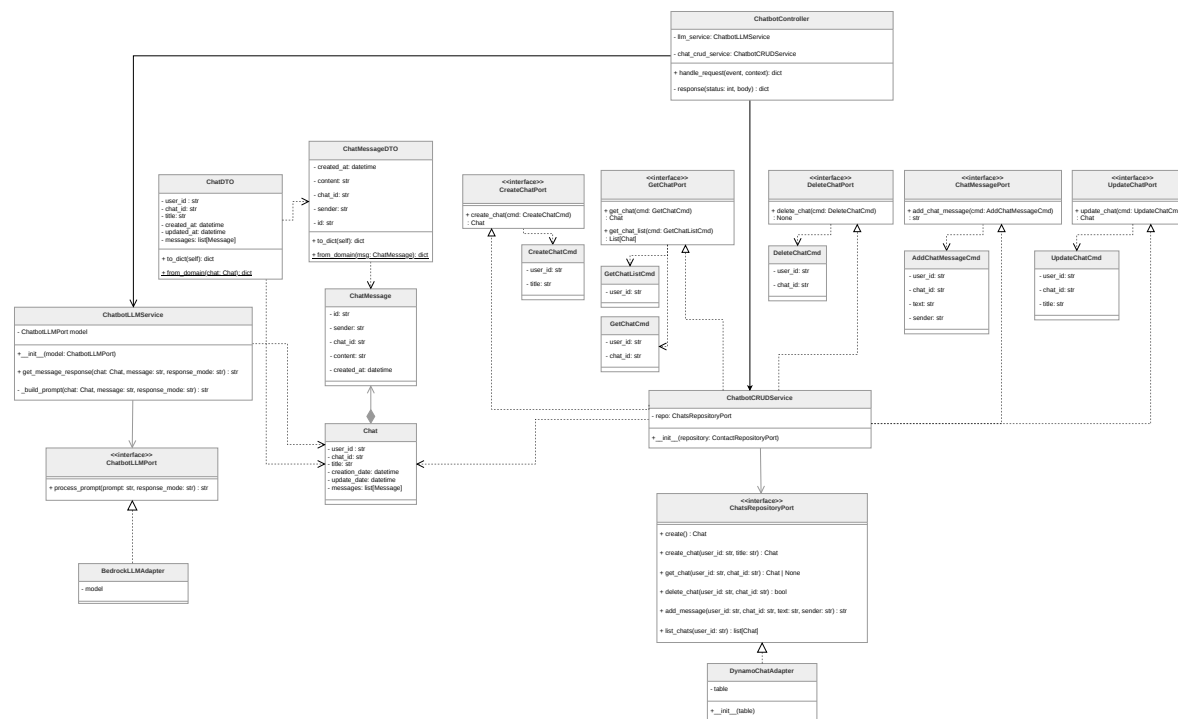


Figura 158: Componenti del microservizio Chatbot



3.6.4.3.1 Chat

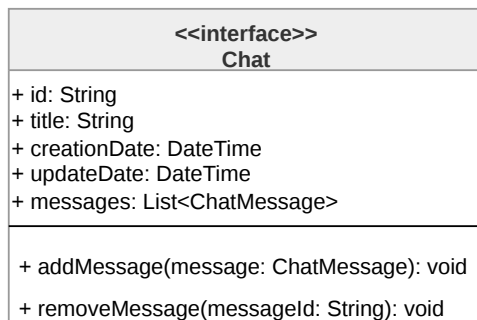


Figura 159: Diagramma della classe **Chat**

La classe **Chat** rappresenta il modello di dominio di una conversazione tra Utente e chatbot. Contiene i metadati identificativi (`user_id`, `chat_id`), il titolo della conversazione, le date di creazione e aggiornamento e l'elenco completo dei messaggi scambiati. Questa classe costituisce l'entità centrale del dominio applicativo e viene utilizzata sia dai servizi di persistenza che da quelli di elaborazione tramite LLM.

3.6.4.3.2 ChatMessage

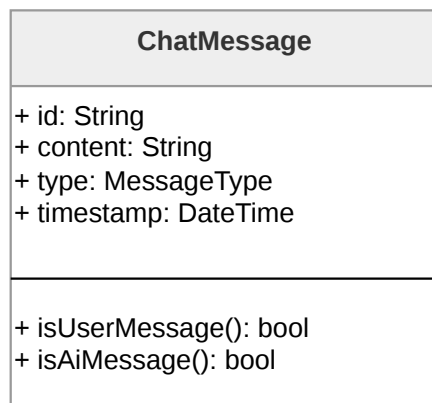


Figura 160: Diagramma della classe **ChatMessage**

ChatMessage modella un singolo messaggio all'interno di una conversazione. Include l'identificativo univoco del messaggio (`id`), il riferimento alla chat di appartenenza (`chat_id`), il contenuto testuale (`content`), il mittente (`sender`, che può assumere il valore `"user"` o `"ai"`) e la data di creazione. Questa classe permette di mantenere traccia completa dello storico conversazionale necessario per il funzionamento del chatbot.



3.6.4.3.3 ChatbotLLMPort



Figura 161: Diagramma dell'interfaccia **ChatbotLLMPort**

L'interfaccia **ChatbotLLMPort** definisce il contratto per gli adapter che si interfacciano con LLM. Espone il metodo **process_prompt**, che accetta un prompt testuale e una modalità di risposta, restituendo la stringa generata dal modello. Questa astrazione permette di disaccoppiare la logica applicativa dall'implementazione specifica del provider LLM utilizzato.

3.6.4.3.4 BedrockLLMAdapter

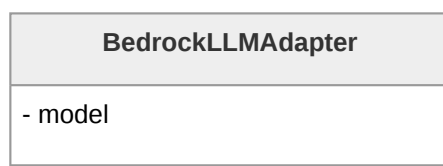


Figura 162: Diagramma della classe **BedrockLLMAdapter**

BedrockLLMAdapter implementa l'interfaccia **ChatbotLLMPort** fornendo l'integrazione concreta con Amazon Bedrock. Questa classe incapsula la logica di comunicazione con il servizio Cloud, traducendo le richieste dell'applicazione in chiamate API specifiche per il modello di linguaggio selezionato.

3.6.4.3.5 ChatbotLLMService

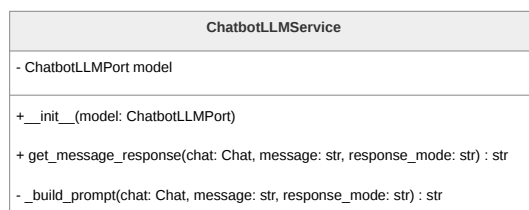


Figura 163: Diagramma della classe **ChatbotLLMService**

Rappresenta la domain logic, coordina la generazione delle risposte del chatbot interagendo con un'implementazione di **ChatbotLLMPort**. Il metodo **get_message_response** orchestra l'intero processo: recupera i messaggi rilevanti dalla cronologia della conversazione



tramite l'algoritmo BM25, costruisce il prompt contestualizzato attraverso `_build_prompt` combinando il messaggio Utente con il contesto recuperato e il prompt di sistema appropriato alla modalità selezionata, e infine delega al modello la generazione della risposta. Questa classe è quindi responsabile di implementare i diversi meccanismi di risposta sulla base di `response_mode`, realizzando così le funzionalità di risposta "Detective delle relazioni" e "Specchio intelligente".

3.6.4.3.6 ChatsRepositoryPort

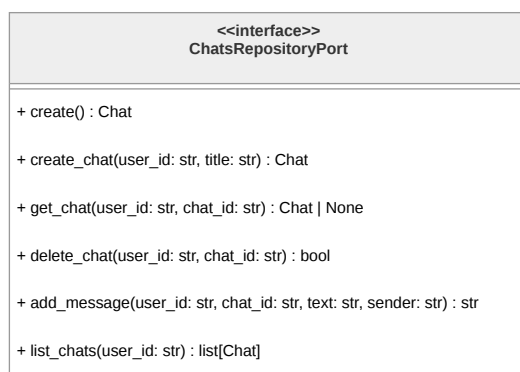


Figura 164: Diagramma dell'interfaccia `ChatsRepositoryPort`

Definisce il contratto per la persistenza delle conversazioni. Espone metodi per tutte le operazioni CRUD: creazione di nuove chat (`create_chat`), recupero singolo (`get_chat`) e multiplo (`list_chats`), eliminazione (`delete_chat`) e aggiunta di messaggi (`add_message`). Questa astrazione garantisce l'indipendenza del dominio dall'implementazione specifica del layer di persistenza.

3.6.4.3.7 DynamoChatAdapter

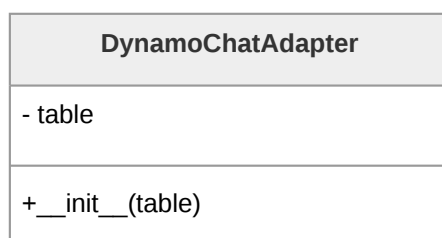


Figura 165: Diagramma della classe `DynamoChatAdapter`

`DynamoChatAdapter` implementa `ChatsRepositoryPort` fornendo la persistenza concreta su DynamoDB. Questa classe gestisce l'interazione con due tabelle: `chats_mvp` per i



metadati delle conversazioni e `chats_messages_mvp` per i messaggi. Implementa la logica di traduzione tra gli oggetti di dominio e le strutture dati di DynamoDB, inclusa la generazione di identificativi ULID e la gestione delle operazioni batch per l'eliminazione dei messaggi. L'adapter incapsula completamente i dettagli tecnici di AWS, esponendo al dominio un'interfaccia pulita e tecnologicamente agnostica.

3.6.4.3.8 CreateChatPort

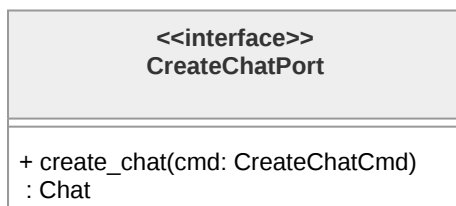


Figura 166: Diagramma dell'interfaccia `CreateChatPort`

L'interfaccia `CreateChatPort` definisce la porta applicativa per la creazione di nuove conversazioni. Espone il metodo `create_chat`, che accetta un comando `CreateChatCmd` e restituisce l'oggetto `Chat` creato. Questa interfaccia rappresenta uno degli use case primari del sistema, separando l'intento dell'operazione dalla sua implementazione.

3.6.4.3.9 GetChatPort

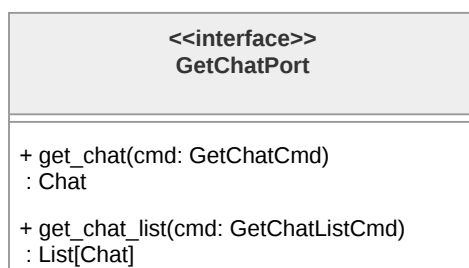


Figura 167: Diagramma dell'interfaccia `GetChatPort`

`GetChatPort` definisce le operazioni di lettura delle conversazioni. Include il metodo `get_chat` per il recupero di una singola chat dato il suo identificativo e `get_chat_list` per ottenere l'elenco di tutte le conversazioni appartenenti a un Utente. Entrambi i metodi operano attraverso comandi dedicati, mantenendo la separazione tra richiesta ed esecuzione.



3.6.4.3.10 UpdateChatPort

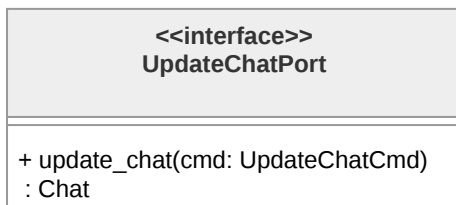


Figura 168: Diagramma dell'interfaccia **UpdateChatPort**

L'interfaccia **UpdateChatPort** gestisce le operazioni di modifica delle conversazioni esistenti. Il metodo `update_chat` accetta un **UpdateChatCmd** e restituisce la chat aggiornata, permettendo la modifica di metadati quali il titolo della conversazione.

3.6.4.3.11 DeleteChatPort

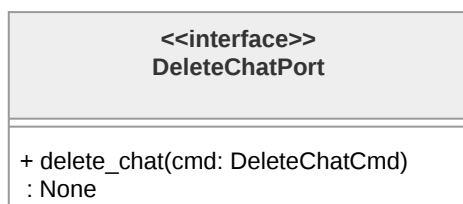


Figura 169: Diagramma dell'interfaccia **DeleteChatPort**

DeleteChatPort definisce l'operazione di eliminazione di una conversazione. Il metodo `delete_chat` riceve un **DeleteChatCmd** contenente gli identificativi necessari e procede con la rimozione della chat e di tutti i messaggi associati dalle rispettive tabelle DynamoDB.

3.6.4.3.12 ChatMessagePort

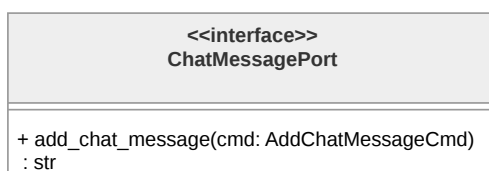


Figura 170: Diagramma dell'interfaccia **ChatMessagePort**

L'interfaccia **ChatMessagePort** astrae l'operazione di aggiunta di messaggi a una conversazione esistente. Il metodo `add_chat_message` accetta un **AddChatMessageCmd** e resti-



tuisce l'identificativo del messaggio appena creato, permettendo al chiamante di tracciare il risultato dell'operazione.

3.6.4.3.13 ChatbotCRUDService

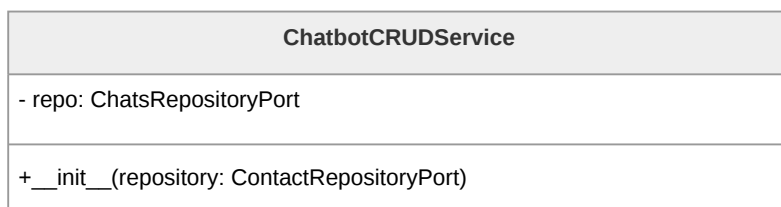


Figura 171: Diagramma della classe **ChatbotCRUDService**

Costituisce la domain logic per le operazione relative alle operazioni CRUD sulle conversazioni, implementando le classi **CreateChatPort**, **GetChatPort**, **UpdateChatPort**, **DeleteChatPort** e **ChatMessagePort**. Questo servizio funge da facade unificato per tutte le operazioni di gestione delle chat, delegando al repository la persistenza effettiva dei dati. Mantiene un riferimento a **ChatsRepositoryPort**, rispettando il principio di inversione delle dipendenze dell'Architettura esagonale.

3.6.4.3.14 CreateChatCmd

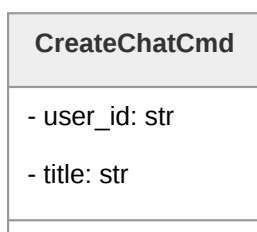


Figura 172: Diagramma della classe **CreateChatCmd**

CreateChatCmd incapsula i parametri necessari per la creazione di una nuova conversazione: l'identificativo dell'Utente (**user_id**) e il titolo della chat (**title**).



3.6.4.3.15 GetChatCmd

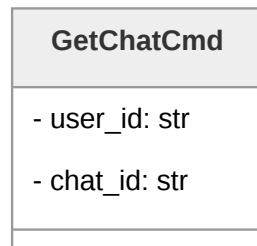


Figura 173: Diagramma della classe **GetChatCmd**

GetChatCmd trasporta i parametri per il recupero di una specifica conversazione: l'identificativo dell'Utente e quello della chat.

3.6.4.3.16 GetChatListCmd



Figura 174: Diagramma della classe **GetChatListCmd**

GetChatListCmd contiene l'identificativo dell'Utente per il quale si richiede l'elenco completo delle conversazioni. Questo comando rappresenta l'operazione di recupero multiplo, fondamentale per la visualizzazione dello storico delle chat nell'interfaccia Utente.

3.6.4.3.17 UpdateChatCmd

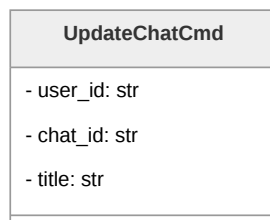


Figura 175: Diagramma della classe **UpdateChatCmd**

UpdateChatCmd incapsula i parametri per la modifica di una conversazione esistente: gli identificativi di Utente e chat, e il nuovo titolo da assegnare.



3.6.4.3.18 DeleteChatCmd

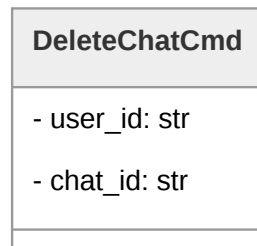


Figura 176: Diagramma della classe **DeleteChatCmd**

DeleteChatCmd trasporta gli identificativi necessari per l'eliminazione di una conversazione e di tutti i messaggi associati.

3.6.4.3.19 AddChatMessageCmd

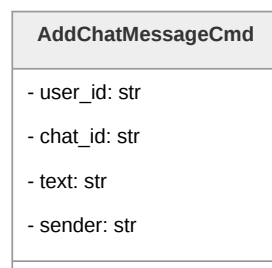


Figura 177: Diagramma della classe **AddChatMessageCmd**

AddChatMessageCmd contiene tutti i parametri necessari per aggiungere un messaggio a una conversazione: identificativi di Utente e chat, contenuto testuale (**text**) e tipologia del mittente (**sender**). Questo comando viene utilizzato sia per memorizzare i messaggi inviati dall'Utente che le risposte generate dal chatbot, mantenendo la coerenza dello storico conversazionale.



3.6.4.3.20 ChatDTO

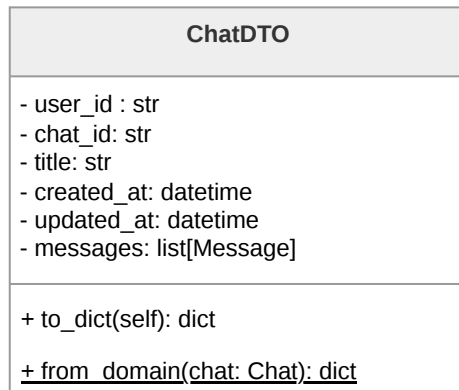


Figura 178: Diagramma della classe **ChatDTO**

ChatDTO è il Data Transfer Object utilizzato per la serializzazione di una chat in un dizionario utilizzato per la risposta della lambda. Contiene tutti i campi dell'entità **Chat** di dominio e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

3.6.4.3.21 ChatMessageDTO

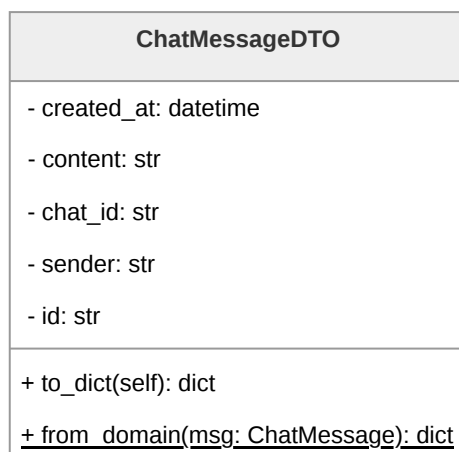


Figura 179: Diagramma della classe **ChatMessageDTO**



ChatMessageDTO rappresenta il Data Transfer Object per i singoli messaggi. Analogamente a **ChatDTO**, fornisce metodi per la conversione bidirezionale tra oggetti di dominio e strutture dati serializzabili.

3.6.4.3.22 ChatBotController

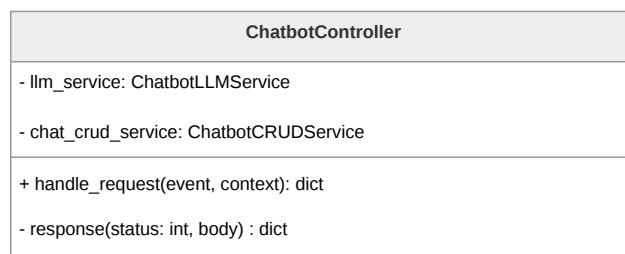


Figura 180: Diagramma della classe **ChatBotController**

Rappresenta l'inbound adapter del microservizio. Esegue il parsing delle richieste inviate da API Gateway in oggetti di dominio. il punto di ingresso della funzione Lambda, li instrada ai gestori appropriati, coordina l'interazione tra **ChatbotCRUDService** e **ChatbotLLMService**, e costruisce le risposte HTTP conformi alle specifiche dell'API.

3.6.5 Contatti fidati, invio SOS e Dead man's switch

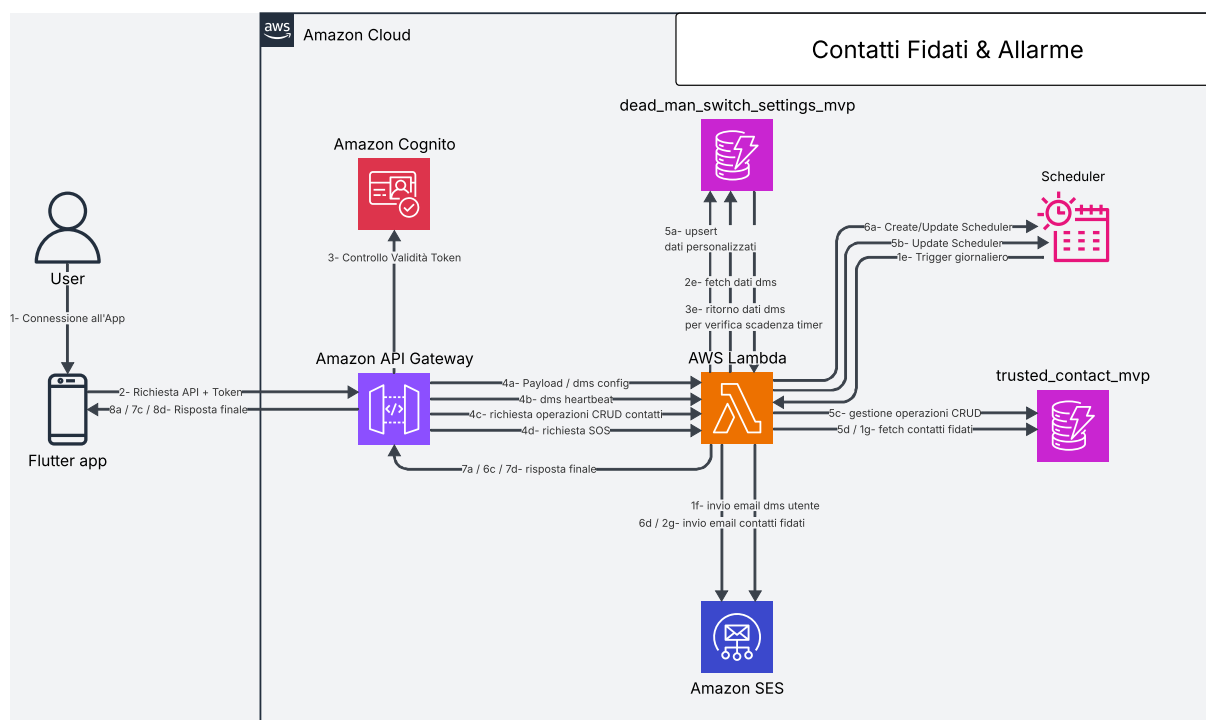


Figura 181: Architettura del sistema di contatti fidati, SOS e Dead man's switch



3.6.5.1 Architettura e Flusso di Elaborazione: contatti fidati, SOS e Dead man's switch

La sezione relativa ai contatti fidati, all'invio di messaggi di emergenza e al meccanismo di Dead man's switch è progettata per garantire la gestione automatizzata dello stato di attività dell'Utente e la possibilità di attivare procedure di emergenza sia manualmente che in modo automatico.

API Gateway gestisce l'esposizione delle API e verifica, per ogni richiesta, che l'Utente sia correttamente autenticato all'applicazione. Le richieste valide vengono inoltrate a una funzione Lambda centrale, responsabile della gestione della logica applicativa relativa ai contatti fidati, alla configurazione del Dead man's switch e all'orchestrazione dell'invio delle email tramite Amazon SES.

I dati sono memorizzati su due database DynamoDB distinti:

- **dead_man_switch_settings_mvp**: contiene la configurazione del Dead man's switch, includendo il titolo e il corpo dell'email di inattività, il primo e secondo timer di inattività, l'ultimo accesso dell'Utente e lo stato corrente del sistema (attivo, prima inattività, seconda inattività).
- **trusted_contact_mvp**: memorizza le informazioni relative ai contatti fidati associati a ciascun Utente, inclusi nome, email e numero di telefono.

Il sistema implementa inoltre un meccanismo di monitoraggio automatico basato su AWS EventBridge. Uno scheduler giornaliero attiva la Lambda centrale che verifica lo stato di attività degli utenti e determina eventuali condizioni di inattività prolungata.

Il flusso di elaborazione del Dead man's switch è il seguente:

- **Trigger programmato**: EventBridge attiva quotidianamente la Lambda, che avvia il controllo dello stato degli utenti.
- **Recupero configurazioni**: la Lambda effettua il fetch dei dati presenti in `dead_man_switch_settings_mvp`, ottenendo i timer di inattività, l'ultimo accesso registrato e lo stato corrente del Dead man's switch.
- **Valutazione dello stato di inattività**: la funzione confronta il tempo corrente con l'ultimo accesso dell'Utente. Se viene superata la prima soglia di inattività, il sistema entra nello stato di "prima inattività". Al raggiungimento della seconda soglia di inattività, viene attivato lo stato di "seconda inattività".
- **Invio email di prima inattività**: al superamento della prima soglia, la Lambda invia tramite Amazon SES un'email all'Utente utilizzando il titolo e il corpo configurati, notificando lo stato di inattività.



- **Attivazione SOS (seconda inattività):** al superamento della seconda soglia, la Lambda recupera l'elenco dei contatti fidati da `trusted_contact_mvp` e invia, tramite SES, un'email di SOS predefinita a ciascun contatto.
- **Aggiornamento stato:** lo stato del sistema viene aggiornato in DynamoDB per evitare invii ripetuti non necessari.

Oltre al flusso automatico, il sistema supporta l'invio manuale di SOS e la gestione in tempo reale dello stato di attività dell'Utente tramite heartbeat.

Il flusso di heartbeat è il seguente:

- **Invio heartbeat:** All'avvio dell'app, il client invia una richiesta all'API `POST /heartbeat` per notificare l'utilizzo attivo dell'applicazione.
- **Aggiornamento stato Backend:** la Lambda aggiorna il campo "ultimo accesso" nella tabella `dead_man_switch_settings_mvp` e, se necessario, resetta lo stato di inattività.

L'invio SOS **manuale** segue invece il seguente flusso:

- **Richiesta di emergenza:** il client invia una richiesta `HTTPS all'API POST /alert`.
- **Recupero contatti:** la Lambda recupera tutti i contatti fidati associati all'Utente da `trusted_contact_mvp`.
- **Invio email SES:** la funzione Lambda invia immediatamente un'email di emergenza a tutti i contatti fidati tramite Amazon SES.

La gestione dei contatti fidati è completamente CRUD e permette all'Utente di mantenere aggiornato il proprio elenco di destinatari di emergenza.

3.6.5.2 API associate

- **POST `/dms_settings`**
 - **Descrizione:** Crea la configurazione del Dead Man's Switch per l'Utente autenticato.
 - **Parametri:**
 - * `user_id`: identificativo dell'Utente;
 - * `user_email`: email dell'Utente;
 - * `user_name`: nome dell'Utente.
 - **Response:**
 - * 200 OK: configurazione creata con successo.
 - * Corpo della risposta:



```
{
  "user_id": "user1",
  "is_active": true,
  "first_timer": 60,
  "second_timer": 120,
  "email_subject": "Messaggio di emergenza",
  "email_body": "Corpo del messaggio"
}
```

– **Errori:**

- * 400 Bad Request: campi non validi.
- * 500 Internal Server Error: errore interno del server.

● **GET /dms_settings**

– **Descrizione:** Recupera la configurazione del Dead Man's Switch dell'Utente.

– **Parametri:**

- * `user_id`: identificativo dell'Utente.

– **Response:**

- * 200 OK: configurazione recuperata con successo.
- * Corpo della risposta:

```
{
  "user_id": "user1",
  "is_active": true,
  "first_timer": 60,
  "second_timer": 120,
  "email_subject": "Messaggio di emergenza",
  "email_body": "Corpo del messaggio"
}
```

– **Errori:**

- * 500 Internal Server Error: errore interno del server.

● **PUT /dms_settings**

– **Descrizione:** Aggiorna le impostazioni del Dead Man's Switch.

– **Parametri:**

- * `user_id`: identificativo dell'Utente;
- * `is_active`: valore booleano
- * `first_timer`: primo timer del dead man's switch;
- * `second_timer`: secondo timer del dead man's switch;



- * **email_subject**: oggetto della mail per la notifica del dead man's switch all'Utente;
- * **email_body**: corpo della mail per la notifica del dead man's switch all'Utente.
- **Response**:
 - * 204 OK: aggiornamento della configurazione avvenuto con successo.
 - * Corpo della risposta:

```
{  
    "user_id": "user1",  
    "is_active": true,  
    "first_timer": 60,  
    "second_timer": 120,  
    "email_subject": "Messaggio di emergenza",  
    "email_body": "Corpo del messaggio"  
}
```
- **Errori**:
 - * 500 Internal Server Error: errore interno del server.
- **PUT /dms_settings/heartbeat**
 - **Descrizione**: registra il segnale di attività dell'Utente aggiornando l'ultimo accesso.
 - **Parametri**:
 - * **user_id**: identificativo dell'Utente.
 - **Response**:
 - * 200: dati aggiornati con successo.
 - * Corpo della risposta:

```
{  
    "message": "Heartbeat processed with success"  
}
```
- **POST /trusted_contact**
 - **Descrizione**: Crea un nuovo contatto fidato associato all'Utente autenticato.
 - **Parametri**:
 - * **user_id**: identificativo dell'Utente;
 - * **contact_name**: nome del contatto fidato;
 - * **contact_email**: email del contatto fidato;



- * `contact_phone_number`: numero di telefono del contatto fidato.

– **Response:**

- * 200 OK: creazione contatti completata con successo.

- * Corpo della risposta:

```
{
  "user_id": "user1",
  "contact_id": "contact123",
  "contact_name": "Mario Rossi",
  "contact_email": "mario@gmail.com",
  "contact_phone_number": "333 1111 111"
}
```

– **Errori:**

- * 400 Bad Request: errore nella Validazione dei campi.
- * 500 Internal Server Error: errore interno del server.

• **GET /trusted_contact**

- **Descrizione:** Recupera tutti i contatti fidati associati all'Utente autenticato.

– **Parametri:**

- * `user_id`: identificativo dell'Utente.

– **Response:**

- * 200 OK: contatti recuperati con successo.

- * Corpo della risposta:

```
{
  "trusted_contacts": [
    {
      "user_id": "user1",
      "contact_id": "contact1",
      "contact_name": "Mario Rossi",
      "contact_email": "mario@gmail.com",
      "contact_phone_number": "111 1111 111"
    },
    {
      "user_id": "user1",
      "contact_id": "contact2",
      "contact_name": "Luigi Verdi",
      "contact_email": "luigi@gmail.com",
      "contact_phone_number": "333 2222 222"
    }
  ]
}
```



```
    ]  
  }  
}
```

- **GET /trusted_contact/{contact_id}**

- **Descrizione:** Recupera un contatto specifico associato all'Utente autenticato.

- **Parametri:**

- * **user_id:** identificativo dell'Utente.

- **Response:**

- * 200 OK: contatto recuperato con successo.

- * Corpo della risposta:

```
{  
  "user_id": "user1",  
  "contact_id": "contact1",  
  "contact_name": "Mario Rossi",  
  "contact_email": "mario@gmail.com",  
  "contact_phone_number": "111 1111 111"  
}
```

- **Errori:**

- * 500 Internal Server Error: errore interno del server.

- **PUT /trusted_contact**

- **Descrizione:** Aggiorna un contatto fidato esistente associato all'Utente autenticato.

- **Parametri:**

- * **user_id:** identificativo dell'Utente;

- * **contact_id:** identificativo del contatto fidato;

- * **contact_name:** nome del contatto fidato;

- * **contact_email:** email del contatto fidato;

- * **contact_phone_number:** numero di telefono del contatto fidato.

- **Response:**

- * 200 OK: contatto aggiornato con successo.

- * Corpo della risposta:

```
{  
  "user_id": "user1",  
  "contact_id": "contact1",  
  "contact_name": "Mario Rossi",  
}
```



```
    "contact_email": "mario@gmail.com",  
    "contact_phone_number": "111 1111 111"  
}
```

– **Errori:**

- * 400 Bad Request: errore nella Validazione dei campi.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /trusted_contact/{contact_id}**

– **Descrizione:** Elimina un contatto fidato esistente associato all'Utente autenticato.

– **Parametri:**

- * user_id: identificativo dell'Utente.

– **Response:**

- * 200 OK: eliminazione contatto avvenuta con successo.

```
{  
    "message": "Contact contact1 deleted"  
}
```

• **PUT /alert**

– **Descrizione:** Invia un alert ai contatti fidati con posizione opzionale.

– **Parametri:**

- * user_id: identificativo dell'Utente;
- * latitude (opzionale);
- * longitude (opzionale).

– **Response:**

- * 200 OK: invio dell'alert avvenuto con successo.

```
{  
    "message": "Alert send processed with success"  
}
```

– **Errori:**

- * 400 Bad Request: Verifica presenza contatti fidati fallita.
- * 500 Internal Server Error: errore interno del server.

3.6.5.3 Architettura logica: Modulo contatti fidati, SOS e Dead man's switch

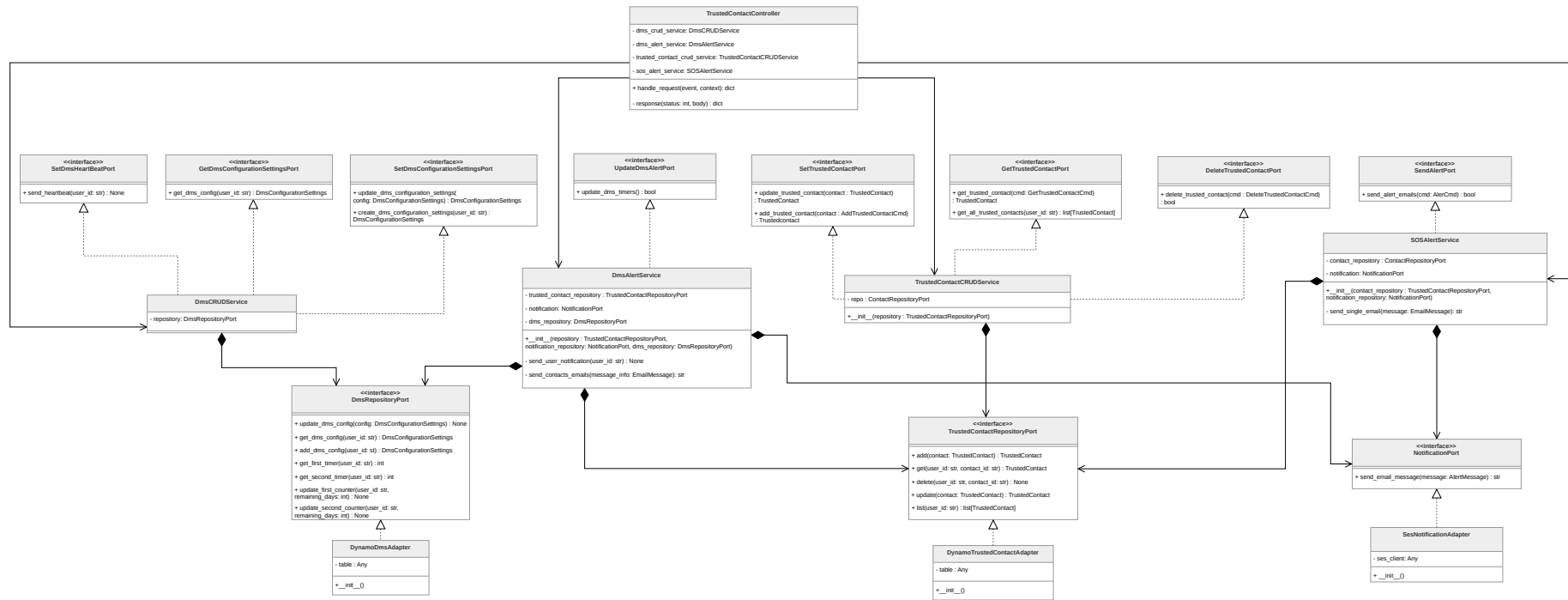


Figura 182: Componenti del microservizio contatti fidati, SOS e Dead man's switch





3.6.5.3.1 TrustedContact

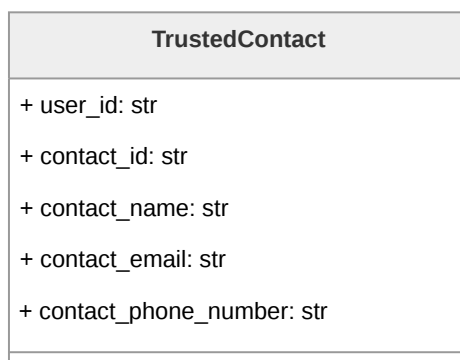


Figura 183: Diagramma della classe **TrustedContact**

Rappresenta un contatto fidato associato a un Utente.

Descrizione attributi:

- **user_id** identifica l'Utente proprietario del contatto;
- **contact_id** identifica univocamente il contatto;
- **contact_name**, **contact_email** e **contact_phone_number** ne rappresentano i dati anagrafici.

3.6.5.3.2 TrustedContactDTO

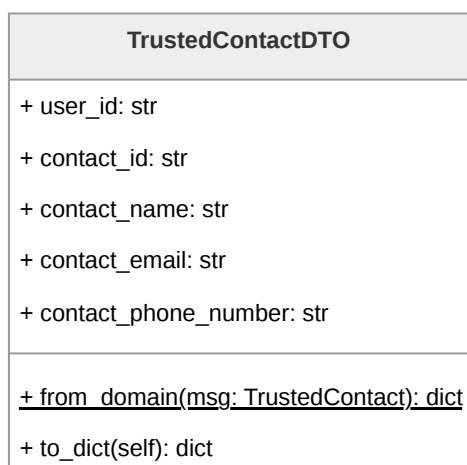


Figura 184: Diagramma della classe **TrustedContactDTO**



Rappresenta il Data Transfer Object utilizzato per la serializzazione di un contatto fidato in un dizionario utilizzato per la risposta della lambda. Contiene tutti i campi dell'entità di dominio **TrustedContact** e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

3.6.5.3.3 AddTrustedContactCmd



Figura 185: Diagramma della classe **AddTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari all'aggiunta di un contatto fidato.

3.6.5.3.4 GetTrustedContactCmd

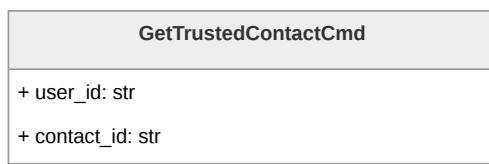


Figura 186: Diagramma della classe **GetTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari per il fetch di un contatto fidato.

3.6.5.3.5 DeleteTrustedContactCmd

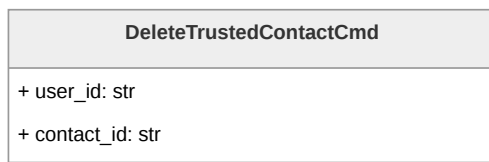


Figura 187: Diagramma della classe **DeleteTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari per la cancellazione di un contatto fidato.



3.6.5.3.6 DmsConfigurationSettings

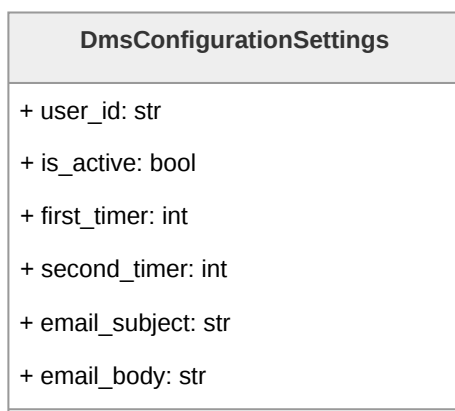


Figura 188: Diagramma della classe **DmsConfigurationSettings**

Rappresenta la configurazione del Dead Man's Switch associata ad un Utente.

Descrizione attributi:

- **user_id** identifica l'Utente a cui è associata la configurazione del dead man's switch;
- **is_active** indica se il dead man's switch è attivo;
- **first_timer** e **second_timer** definiscono la durata in giorni dei due timer;
- **email_body** contiene il corpo della mail riferita al dead man's switch;
- **email_subject** contiene l'oggetto della mail riferita al dead man's switch.

3.6.5.3.7 DmsConfigurationSettingsDTO

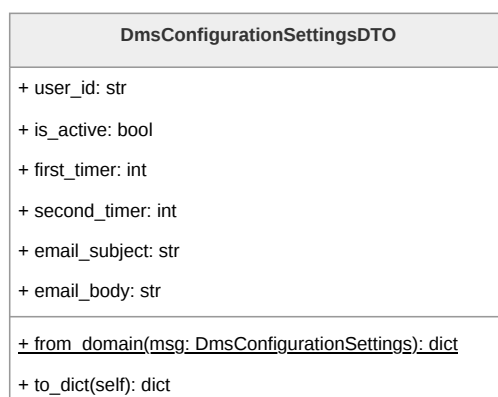


Figura 189: Diagramma della classe **DmsConfigurationSettingsDTO**



Data Transfer Object utilizzato per la serializzazione delle impostazioni di un Utente in un dizionario utilizzato per la risposta della lambda. . Contiene tutti i campi dell'entità di dominio **DmsConfigurationSettings** e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

3.6.5.3.8 AlertCmd

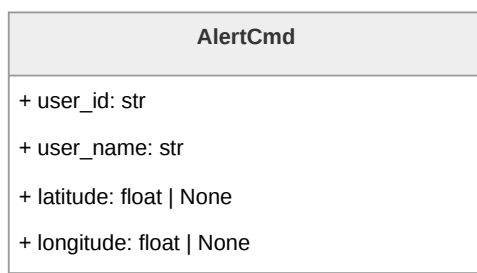


Figura 190: Diagramma della classe **AlertCmd**

Command Object utilizzato dal controller per trasportare i dati necessari all'invio dell'alert SOS.

Descrizione attributi:

- **user_id**, identificativo dell'Utente;
- **user_name**, nome dell'Utente che ha azionato l'alert;
- **latitude** e **longitude** indicano le coordinate geografiche in cui si trova l'Utente al momento dell'invio dell'alert.

3.6.5.3.9 EmailMessage

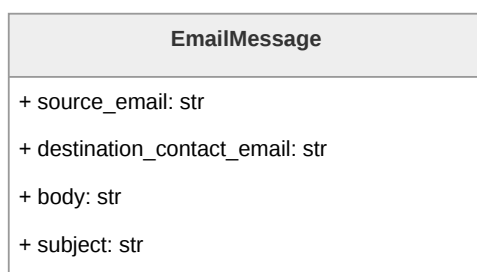


Figura 191: Diagramma della classe **EmailMessage**

Rappresenta la struttura delle email inviabili dal sistema.

Descrizione attributi:



- **source_email**, indirizzo con cui viene mandata l'email di alert;
- **destination_contact_email**, indirizzo email a cui viene inviato l'alert;
- **body**, corpo della mail di alert,
- **subject**, oggetto della mail di alert.

3.6.5.3.10 TrustedContactController

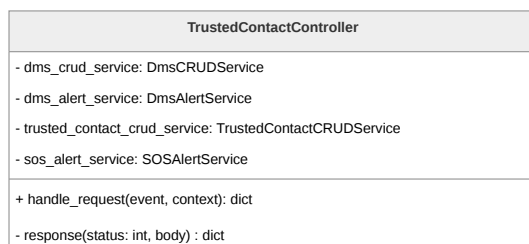


Figura 192: Diagramma della classe **TrustedContactController**

Rappresenta l'inbound adapter del microservizio. Riceve la richiesta grezza di API Gateway e la instrada verso la port primaria appropriata in base alla route.

Descrizione metodi:

- **handle_request(event, context)**, punto di ingresso del controller, legge la **routeKey** dall'evento e chiama la port primaria corrispondente;
- **response(status, body)**, metodo privato di utilità che costruisce la risposta nel formato atteso da API Gateway.

3.6.5.3.11 SetDmsHeartBeatPort

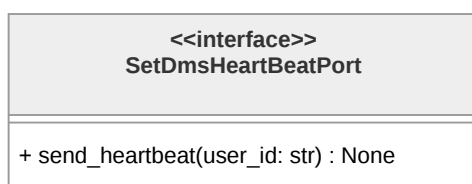


Figura 193: Diagramma della classe **SetDmsHeartBeatPort**

Espone **send_heartbeat(user_id: str)** per il reset dei timer del Dead Man's Switch al rientro dell'Utente nell'applicazione.



3.6.5.3.12 GetDmsConfigurationSettingsPort

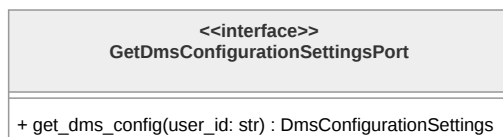


Figura 194: Diagramma della classe **GetDmsConfigurationSettingsPort**

Espone **get_dms_config(user_id: str)** per il recupero della configurazione del Dead Man's Switch.

3.6.5.3.13 SetDmsConfigurationSettingsPort

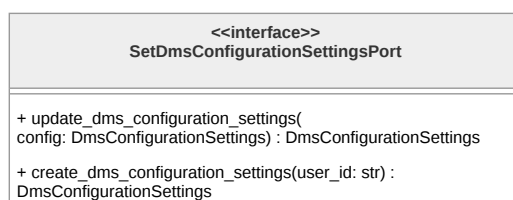


Figura 195: Diagramma della classe **SetDmsConfigurationSettingsPort**

Descrizione metodi:

- **update_dms_configuration_settings(config: DmsConfigurationSettings)**, per l'aggiornamento delle impostazioni del dead man's switch;
- **create_dms_configuration_settings(user_id: str)**, per la creazione dei valori di default per le impostazioni del dead man's switch.

3.6.5.3.14 UpdateDmsAlertPort

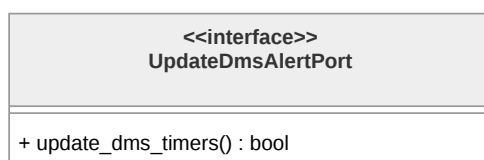


Figura 196: Diagramma della classe **UpdateDmsAlertPort**

Espone **update_dms_timers()** per l'aggiornamento dei timer lato Backend.



3.6.5.3.15 SetTrustedContactPort

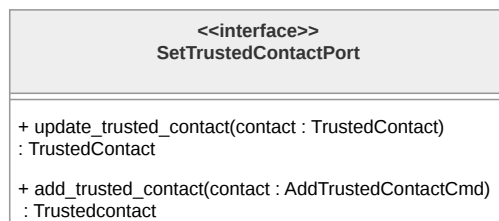


Figura 197: Diagramma della classe **SetTrustedContactPort**

Descrizione dei metodi:

- **add_trusted_contact(contact: AddTrustedContactCmd)** per la creazione di un nuovo contatto;
- **update_trusted_contact(request: TrustedContact)** per la modifica di uno esistente.

3.6.5.3.16 GetTrustedContactPort



Figura 198: Diagramma della classe **GetTrustedContactPort**

Descrizione dei metodi:

- **get_trusted_contact(cmd: GetTrustedContactCmd)** per il recupero di un singolo contatto;
- **get_all_trusted_contacts(user_id: str)** per il recupero della lista completa.

3.6.5.3.17 DeleteTrustedContactPort

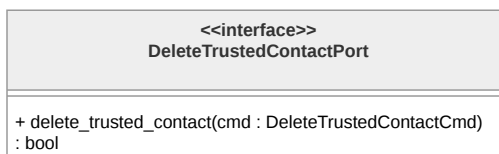


Figura 199: Diagramma della classe **DeleteTrustedContactPort**

Espone **delete_trusted_contact(cmd: DeleteTrustedContactCmd)** per l'eliminazione di un contatto.



3.6.5.3.18 SendAlertPort

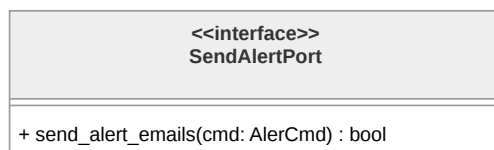


Figura 200: Diagramma della classe **SendAlertPort**

Espone `send_alert_emails(alert_info: AlertCmd)` per l'invio delle email di soccorso ai contatti fidati.

3.6.5.3.19 DmsCRUDService

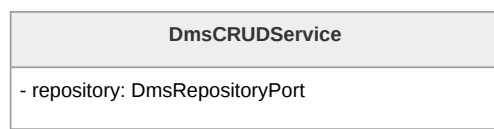


Figura 201: Diagramma della classe **DmsCRUDService**

Rappresenta la domain logic del microservizio, è responsabile delle operazioni di lettura e scrittura sui valori delle impostazioni del Dead Man's Switch. Implementa le inbound port che definiscono le operazioni crud possibili sui dati.

3.6.5.3.20 DmsAlertService

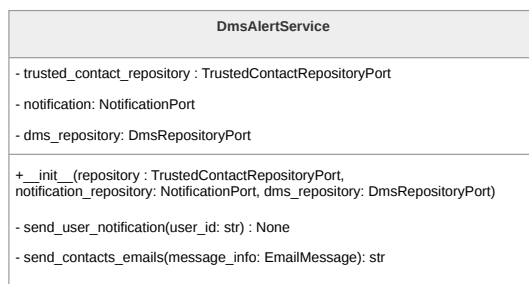


Figura 202: Diagramma della classe **DmsAlertService**

Rappresenta la domain logic del microservizio, è responsabile dell'invio delle notifiche del Dead Man's Switch. Si occupa del recupero dei contatti fidati, la configurazione del DMS e la costruzione dei messaggi email.

Descrizione metodi:

- `__init__(trusted_contact_repository, notification_repository, dms_repository)`, costruttore che riceve le tre port secondarie tramite dependency injection;



- `send_user_notification(user_id: str)`, metodo privato che recupera la configurazione DMS dell'Utente e costruisce e invia la notifica all'Utente alla scadenza del primo timer;
- `send_contacts_emails(message_info: EmailMessage)`, metodo privato che costruisce e invia le email ai contatti fidati dell'Utente alla scadenza del secondo timer.

3.6.5.3.21 TrustedContactCRUDService

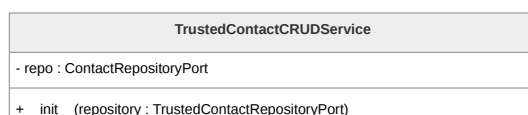


Figura 203: Diagramma della classe `TrustedContactCRUDService`

Rappresenta la domain logic, è responsabile delle operazioni CRUD sui contatti fidati. Descrizione metodi:

- `__init__(repository: TrustedContactRepositoryPort)`, costruttore che riceve la port secondaria tramite dependency injection.

3.6.5.3.22 SOSAlertService

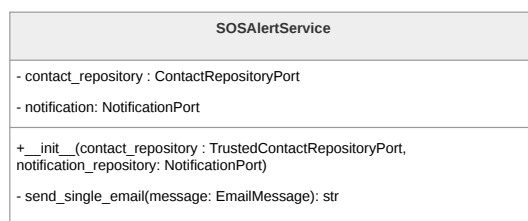


Figura 204: Diagramma della classe `SOSAlertService`

Rappresenta la domain logic, è responsabile dell'invio immediato delle email di soccorso al momento della pressione del pulsante SOS da parte dell'Utente. Implementa la port primaria `SendAlertPort`.

Descrizione metodi:

- `__init__(contact_repository, notification_repository)`, costruttore che riceve le due port secondarie tramite dependency injection;
- `send_single_email(message: EmailMessage)`, metodo privato che costruisce e invia una singola email a partire da un oggetto `EmailMessage`, delegando l'invio alla `NotificationPort`.



3.6.5.3.23 DmsRepositoryPort

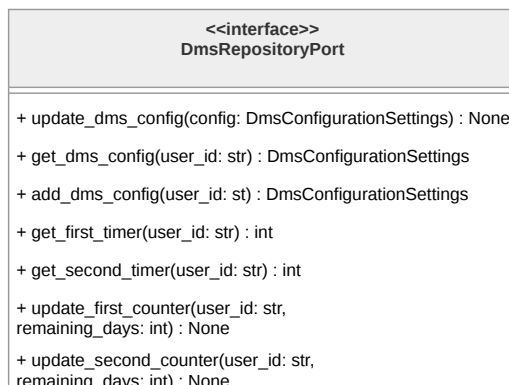


Figura 205: Diagramma della classe **DmsRepositoryPort**

Costituisce l'outbound port, fornisce il contratto per la persistenza delle impostazioni del Dead man's Switch. Espone i metodi utilizzati dalla domain logic. Descrizione metodi:

- **update_dms_config(config: DmsConfigurationSettings)**, aggiorna la configurazione del dead man's switch dell'Utente;
- **get_dms_config(user_id: str)**, recupera la configurazione del dead man's switch associata all'Utente;
- **add_dms_config(user_id: str)**, crea una nuova configurazione del dead man's switch per l'Utente;
- **get_first_timer(user_id: str)**, recupera il valore corrente del primo timer;
- **get_second_timer(user_id: str)**, recupera il valore corrente del secondo timer;
- **update_first_counter(user_id: str, remaining_days: int)**, aggiorna il contatore residuo del primo timer;
- **update_second_counter(user_id: str, remaining_days: int)**, aggiorna il contatore residuo del secondo timer.



3.6.5.3.24 TrustedContactRepositoryPort

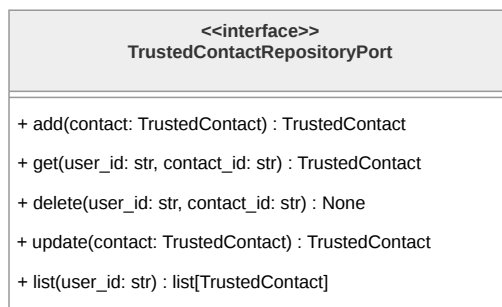


Figura 206: Diagramma della classe **TrustedContactRepositoryPort**

Definisce il contratto per la persistenza dei contatti fidati. Espone i metodi per le operazioni CRUD. Descrizione metodi:

- **add(contact: TrustedContact)**, aggiunge un nuovo contatto fidato e restituisce l'entità creata;
- **get(user_id: str, contact_id: str)**, recupera un singolo contatto fidato tramite le chiavi identificative, restituendo **None** se non trovato;
- **delete(user_id: str, contact_id: str)**, elimina il contatto fidato corrispondente alle chiavi fornite;
- **update_trusted_contact(contact: TrustedContact)**, aggiorna i dati anagrafici di un contatto fidato esistente;
- **list(user_id: str)**, restituisce la lista completa dei contatti fidati associati all'Utente.

3.6.5.3.25 NotificationPort

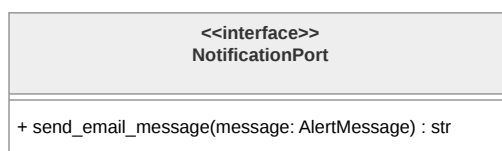


Figura 207: Diagramma della classe **NotificationPort**

È la inbound port del pattern esagonale per l'invio delle notifiche email. Descrizione metodi:

- **send_email_message(message: EmailMessage)**, invia un messaggio email a partire da un oggetto **AlertMessage**.



3.6.5.3.26 DynamoDmsAdapter

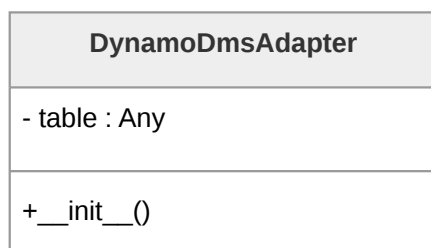


Figura 208: Diagramma della classe **DynamoDmsAdapter**

Contiene la logica specifica per la persistenza della configurazione e dei contatori del Dead Man's Switch su DynamoDB.

3.6.5.3.27 DynamoTrustedContactAdapter

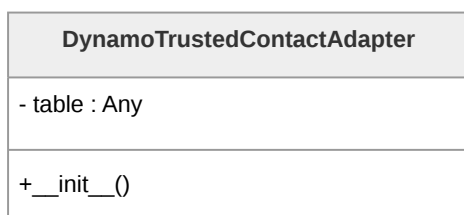


Figura 209: Diagramma della classe **DynamoTrustedContactAdapter**

Contiene tutta la logica specifica per la persistenza dei contatti fidati su DynamoDB.

3.6.5.3.28 SesNotificationAdapter

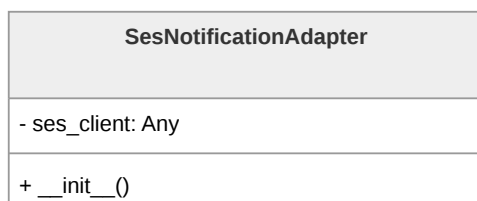


Figura 210: Diagramma della classe **SesNotificationAdapter**

Contiene tutta la logica specifica per l'invio delle email tramite il servizio Amazon SES.



4. Stato dei requisiti funzionali

4.1. Requisiti Funzionali obbligatori

Codice	Descrizione	Stato
RF-OB-1	L'Utente deve poter autenticarsi all'applicazione tramite l'utilizzo di Google Auth	Soddisfatto
RF-OB-2	Il sistema deve attivare automaticamente la funzionalità di Dead Man's Switch a seguito del login dell'Utente	Soddisfatto
RF-OB-3	L'Utente deve poter visualizzare un messaggio di errore esplicativo nel caso in cui l'autenticazione tramite Google Auth fallisca, venga annullata o restituisca parametri non validi	Soddisfatto
RF-OB-5	L'Utente deve poter visualizzare la lista dei contatti fidati	Soddisfatto
RF-OB-6	L'Utente deve poter visualizzare un messaggio di errore esplicativo nel caso in cui il recupero dei dati dal database fallisca	Soddisfatto
RF-OB-7	L'Utente deve poter ricaricare la sezione dei contatti fidati dopo la visualizzazione dell'errore di caricamento dei dati dal database	Soddisfatto
RF-OB-8	Il sistema deve permettere la visualizzazione di un singolo contatto fidato dalla lista dei contatti fidati	Soddisfatto
RF-OB-9	L'Utente deve poter visualizzare la preview delle informazioni (il nome) del contatto fidato che sta visualizzando dalla lista dei contatti fidati	Soddisfatto
RF-OB-10	L'Utente deve poter visualizzare i dettagli di un contatto fidato selezionato dalla lista dei contatti fidati	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-11	L'Utente deve poter visualizzare il nome del contatto fidato selezionato dalla lista dei contatti fidati	Soddisfatto
RF-OB-12	L'Utente deve poter visualizzare il numero di telefono del contatto fidato selezionato dalla lista dei contatti fidati	Soddisfatto
RF-OB-13	L'Utente deve poter visualizzare l'indirizzo email del contatto fidato selezionato dalla lista dei contatti fidati	Soddisfatto
RF-OB-14	L'Utente deve poter aggiungere un nuovo contatto fidato alla lista dei contatti fidati	Soddisfatto
RF-OB-15	L'Utente deve poter inserire il nome del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Soddisfatto
RF-OB-16	L'Utente deve poter inserire il numero di telefono del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Soddisfatto
RF-OB-17	L'Utente deve poter inserire l'indirizzo email del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Soddisfatto
RF-OB-18	Il sistema deve riconoscere e gestire le informazioni di numero di telefono e email già presenti nel sistema	Soddisfatto
RF-OB-19	L'Utente deve poter visualizzare un messaggio di errore esplicativo nel caso in cui le informazioni di numero di telefono e email siano già presenti nel sistema	Soddisfatto
RF-OB-20	L'Utente deve poter reinserire le informazioni di numero di telefono e email dopo la visualizzazione del messaggio di errore esplicativo	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-21	Il sistema deve validare la conformità del numero di telefono inserito o modificato	Soddisfatto
RF-OB-22	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui il numero di telefono inserito o modificato non sia conforme ai criteri prestabiliti	Soddisfatto
RF-OB-23	Il sistema deve validare la conformità dell'indirizzo email inserito o modificato	Soddisfatto
RF-OB-24	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui l'indirizzo email inserito o modificato non sia conforme ai criteri prestabiliti	Soddisfatto
RF-OB-25	L'Utente deve poter modificare i dati di un contatto fidato esistente	Soddisfatto
RF-OB-26	L'Utente deve poter modificare il nome di un contatto fidato esistente	Soddisfatto
RF-OB-27	L'Utente deve poter modificare il numero di telefono di un contatto fidato esistente	Soddisfatto
RF-OB-28	L'Utente deve poter modificare l'indirizzo email di un contatto fidato esistente	Soddisfatto
RF-OB-29	L'Utente deve poter eliminare un contatto fidato	Soddisfatto
RF-OB-30	Il sistema deve richiedere conferma prima dell'eliminazione di un contatto fidato	Soddisfatto
RF-OB-31	L'Utente deve poter confermare, con un'apposita voce, l'eliminazione di un contatto fidato quando il sistema richiede la conferma	Soddisfatto
RF-OB-32	L'Utente deve poter effettuare il logout dopo essersi autenticato all'applicazione	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-34	L'Utente deve poter modificare le informazioni della mail di avviso	Soddisfatto
RF-OB-35	L'Utente deve poter modificare il titolo della mail di avviso	Soddisfatto
RF-OB-36	L'Utente deve poter modificare il corpo della mail di avviso	Soddisfatto
RF-OB-37	L'Utente deve poter visualizzare un'anteprima della mail di avviso	Soddisfatto
RF-OB-38	L'Utente deve poter visualizzare l'anteprima del corpo della mail di avviso	Soddisfatto
RF-OB-39	L'Utente deve poter visualizzare un'anteprima del titolo della mail di avviso	Soddisfatto
RF-OB-40	L'Utente deve poter attivare il Dead Man's Switch	Soddisfatto
RF-OB-41	L'Utente deve poter disattivare la funzionalità di Dead Man's Switch	Soddisfatto
RF-OB-42	L'Utente deve poter impostare una password per il diario criptato	Soddisfatto
RF-OB-43	Il sistema deve validare la conformità della password del diario criptato secondo i criteri pre-stabiliti	Soddisfatto
RF-OB-44	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario criptato non sia conforme ai criteri pre-stabiliti	Soddisfatto
RF-OB-45	L'Utente deve poter reinserire la password del diario criptato dopo la visualizzazione del messaggio di errore esplicativo	Soddisfatto
RF-OB-46	L'Utente deve poter impostare una password per il diario fittizio	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-47	Il sistema deve validare la conformità e l'unicità della password del diario fittizio rispetto a quella del diario criptato	Soddisfatto
RF-OB-48	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio non sia conforme ai criteri prestabiliti	Soddisfatto
RF-OB-49	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio sia uguale a quella del diario criptato	Soddisfatto
RF-OB-50	L'Utente deve poter reinserire la password del diario fittizio dopo la visualizzazione del messaggio di errore esplicativo	Soddisfatto
RF-OB-51	L'Utente deve poter inserire la password per accedere al diario desiderato	Soddisfatto
RF-OB-52	L'Utente deve poter inserire la password per accedere al diario criptato	Soddisfatto
RF-OB-53	L'Utente deve poter inserire la password per accedere al diario fittizio	Soddisfatto
RF-OB-54	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password inserita per l'accesso al diario risulti errata	Soddisfatto
RF-OB-55	L'Utente deve poter reinserire la password di accesso al diario dopo la visualizzazione del messaggio di errore esplicativo	Soddisfatto
RF-OB-57	L'Utente deve poter visualizzare il contenuto del diario criptato	Soddisfatto
RF-OB-58	L'Utente deve poter visualizzare la lista delle note presenti nel diario criptato	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-59	L'Utente deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario criptato	Soddisfatto
RF-OB-60	L'Utente deve poter ricaricare la sezione delle note del diario criptato dopo la visualizzazione dell'errore di caricamento dei dati dal database	Soddisfatto
RF-OB-61	L'Utente deve poter creare una nuova nota vuota all'interno del diario criptato	Soddisfatto
RF-OB-62	L'Utente deve poter eliminare una nota esistente dal diario criptato	Soddisfatto
RF-OB-63	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario criptato	Soddisfatto
RF-OB-64	L'Utente deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario criptato quando il sistema ne richiede la conferma	Soddisfatto
RF-OB-65	L'Utente deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario criptato	Soddisfatto
RF-OB-66	L'Utente deve poter visualizzare il contenuto testuale della nota del diario criptato	Soddisfatto
RF-OB-67	L'Utente deve poter visualizzare il contenuto multimediale della nota del diario criptato	Soddisfatto
RF-OB-68	L'Utente deve poter visualizzare il contenuto audio presente all'interno della nota del diario criptato	Soddisfatto
RF-OB-69	L'Utente deve poter visualizzare le immagini presenti all'interno della nota del diario criptato	Soddisfatto
RF-OB-70	L'Utente deve poter inserire del contenuto testuale all'interno di una nota del diario criptato	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-71	L'Utente deve poter inserire del contenuto multimediale all'interno di una nota del diario criptato	Soddisfatto
RF-OB-72	L'Utente deve poter inserire un'immagine all'interno di una nota del diario criptato	Soddisfatto
RF-OB-73	L'Utente deve poter inserire una traccia audio all'interno di una nota del diario criptato	Soddisfatto
RF-OB-74	L'Utente deve poter modificare il contenuto testuale presente all'interno di una nota del diario criptato	Soddisfatto
RF-OB-75	L'Utente deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario criptato	Soddisfatto
RF-OB-76	L'Utente deve poter rimuovere un'immagine presente all'interno di una nota del diario criptato	Soddisfatto
RF-OB-77	L'Utente deve poter rimuovere una traccia audio presente all'interno di una nota del diario criptato	Soddisfatto
RF-OB-78	L'Utente deve poter visualizzare la schermata del diario fittizio	Soddisfatto
RF-OB-79	L'Utente deve poter visualizzare la lista delle note presenti nel diario fittizio	Soddisfatto
RF-OB-80	L'Utente deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario fittizio	Soddisfatto
RF-OB-81	L'Utente deve poter ricaricare la sezione delle note del diario fittizio dopo la visualizzazione dell'errore di caricamento dei dati dal database	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-82	L'Utente deve poter eliminare una nota esistente dal diario fittizio	Soddisfatto
RF-OB-83	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario fittizio	Soddisfatto
RF-OB-84	L'Utente deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario fittizio quando il sistema ne richiede la conferma	Soddisfatto
RF-OB-85	L'Utente deve poter creare una nuova nota vuota all'interno del diario fittizio	Soddisfatto
RF-OB-86	L'Utente deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario fittizio	Soddisfatto
RF-OB-87	L'Utente deve poter visualizzare il contenuto testuale della nota del diario fittizio	Soddisfatto
RF-OB-88	L'Utente deve poter visualizzare il contenuto multimediale della nota del diario fittizio	Soddisfatto
RF-OB-89	L'Utente deve poter visualizzare il contenuto audio presente all'interno della nota del diario fittizio	Soddisfatto
RF-OB-90	L'Utente deve poter visualizzare le immagini presenti all'interno della nota del diario fittizio	Soddisfatto
RF-OB-91	L'Utente deve poter inserire del contenuto testuale all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-92	L'Utente deve poter inserire del contenuto multimediale all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-93	L'Utente deve poter inserire un'immagine all'interno di una nota del diario fittizio	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-94	L'Utente deve poter inserire una traccia audio all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-95	L'Utente deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-96	L'Utente deve poter rimuovere un'immagine presente all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-97	L'Utente deve poter rimuovere una traccia audio presente all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-98	L'Utente deve poter modificare il contenuto testuale presente all'interno di una nota del diario fittizio	Soddisfatto
RF-OB-99	L'Utente deve poter inviare un messaggio di SOS ai contatti fidati tramite un'apposita voce	Soddisfatto
RF-OB-100	Il sistema deve notificare l'Utente dell'assenza di connessione internet durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Soddisfatto
RF-OB-101	Il sistema deve notificare l'Utente della mancanza di contatti fidati configurati durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Soddisfatto
RF-OB-102	L'Utente deve poter avviare un segnale acustico di emergenza indipendente dalle impostazioni di volume del dispositivo	Soddisfatto
RF-OB-103	L'Utente deve poter inviare un messaggio al chatbot e ricevere una risposta generata dall'LLM in base al contesto scelto	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-105	Il sistema deve notificare l'Utente della mancanza di connessione internet durante la consultazione del chatbot	Soddisfatto
RF-OB-106	L'Utente deve poter ripetere l'invio del messaggio o scriverne uno nuovo dopo la notifica di assenza di connessione internet	Soddisfatto
RF-OB-107	L'Utente deve poter cambiare il contesto della chat a "Detective delle Relazioni"	Soddisfatto
RF-OB-108	L'Utente deve poter cambiare il contesto della chat a "Specchio Intelligente"	Soddisfatto
RF-OB-111	L'Utente deve poter creare una nuova sessione di chat con il chatbot	Soddisfatto
RF-OB-112	L'Utente deve poter eliminare una chat dalla cronologia delle chat	Soddisfatto
RF-OB-113	Il sistema deve richiedere conferma esplicita prima dell'eliminazione definitiva di una chat	Soddisfatto
RF-OB-114	L'Utente deve poter confermare, con un'apposita voce, l'eliminazione di una chat quando il sistema ne richiede la conferma	Soddisfatto
RF-OB-115	L'Utente deve poter visualizzare il contenuto di una chat selezionata, inclusa la lista dei messaggi scambiati	Soddisfatto
RF-OB-116	Il sistema deve notificare l'Utente di un errore nel caricamento del contenuto della chat	Soddisfatto
RF-OB-117	Il sistema deve visualizzare la lista dei messaggi scambiati tra l'Utente e il chatbot all'interno di una chat	Soddisfatto
RF-OB-118	Il sistema deve distinguere visivamente i messaggi inviati dall'Utente da quelli generati dal chatbot	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-119	Il sistema deve visualizzare i messaggi inviati dall'Utente con la formattazione grafica allineata a destra	Soddisfatto
RF-OB-120	Il sistema deve visualizzare i messaggi generati dal chatbot con la formattazione grafica allineata a sinistra	Soddisfatto
RF-OB-121	L'Utente deve poter visualizzare la cronologia delle chat con il chatbot	Soddisfatto
RF-OB-122	L'Utente deve poter selezionare e visualizzare una singola chat dalla cronologia	Soddisfatto
RF-OB-123	Il sistema deve mostrare il titolo identificativo di ogni chat presente nella cronologia	Soddisfatto
RF-OB-124	L'Utente deve poter visualizzare la lista delle community disponibili	Soddisfatto
RF-OB-125	Il sistema deve notificare l'Utente di un errore nel caricamento della lista delle community	Soddisfatto
RF-OB-126	Il sistema deve mostrare i dati riassuntivi di ogni singola community nella lista	Soddisfatto
RF-OB-127	Il sistema deve mostrare il nome di ogni singola community nella lista	Soddisfatto
RF-OB-128	Il sistema deve mostrare il link di ogni singola community nella lista	Soddisfatto
RF-OB-129	L'Utente deve poter aprire il link di una community per visitarne il sito web esterno	Soddisfatto
RF-OB-130	L'Utente deve poter visualizzare la lista delle leggi e normative disponibili	Soddisfatto
RF-OB-131	Il sistema deve mostrare il titolo di ogni singola legge nella lista	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-132	L'Utente deve poter visualizzare i dettagli completi di una legge selezionata dalla lista	Soddisfatto
RF-OB-133	Il sistema deve notificare l'Utente di un errore nel caricamento dei dati di una legge	Soddisfatto
RF-OB-134	L'Utente deve poter visualizzare la lista dei contatti e degli enti d'emergenza	Soddisfatto
RF-OB-135	Il sistema deve mostrare i dati riassuntivi di ogni singolo contatto d'emergenza nella lista	Soddisfatto
RF-OB-136	Il sistema deve mostrare il nome identificativo di ogni contatto d'emergenza	Soddisfatto
RF-OB-137	Il sistema deve mostrare il numero di telefono associato ad ogni contatto d'emergenza	Soddisfatto
RF-OB-139	L'Utente deve poter visualizzare la lista del materiale informativo disponibile	Soddisfatto
RF-OB-140	Il sistema deve notificare l'Utente di un errore nel caricamento della lista del materiale informativo	Soddisfatto
RF-OB-141	Il sistema deve mostrare il titolo e il formato di ogni singolo materiale informativo nella lista	Soddisfatto
RF-OB-142	L'Utente deve poter visualizzare il contenuto di un materiale informativo selezionato dalla lista	Soddisfatto
RF-OB-143	Il sistema deve notificare l'Utente di un errore nel recupero dei dati del materiale informativo	Soddisfatto
RF-OB-144	L'Utente deve poter visualizzare la mappa interattiva con i marker dei luoghi sicuri nelle vicinanze	Soddisfatto
RF-OB-145	L'Utente deve poter aprire un luogo sicuro nella mappa esterna Google Maps	Soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-146	Il sistema deve notificare l'Utente di un errore nell'apertura dell'applicazione Google Maps e mantenere la visualizzazione della mappa interna	Soddisfatto

Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori

4.2. Requisiti Funzionali desiderabili

Codice	Descrizione	Stato
RF-DE-1	Il sistema deve mostrare all'Utente una spiegazione della differenza tra "per me" e "per altri"	Non Soddisfatto
RF-DE-2	Il sistema mostra all'Utente una breve descrizione del funzionamento del Dead Man's Switch	Non Soddisfatto
RF-DE-3	Il sistema deve permettere all'Utente di inserire più contenuti multimediali con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-4	Il sistema deve permettere all'Utente di inserire più immagini con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-5	Il sistema deve permettere all'Utente di inserire più tracce audio con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-6	Il sistema deve permettere all'Utente di inserire più contenuti multimediali con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-7	Il sistema deve permettere all'Utente di inserire più immagini con una singola azione nel diario fittizio	Non Soddisfatto



Tabella 3: Tabella Requisiti Funzionali desiderabili (continua)

Codice	Descrizione	Stato
RF-DE-8	Il sistema deve permettere all'Utente di inserire più tracce audio con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-9	Il sistema deve permettere all'Utente di riprodurre le tracce audio inserite nel diario criptato	Soddisfatto
RF-DE-10	Il sistema deve permettere all'Utente di riprodurre le tracce audio inserite nel diario fittizio	Soddisfatto
RF-DE-11	Il sistema deve indicare all'Utente che il messaggio del chatbot è in fase di caricamento	Non Soddisfatto
RF-DE-12	Il sistema deve utilizzare i contenuti del diario criptato per personalizzare le risposte del chatbot	Non Soddisfatto
RF-DE-13	Il sistema deve mostrare all'Utente una spiegazione del tipo di contesto a cui può passare (Detective delle relazioni)	Soddisfatto
RF-DE-14	Il sistema deve mostrare all'Utente una spiegazione del tipo di contesto a cui può passare (Specchio intelligente)	Soddisfatto
RF-DE-15	L'Utente deve poter comporre automaticamente il numero di un contatto d'emergenza tramite l'apposita voce	Non Soddisfatto

Tabella 3: Tabella Requisiti Funzionali Desiderabili

4.3. Requisiti Funzionali opzionali



Codice	Descrizione	Riferimento
RF-OP-1	L'Utente deve poter impostare il timer di prima inattività (per la funzionalità di Dead Man's Switch)	Soddisfatto
RF-OP-2	L'Utente deve poter impostare il timer di seconda inattività (per la funzionalità di Dead Man's Switch)	Soddisfatto
RF-OP-3	Il sistema deve inviare un SMS ai numeri di telefono dei contatti fidati all'invio del segnale SOS	Non Soddisfatto
RF-OP-4	Il sistema deve indicare all'Utente se le risorse sono autorevoli e verificate	Non Soddisfatto
RF-OP-5	Il sistema deve mettere a disposizione leggi estere	Non Soddisfatto
RF-OP-6	Il sistema deve mostrare leggi aggiornate, giuridicamente valide e complete	Non Soddisfatto
RF-OP-7	Il sistema deve indicare all'Utente se i luoghi sicuri visualizzati nella mappa sono autorevoli e verificati	Non Soddisfatto
RF-OP-8	L'Utente deve poter selezionare il profilo di utilizzo ("Per me" o "Per altri")	Non soddisfatto
RF-OP-9	Il sistema deve bloccare temporaneamente l'accesso al diario in seguito al superamento del limite massimo di tentativi errati di inserimento password	Non soddisfatto
RF-OP-10	L'Utente deve poter interrompere la generazione della risposta dell'LLM durante la consultazione del chatbot	Non soddisfatto
RF-OP-11	Il sistema deve notificare l'Utente di un errore durante il cambio di contesto del chatbot e mantenere il contesto precedente	Non soddisfatto
RF-OP-12	L'Utente deve poter ritentare il cambio di contesto del chatbot dopo la notifica di errore	Non soddisfatto

Tabella 4: Tabella dello stato dei requisiti funzionali opzionali

